

FUJINET FOR THE INTELLIVISION

PROGRAMMER'S GUIDE

IntyBASIC Edition

Covers the cartridge mailbox, the Network and Fuji devices, AppKeys, and three complete networked games with their servers.

FujiNet is a community project. Intellivision is a registered trademark of its respective owner; this guide is not affiliated with or endorsed by Mattel. Illustration style after the 1983 Computer Module Owner's Guide.

FUJINET PROGRAMMER'S GUIDE

for the Mattel Intellivision — IntyBASIC Edition

Every command, register cell, byte offset and wire format in this guide was verified against the sources it documents: the cartridge firmware in `fujinet-firmware/pico/intellivision/`, the ESP32-S3 command handlers in `lib/device/rs232/` and `lib/bus/rs232/`, the CONFIG client in `fujinet-config/intv/`, the three game clients, and the game servers in `fujinet-game-system/`.

The example programs `hello.bas` and `netcat.bas` compile with IntyBASIC v1.4.2 and assemble with `as1600` without errors.

The network is as easy as PEEK and POKE.

TABLE OF CONTENTS

CHAPTER 1	Introduction	1
CHAPTER 2	The Cartridge Mailbox	3
CHAPTER 3	Your First Transaction	7
CHAPTER 4	The Network Device	11
CHAPTER 5	The Fuji Device	18
CHAPTER 6	The Clock Device	28
CHAPTER 7	The fujinet.bas Library	30
CHAPTER 8	Game One: 5 Card Stud	33
CHAPTER 9	Game Two: Battleship	37
CHAPTER 10	Game Three: Fujitzee	40
CHAPTER 11	The Lobby	43
APPENDIX A	Error Codes	44
APPENDIX B	Quick Reference	47
APPENDIX C	Program Listings	51
APPENDIX D	Netcat	111

CHAPTER 1

INTRODUCTION

What a FujiNet is, what is inside the cartridge, and what your IntyBASIC program can do with it.

FujiNet brings the Internet to the Intellivision the same way it came to the Atari, the ADAM, the Apple II and a dozen other classic machines: as a peripheral that does the hard part — WiFi, TCP/IP, TLS, HTTP, JSON — on a modern processor, and hands your 8-bit (well, 16-bit!) program a simple, byte-oriented command interface.

On most FujiNet platforms that interface is the machine's own peripheral bus. The Intellivision has no such bus for cartridges to speak on, so the Intellivision FujiNet takes a different road: **the cartridge itself is the peripheral**. Inside the cartridge are two computers:



The RP2040 (or RP2350) runs the cartridge: it serves the ROM your program executes from, and it maps a small window of cartridge RAM at addresses \$9C00–\$9F3F that both sides can read and write. That window is the **mailbox**. Your program fills in a request — a device number, a command number, parameters, maybe a payload — and rings a bell. The RP2040 notices, re-encodes the request as a FujiBus packet, and sends it over an internal USB serial link to the ESP32-S3, which is running the same fujinet-firmware that powers every other FujiNet. The reply comes back the same way and lands in the mailbox, where your program PEEKs it out.

The beautiful consequence: **everything is PEEK and POKE**. No handlers, no interrupts, no bus timing. If you can poke a byte and wait for another byte to change, you can talk to the Internet.

WHAT YOU CAN DO

- Open network connections by name — `N:HTTPS://server/path`, `N:TCP://host:port/`, TELNET, UDP, TNFS and more — then read, write and poll them (Chapter 4).
- Ask the Fuji control device to scan WiFi, manage host and device slots, browse server directories, mount disk images — and on the Intellivision, **boot another ROM over the network** (Chapter 5).
- Store small records — a player name, a saved game, a server URL — on the FujiNet itself with AppKeys, so they survive power-off (Chapter 5).
- Read the real-world clock in seven formats (Chapter 6).
- Encode and decode Base64, compute MD5/SHA hashes, even build QR codes, using the FujiNet as a coprocessor (Chapter 5).

HOW THIS GUIDE IS ARRANGED

Chapters 2 and 3 teach the mailbox itself — the memory map, the handshake, and a first working program. Chapters 4 through 6 are the command reference, one device at a time. Chapter 7 documents `fujinet.bas`, the drop-in IntyBASIC library used by every program in this guide. Chapters 8 through 10 dissect three complete, playable network games — **5 Card Stud**, **Battleship** and **Fujitzee** — client and server both, down to the byte. Chapter 11 covers the Lobby that ties them together. The appendices hold the error tables, a quick reference, the full program listings (with numbered call-outs ¹ referenced from the chapters), and a network terminal you can type in.

NOTE: Numbers written like `$9C00` are hexadecimal; plain numbers are decimal. Mailbox cells are named `FN_` the way `fujinet.bas` names them. Controller keys look like `ENTER`.

CHAPTER 2

THE CARTRIDGE MAILBOX

One page of shared RAM is the whole hardware interface. Learn it and you know everything the cartridge can do.

The mailbox is a window of cartridge RAM the RP2040 maps into the Intellivision's address space at \$9C00–\$9F3F — 832 locations. Each location holds one byte in the low 8 bits of a 16-bit word (writes from the Intellivision side are hardware-truncated to 8 bits), so always mask what you read: `PEEK(addr) AND 255`.

It sits at the **top** of the \$8000–\$9FFF region deliberately: on a JLP-enabled cartridge that whole region is JLP RAM, and parking the mailbox at the top leaves \$8000–\$9BFF — 7168 words, 87% of the window — free for JLP while the mailbox keeps its 832.

THE MEMORY MAP

ADDRESS	NAME	WHO WRITES	MEANING
\$9C00	MAGIC0	RP2040	ASCII F (70) — mailbox present
\$9C01	MAGIC1	RP2040	ASCII N (78)
\$9C02	PROTO_VER	RP2040	protocol version, currently 1
\$9C03	SEQ	Inty	bump to start a transaction (wraps, skip 0)
\$9C04	ACKSEQ	RP2040	set equal to SEQ when the reply is ready
\$9C05	DEVICE	Inty	device id (\$70 Fuji, \$71 N1: ...)
\$9C06	CMD	Inty	command id
\$9C07	NPARAM	Inty	parameter count, 0–8
\$9C08/09	TXLEN	Inty	payload length, little-endian
\$9C0A	STATUS	RP2040	diagnostic only: 0 idle, 1 busy, 2 ok, 3 err
\$9C0B	ERR	RP2040	link result of last transaction (Appendix A)

ADDRESS	NAME	WHO WRITES	MEANING
\$9C0C/0D	RXLEN	RP2040	reply payload length, little-endian
\$9C0E	REPLY_CMD	RP2040	\$06 ACK or \$15 NAK
\$9C0F	LINK	RP2040	1 if the ESP32-S3 is up on the USB link
\$9C10-17	PARAM_SIZE	Inty	size of each parameter: 1, 2 or 4 bytes
\$9C18	BOOT_STATE	RP2040	network-boot progress (see below)
\$9C19	BOOT_PCT	RP2040	network-boot progress, 0–100
\$9C1A	BOOT_ERR	RP2040	reason, when BOOT_STATE = failed
\$9C1B	DOORBELL	Inty	write \$B5 → RP2040 reboots to BOOTSEL
\$9C20-3F	PARAM_VAL	Inty	8 slots × 4 bytes, little-endian
\$9C40-	TX	Inty	request payload, up to 256 bytes
\$9D40-	RX	RP2040	reply payload, up to 512 bytes

THE HANDSHAKE

A transaction is four steps, and the order of the last one is the whole protocol:

1. Fill in **DEVICE**, **CMD**, **NPARAM**, the parameter table, **TXLEN** and the TX payload — everything **except** **SEQ**.
2. Read **ACKSEQ**, add 1 (wrap past 255, and skip 0), and write that to **SEQ**. **Last**. The RP2040 polls for **SEQ** ≠ **ACKSEQ** writing **SEQ** is what submits the request.
3. Wait for **ACKSEQ** to become equal to the **SEQ** you wrote. Give up after a generous timeout — the library allows 900 frames (15 seconds).
4. Check **REPLY_CMD**: **\$06** is ACK — the reply payload, if any, is at **RX** with its length in **RXLEN**. Anything else, look at **ERR** for the link-level reason (Appendix A).

WATCH OUT: Always derive the new SEQ from the ACKSEQ the RP2040 itself publishes, never from a variable of your own. Pressing Reset restarts your program and zeroes your variables — but not the RP2040. A locally-counted SEQ would recompute the same value it already used, the RP2040 would see nothing new, and your first transaction after every reset would hang. This is call-out 2 in Listing 1.

The interlock is a **sequence number**, not a busy flag: re-poking the same SEQ while a transaction is in flight is a no-op, so a nervous retry can never make the RP2040 run a command twice. STATUS at \$9C0A is a debugging aid only — poll ACKSEQ, not STATUS.

PARAMETERS AND PAYLOADS

Commands take up to eight small numeric parameters — the things other FujiNet platforms pass in “aux” bytes. Parameter *i* occupies one word at `PARAM_SIZE + i` giving its size (1, 2 or 4 bytes) and up to four words at `PARAM_VAL + i×4` holding its value, little-endian, one byte per word. Bulk data — a URL, a filename, a WiFi password — goes in the TX window instead, as consecutive bytes, with the count in TXLEN.

Replies land in the RX window, length in RXLEN, and **stay there until the next transaction overwrites them**. The IntyBASIC discipline that follows from this is the most important habit in this guide: **read replies in place**. IntyBASIC gives you 228 eight-bit variables; a / state reply from a game server is 400+ bytes. Never copy a reply into variables — PEEK what you need straight out of `FN_RX`, and copy only the few bytes that must survive the next transaction into cartridge scratch RAM (\$9000–\$9BFF is free on a FujiNet cartridge when JLP is off).

TIMEOUTS AND THE LINK BYTE

The RP2040 gives an ordinary command 5 seconds on the USB link, and `MOUNT_IMAGE 60` (a ROM can take a while to fetch). It also waits up to 3 seconds for the ESP32 to enumerate before the very first command — the Intellivision boots much faster than the ESP32 does. `LINK` at \$9C0F shows the live state of the USB link; a program can show “waiting for FujiNet” instead of a mysterious timeout.

BOOTING A ROM THROUGH THE MAILBOX

`MOUNT_IMAGE` on the Fuji device does something on this platform it does nowhere else: the ESP32 pushes the selected ROM — plus its `.cfg` sibling, if one exists — back over the same USB link, the RP2040 decodes it straight into cartridge ROM space, remaps, and resets your console into the new program. While that happens the ordinary handshake is still outstanding, so progress is published out-of-band in three cells your wait loop can PEEK: `BOOT_STATE` (0 idle, 1 opening, 2 transferring, 3 mapping, \$80 failed), `BOOT_PCT` (0–100), and `BOOT_ERR` (Appendix A). Flat `.bin`, self-describing Intellicart `.rom`, and `.bin+.cfg` (with JLP variables) all load; a mapping that would collide with the mailbox itself is refused.

DECLARING THE WINDOW IN INTYBASIC

ASM MEMATTR \$8000, \$9BFF, "+RWN"

Tells the assembler that the cartridge RAM below the mailbox is readable and writable, so POKE reaches your scratch buffers.

Declare `$8000–$9BFF` — and stop there. On real hardware the RP2040 maps the whole window regardless of what your `.cfg` says, but `jzIntv's --fujinet` emulation registers its own handler for `$9C00–$9FFF`, and a cartridge that claims those addresses as plain RAM **shadows the emulated FujiNet** — the mailbox never comes up under emulation even though the same ROM works on the real cartridge. Call-out 8 in Listing 1.

CHAPTER 3

YOUR FIRST TRANSACTION

A complete program in under fifty lines, and the IntyBASIC survival rules that keep the rest of them working.

Here is the whole idea of the library in three procedures. First, wait for the mailbox to introduce itself:

GOSUB fn_wait_mailbox

Polls for the two magic bytes — “F”, “N” — at \$9C00 for up to three seconds. Sets fn_ok to 1 when the cartridge answers, 0 when it does not (this is not a FujiNet cartridge, or the mailbox is shadowed — see Chapter 2).

Then describe a transaction and run it. Ask the Fuji device (\$70) for its WiFi status (command \$FA):

```
mb_dev = $70 : mb_cmd = $FA : mb_nparam = 0
```

```
#fn_txlen = 0
```

GOSUB fn_transact

Stages the request, bumps SEQ, blocks until ACKSEQ answers or 900 frames pass. On return fn_ok is 1 and the one-byte reply is waiting at FN_RX; a 3 means the WiFi is up.

```
IF (PEEK(FN_RX) AND 255) = 3 THEN PRINT AT 40, "CONNECTED"
```

Reads the reply in place. The AND 255 mask matters — mailbox words carry a byte in their low half and whatever in their high half.

That is genuinely all there is. `hello.bas` below is the complete program; type it in, or find it with the listings.

HELLO.BAS — ONE COMPLETE TRANSACTION

```
' hello.bas -- the smallest possible FujiNet program: ask the Fuji device
' for its WiFi status and report it. Demonstrates one complete mailbox
' transaction with no payload and no parameters.
'
```

```

' Build: intybasic hello.bas hello.asm && as1600 -o hello -l hello.lst hello.asm
GOTO main

INCLUDE "fujinet.bas"

CONST COL_WHITE = 7

main:
MODE 0, 0, 0, 0, 0, 0 : WAIT
CLS
PRINT AT 0 COLOR COL_WHITE, "FUJINET HELLO"
PRINT AT 20, "WAITING FOR MAILBOX"

GOSUB fn_wait_mailbox
IF fn_ok = 0 THEN
    PRINT AT 40, "NO MAILBOX - IS THIS"
    PRINT AT 60, "A FUJINET CARTRIDGE?"
    GOTO halt
END IF

' One transaction: device $70 (Fuji), command $FA (GET WIFI STATUS),
' no parameters, no payload. The one-byte reply lands in FN_RX.
mb_dev = $70
mb_cmd = $FA
mb_nparam = 0
#fn_txlen = 0
GOSUB fn_transact

IF fn_ok = 0 THEN
    PRINT AT 40, "TRANSACTION FAILED "
    PRINT AT 60, "FN ERR = "
    PRINT AT 69, "<2>#mb_err"
    GOTO halt
END IF

IF (PEEK(FN_RX) AND 255) = 3 THEN
    PRINT AT 40, "WIFI IS CONNECTED "
ELSE
    PRINT AT 40, "WIFI NOT CONNECTED "
END IF

```

```
halt:
    WAIT
    GOTO halt
```

SURVIVAL RULES FOR INTYBASIC NETWORK CODE

Every rule below was paid for in debugging time on the three games. They are worth a page of anyone's attention.

JUMP OVER YOUR INCLUDES

An `INCLUDE` pastes its text where it stands, and falling into a `PROCEDURE` or `DATA` block by straight-line execution corrupts the return stack. Make line one of your program `GOTO main` (or `boot_start`), and put every `INCLUDE` between it and the label.

RESPECT THE MEMORY MAP

IntyBASIC compiles into `$5000–$6FFF` and keeps going if you outgrow it — straight into `$7000`, which is **not** in the manual's list of ranges usable on modern cartridge PCBs. Both Battleship and Fujitzee overflowed; the fix is an explicit `ASM ORG $D000` placed so the overflow lands in documented-safe `$C100–$FFFF` territory. Fujitzee's boot loader genuinely hung on real hardware from a 20-word spill that jzIntv forgave. Safe ranges: `$2000–$2FFF`, `$5000–$6FFF`, `$A000–$BFFF`, `$C100–$FFFF`.

PARENTHE SIZE (PEEK(X) AND 255)

IntyBASIC's `=` binds **tighter** than `AND` — the same trap as C's `a & b == c`. An unparenthesized `DEF FN v = PEEK(x) AND 255` substituted into `IF v = 0` compiles as `PEEK(x) AND (255 = 0)`, which is always zero. Fujitzee's turn logic silently never ran until the generated assembly was read. Write `(PEEK(x) AND 255)`, always.

MASK INTO 16-BIT VARIABLES

IntyBASIC v1.4.2 silently drops the `AND 255` when the destination of `var = PEEK(...)` `AND 255` is an 8-bit variable — no `ANDI` is emitted at all, and high-byte garbage survives. `#`-prefixed 16-bit destinations compile correctly. The library's `#mb_err` and `#net_err` are 16-bit for exactly this reason.

NEST IFS INSTEAD OF CHAINING AND

Two equality comparisons joined by **AND**, where a side comes from a **DEF FN** containing its own **AND** 255, can fuse into code that is always false. Write nested single-condition **IFs**. (Fujitzee again; confirmed in the compiled listing.)

NUMBER FIELDS: <.N>, NOT <N>

PRINT <3> (zero-padded) can print garbage in its leading digit — its runtime entry point never initializes the pad register. **PRINT** <.3> (space-padded) initializes it. Use <.n> for any value that can need fewer digits than the field.

WAIT FOR KEY RELEASE

CONT1.KEY reads 12 when nothing is pressed. When a key opens a menu, wait for 12 before accepting input, or the same press instantly triggers the menu's own handler. And check **CONT1.B1/B2 before** **CONT1.BUTTON** — **BUTTON** is a mask that is non-zero for all three action buttons.

CHAPTER 4

THE NETWORK DEVICE

Eight channels to anywhere: URLs in, bytes out. Devices \$71 through \$78.

The Network device turns a URL into a byte channel. Eight independent units — N1: through N8:, device ids \$71 through \$78 — can each hold one open connection. Everything in this guide uses N1:.

A connection is named by a **devicespec** — an ASCII string staged in the TX window when you OPEN:

N:PROTOCOL://HOST[:PORT]/PATH[?QUERY]

The protocols in current firmware: HTTP, HTTPS, TCP, UDP, TELNET, TNFS, FTP, SMB, NFS, SSH, plus a few specials. HTTPS certificates, redirects, name resolution — the ESP32 handles all of it, so you never see it.

THE LIFE OF A CONNECTION

1. OPEN with an access mode and translation mode.
2. STATUS until data is waiting (or the connection reports an error).
3. READ the bytes; WRITE your own.
4. CLOSE.

Two subtleties both games hit, worth learning before writing code. First: for an HTTP devicespec, **the actual request happens at the first STATUS**, not at OPEN. OPEN parses and stores; the first STATUS performs the transaction and its result code lands in the status reply's error byte. Second: STATUS reports bytes **available so far** — a response still arriving over WiFi grows between polls. The library's `api_call` polls until two consecutive STATUS reads agree (call-out 6, Listing 1) before trusting the count. An HTTP error page is still a readable body, so the byte count alone cannot tell success from a 400 — check the error byte, which reads 1 (SUCCESS) on a good channel (call-out 5).

COMMAND REFERENCE

OPEN

DEV \$71-78

CMD \$4F '0'

PARAM	SIZE	VALUE
0	1	access mode — table below
1	1	translation — 0 none, 1 CR, 2 LF, 3 CR LF, 4 PETSCII

PAYLOAD devicespec, ASCII, no terminator needed — TXLEN is the length

REPLY ACK on success; NAK if the protocol could not connect

Opens the channel. Re-opening an open unit closes it first. Translation rewrites line endings between the wire and your program; the games use 0 and handle bytes raw.

MODE	MEANING	PROTOCOLS
4	READ / HTTP GET	all
5	HTTP DELETE	HTTP(S)
6	DIRECTORY	filesystem protocols
8	WRITE / HTTP PUT	all
9	APPEND / HTTP DELETE with headers	file / HTTP(S)
12	READ-WRITE / HTTP GET with header access	sockets, HTTP(S)
13	HTTP POST	HTTP(S)
14	HTTP PUT with header access	HTTP(S)

NOTE: The games open with mode 12 — on HTTP it is a GET that also permits header interaction through the M command, and on TCP it is plain read-write. Mode 4 works for fire-and-forget GETs.

CLOSE

DEV \$71-78

CMD \$43 'C'

Tears the connection down. Always close — even after an error — so the unit is free for the next OPEN. Closing writes a fresh (empty) reply length, so capture **RXLEN** from a READ before you CLOSE (the library does: call-out ⁴ territory in Listing 1).

READ

DEV \$71-78

CMD \$52 'R'

PARAM	SIZE	VALUE
0	2	byte count to read

REPLY the bytes, in RX; RXLEN says how many arrived

Never ask for more than STATUS said was available — a short channel pads with NULs and flags an error. Ask STATUS first, clamp, then READ.

WRITE

DEV \$71-78

CMD \$57 'W'

PARAM	SIZE	VALUE
0	2	byte count to send

PAYLOAD the bytes

Sends payload bytes out the channel. The count rides twice — as parameter 0 and as TXLEN — and both must agree.

STATUS

DEV \$71-78

CMD \$53 'S'

CHAPTER 4

PARAM	SIZE	VALUE
0	1	0
1	1	request type — 0 for channel status

REPLY 4 bytes: available-low, available-high, connected, error

The heartbeat of every network program. **available** is a 16-bit little-endian count of bytes ready to READ; **connected** is 1 while the far end holds the connection; **error** is 1 for success or a code from Appendix A. With no channel open, request types 1–4 return instead the adapter's IP address, netmask, gateway and DNS, 4 bytes each.

CHANNEL MODE

DEV \$71-78

CMD \$FC

PARAM	SIZE	VALUE
0	1	0
1	1	0 protocol (raw), 1 JSON, 2 SGML

Switches the open channel between raw bytes and a parsed view. In JSON mode, **PARSE** ingests the body and **QUERY** extracts values by path — see the worked sequence below.

PARSE

DEV \$71-78

CMD \$50 'P'

JSON/SGML mode: reads the entire response body off the protocol and parses it. Do this once, after **STATUS** says the body has arrived.

QUERY

DEV \$71-78

CMD \$51 'Q'

PAYLOAD a query path, e.g. `/0/name` — 256 bytes, NUL-padded

Selects a value out of the parsed document. After **QUERY**, **STATUS** reports the extracted value's length and **READ** returns it as text.

SET CHANNEL MODE (HTTP)

DEV \$71-78

CMD \$4D 'M'

PARAM	SIZE	VALUE
0	1	0
1	1	0 body, 1 collect headers, 2 get headers, 3 set headers, 4 set POST data

HTTP only: redirects READ/WRITE at the header machinery instead of the body — set request headers before the transaction, read response headers after.

TRANSLATION

DEV \$71-78

CMD \$54 'T'

PARAM	SIZE	VALUE
0	1	0
1	1	as OPEN parameter 1

Changes line-ending translation on an open channel.

SET EOL

DEV \$71-78

CMD \$4C 'L'

PARAM	SIZE	VALUE
0	1	first EOL byte, or 0 to restore the default
1	1	optional second EOL byte

Overrides what the translation layer treats as your machine's native line ending.

SEEK / TELL

DEV \$71-78

CMD \$25 / \$26

PARAM	SIZE	VALUE
0	4	SEEK only: absolute byte offset

REPLY TELL: 4 bytes, little-endian, current offset

Repositions within a seekable resource. On HTTP this issues a Range request — valid for GET reads in body mode.

GETCWD / CHDIR

DEV \$71-78

CMD \$30 / \$2C

PAYLOAD CHDIR: the new prefix/path

REPLY GETCWD: 256 bytes, the prefix

Maintains a working-directory prefix for filesystem protocols, so later devicespecs can be relative.

USERNAME / PASSWORD

DEV \$71-78

CMD \$FD / \$FE

PAYLOAD the credential, up to 256 bytes

Stages login credentials before OPEN, for protocols that need them (SMB, FTP, SSH...).

TCP: ACCEPT / CLOSE CLIENT

DEV \$71-78

CMD \$41 / \$63

A TCP devicespec with no host — **N:TCP://:6502/** — listens. **\$41** accepts a waiting caller onto the channel; **\$63** hangs up on the caller and resumes listening. Your Intellivision can be the server.

UDP: SET DESTINATION

DEV \$71-78

CMD \$44 'D'

PAYLOAD host:port for subsequent datagrams

UDP is connectionless; this sets where WRITEs go. (The companion \$72 GET-REMOTE query exists only in the PC build of the firmware, not on the ESP32.)

FILE OPS

DEV \$71-78

CMD \$20/\$21/\$23/\$24/\$2A/\$2B

PAYLOAD a devicespec (RENAME: old,new)

RENAME \$20, DELETE \$21, LOCK \$23, UNLOCK \$24, MKDIR \$2A, RMDIR \$2B — for protocols with a filesystem behind them (TNFS, SMB, FTP...).

A JSON ROUND TRIP

The game servers in this guide sidestep JSON with ?bin=1 binary replies (Chapter 8), but the parser is there when you want it:

```
' open, then: switch the channel to JSON
mb_cmd = $FC : mb_nparam = 2
pm_i = 0 : pm_size = 1 : #pm_val = 0 : GOSUB fn_param
pm_i = 1 : pm_size = 1 : #pm_val = 1 : GOSUB fn_param
GOSUB fn_transact
```

Channel mode 1 = JSON.

```
mb_cmd = $50 : mb_nparam = 0 : #fn_txlen = 0 : GOSUB fn_transact
```

PARSE — the FujiNet reads and digests the whole body.

```
' stage "/O/title" in FN_TX, NUL-padded, then:
```

```
mb_cmd = $51 : GOSUB fn_transact
```

QUERY selects one value by path.

```
GOSUB net_status : #net_readlen = #net_avail : GOSUB net_read
```

STATUS now reports the extracted value's length; READ fetches it as ASCII, ready to draw with draw_field.

THE FUJI DEVICE

Device \$70: the FujiNet's own control panel — WiFi, hosts, directories, disk images, AppKeys, and a bag of coprocessor tricks.

Everything that is **about the FujiNet itself** — rather than about one network connection — lives on device \$70. CONFIG is nothing but a client of this device, and everything CONFIG does, your program can do.

WIFI

GET WIFI STATUS

DEV \$70

CMD \$FA

REPLY 1 byte: 3 = connected; other values are the ESP32 WL codes (0 idle, 1 no SSID, 4 connect failed, 5 connection lost, 6 disconnected)

The first thing `hello.bas` asks. Poll it after SET SSID — the join takes a few seconds.

SCAN NETWORKS

DEV \$70

CMD \$FD

REPLY 1 byte: number of networks found

Blocks while the ESP32 scans. Follow with GET SCAN RESULT for each index.

GET SCAN RESULT

DEV \$70

CMD \$FC

PARAM	SIZE	VALUE
0	1	network index, 0-based

REPLY 34 bytes: SSID (33, NUL-terminated) + RSSI (1, signed)

SET SSID

DEV \$70

CMD \$FB

PARAM	SIZE	VALUE
0	1	any value — at least one parameter must be present

PAYLOAD 97 bytes exactly: SSID (33) + password (64), NUL-padded

Joins and **saves** the network. The parameter is ignored but required. The 97-byte shape is not negotiable — see the payload rule below.

GET SSID

DEV \$70

CMD \$FE

REPLY 97 bytes: the stored SSID + password in the same shape

GET WIFI ENABLED

DEV \$70

CMD \$EA

REPLY 1 byte, 0/1

WATCH OUT: The exact-length payload rule. The ESP32 handler for a fixed-size payload fails the transaction if **fewer** bytes arrive than the structure it expects — SET SSID wants exactly 97, OPEN DIRECTORY and SET DEVICE FULLPATH want exactly 256. Sending a short, “obviously enough” payload reads back as a NAK. Pad with NULs to the documented size, every time.

HOSTS AND MOUNTS

Eight **host slots** name the file servers CONFIG browses — TNFS hosts, SMB shares. Eight **device slots** bind a file on some host to an emulated disk. On the Intellivision the disk story is what powers network booting.

READ HOST SLOTS

DEV \$70

CMD \$F4

REPLY 256 bytes: 8 hostnames × 32, NUL-padded

WRITE HOST SLOTS

DEV \$70

CMD \$F3

PAYLOAD the same 256-byte block, all 8 slots at once

There is no single-slot write — read, modify one, write all.

MOUNT HOST

DEV \$70

CMD \$F9

PARAM	SIZE	VALUE
0	1	host slot, 0–7

Connects to the named server. Required before directory or file operations on that slot.

SET HOST PREFIX / GET HOST PREFIX

DEV \$70

CMD \$E1 / \$E0

PARAM	SIZE	VALUE
0	1	host slot

PAYLOAD SET: the prefix string

REPLY GET: the prefix

A per-host working directory.

OPEN DIRECTORY

DEV \$70

CMD \$F7

PARAM	SIZE	VALUE
0	1	host slot

PAYLOAD 256 bytes exactly: path, NUL, wildcard filter (may be empty), NUL, then NUL padding

Opens a directory listing on a mounted host. The filter understands * and ? only.

READ DIR ENTRY

DEV \$70

CMD \$F6

PARAM	SIZE	VALUE
0	1	maximum reply length
1	1	flags: 0 plain, \$80 with details

REPLY plain: the filename, NUL-terminated — directories carry a trailing / end of directory is two \$7F bytes. With \$80: a 10-byte prefix (modified date 6, size 4 LE) plus flags and media type precede the name

CONFIG's file browser is this command in a loop.

CLOSE DIRECTORY

DEV \$70

CMD \$F5

GET / SET DIRECTORY POSITION

DEV \$70

CMD \$E5 / \$E4

PARAM	SIZE	VALUE
0	2	SET: absolute entry index

REPLY GET: 2 bytes, little-endian

Random access into the listing — how CONFIG pages backward without rereading from the top.

READ / WRITE DEVICE SLOTS

DEV \$70

CMD \$F2 / \$F1

PAYLOAD WRITE: 8 records of {host slot, mode, filename[36]} = 304 bytes

REPLY READ: the same 304-byte block

The table binding device slots to files.

SET DEVICE FULLPATH

DEV \$70

CMD \$E2

PARAM	SIZE	VALUE
0	1	device slot
1	1	host slot
2	1	mode: 1 read

PAYLOAD 256 bytes exactly: full path, NUL-padded

Points a device slot at a file. On the Intellivision this is also where the cartridge learns the filename it will derive a JLP flash save-file name from — CONFIG always sends it immediately before MOUNT IMAGE.

GET DEVICE FULLPATH

DEV \$70

CMD \$DA

PARAM	SIZE	VALUE
0	1	device slot

REPLY the path

MOUNT IMAGE

DEV \$70

CMD \$F8

PARAM	SIZE	VALUE
0	1	device slot
1	1	access flags: 1 read

On every FujiNet: mounts the image. On the Intellivision: **boots it** — the ESP32 pushes the ROM back through the cartridge as described in Chapter 2, and this transaction is allowed 60 seconds. Watch `B00T_PCT` for a progress bar while you wait.

UNMOUNT IMAGE

DEV \$70

CMD \$E9

PARAM	SIZE	VALUE
0	1	device slot

MOUNT ALL / NEW DISK / COPY FILE

DEV \$70

CMD \$D7 / \$E7 / \$D8

Housekeeping from the wider FujiNet family: mount every configured slot; create a blank image ({sectors u16, sector size u16, host slot, device slot, filename[256]} payload); copy between hosts (parameters source and destination slot, payload the copy spec).

CONFIG BOOT / SET BOOT MODE

DEV \$70

CMD \$D9 / \$D6

PARAM	SIZE	VALUE
0	1	flag / mode

Controls whether the FujiNet offers CONFIG at power-up. The Intellivision cartridge boots CONFIG from its own flash, so CONFIG BOOT has no effect here — and CONFIG deliberately never sends it.

APPKEYS — SMALL FACTS THAT SURVIVE POWER-OFF

An AppKey is up to 64 bytes stored on the FujiNet, addressed by a registered **creator id** (16-bit), an **app id** and a **key id**. The games use them for the player name and the Lobby handoff (Chapter 11). The protocol is open–operate–close:

OPEN APPKEY

DEV \$70

CMD \$DC

PAYLOAD 6 bytes: creator low, creator high, app, key, mode (0 read, 1 write, 2 read-256), reserved (0)

Selects which key the next read or write touches. Mode 2 selects 256-byte keys; plain mode 0 reads 64-byte keys. **All six bytes are required** — a 5-byte payload leaves the firmware waiting for the last byte and reads back as a timeout (call-out 7 neighborhood, Listing 1).

READ APPKEY

DEV \$70

CMD \$DD

REPLY 2-byte little-endian length, then the data

The length prefix is specific to this bus (an appkey read takes no length parameter, so the reply describes itself). RXLEN covers prefix + data. Call-out 7 in Listing 1 shows the client-side handling.

WRITE APPKEY

DEV \$70

CMD \$DE

PAYLOAD the value — length is TXLEN

CLOSE APPKEY

DEV \$70

CMD \$DB

NOTE: Creator 1 / app 1 is the FujiNet game system. Key 0 holds the shared player name every lobby game reads; keys 1, 3 and 5 are the 5 Card Stud, Fujitzee and Battleship server-handoff slots. New creator ids are registered on the fujinet-firmware wiki's AppKey page.

ADAPTER INFORMATION

GET ADAPTERCONFIG EXTENDED

DEV \$70

CMD \$C4

REPLY 240 bytes — table below. The older \$E8 form is the first 140 bytes only, without the printable strings
One call answers “what is my IP” for an About screen — CONFIG’s info page is this struct drawn on screen.

OFFSET	SIZE	FIELD	NOTES
0	33	ssid	NUL-terminated
33	64	hostname	
97	4	localIP	binary, one octet per byte
101	4	gateway	
105	4	netmask	
109	4	dnsIP	
113	6	macAddress	binary
119	6	bssid	binary
125	15	fn_version	printable
140	16	sLocalIP	dotted-quad string
156	16	sGateway	
172	16	sNetmask	
188	16	sDnsIP	
204	18	sMacAddress	colon string
222	18	sBssid	

STATUS

DEV \$70

CMD \$53 'S'

PARAM	SIZE	VALUE
0	1	1 = mount-time report

REPLY one timestamp per disk device — 0 means unmounted

RESET / DEVICE READY / GENERATE GUID

DEV \$70

CMD \$FF / \$00 / \$BB

RESET reboots the ESP32 (the mailbox link drops and comes back). DEVICE READY answers 512 bytes of A — a link self-test. GENERATE GUID returns a fresh unique id, handy for session tokens.

THE COPROCESSOR BAG

Three little engines share a buffer-in / compute / length / buffer-out shape. Feed input in chunks, compute, ask the length, then drain.

BASE64 ENCODE

DEV \$70

CMD \$D0/\$CF/\$CE/\$CD

PARAM	SIZE	VALUE
0	2	INPUT and OUTPUT: byte count

PAYLOAD INPUT: the bytes

REPLY LENGTH: 4 bytes; OUTPUT: the requested bytes, consumed as read

INPUT \$D0 accumulates, COMPUTE \$CF transforms the buffer in place, LENGTH \$CE, OUTPUT \$CD. Decode is the mirror set \$CC–\$C9.

HASH

DEV \$70

CMD \$C8/\$C7/\$C6/\$C5/\$C2

PARAM	SIZE	VALUE
0	2	INPUT: byte count. COMPUTE: algorithm — 0 MD5, 1 SHA-1, 2 SHA-256, 3 SHA-512. LENGTH/OUTPUT: 1 = hex text, 0 = binary

PAYLOAD INPUT: the bytes

REPLY LENGTH: 1 byte; OUTPUT: the digest

\$C3 computes without clearing the input, for incremental use; CLEAR \$C2 abandons it. A SHA-256 of a game state in one transaction — try that on a CP-1610.

QR CODE

DEV \$70

CMD \$BC/\$BD/\$BE/\$BF

PARAM	SIZE	VALUE
0	2	INPUT: byte count. ENCODE: version. LENGTH/OUTPUT: output mode
1	1	ENCODE: error-correction level
2	1	ENCODE: shorten flag

PAYLOAD INPUT: the text

REPLY LENGTH: 4 bytes; OUTPUT: the module bitmap

Render the output on the colored-squares screen (the kernel in Chapter 9 draws 40x24 pixels) and you have a scannable screen.

THE CLOCK DEVICE

Device \$45: real time, seven ways, time-zone aware.

The FujiNet keeps real time via NTP. Device \$45 serves it in every format the wider FujiNet world has ever asked for. All GET commands accept an optional parameter 0: a value of 1 selects the **alternate** time zone set by \$74, letting a program show a second clock without touching the saved configuration.

CMD	NAME	REPLY
\$93	GET TIME	6 bytes: day, month, year-2000, hour, minute, second
\$9A	GET TZ TIME	the same 6 bytes, always in the alternate zone
\$47 'G'	GET SIMPLE	7 bytes: century, year, month, day, hour, minute, second
\$4D 'M'	GET SIMPLE + HUNDREDTHS	8 bytes: simple + hundredths (0–99)
\$50 'P'	GET PRODOS	4 bytes, ProDOS packed date/time
\$53 'S'	GET SOS	NUL-terminated Apple SOS string
\$49 'I'	GET ISO LOCAL	NUL-terminated ISO-8601 string, local
\$5A 'Z'	GET ISO UTC	NUL-terminated ISO-8601 string, UTC
\$99	SET TZ	— payload: POSIX TZ string, saved to config
\$74 't' / \$54 'T'	SET ALT TZ	— payload: POSIX TZ string, this session only
\$4C 'L'	GET TZ LENGTH	1 byte: length of the saved TZ string
\$47	GET TZ	the saved TZ string, NUL-terminated

```
mb_dev = $45 : mb_cmd = $93 : mb_nparam = 0  
#fn_txlen = 0 : GOSUB fn_transact  
day = PEEK(FN_RX) AND 255
```

Six PEEKs later your game has a real-world timestamp — for a tournament clock, a daily challenge seed, or just a clock on the title screen.

THE FUJINET.BAS LIBRARY

Sixteen procedures, tested in three shipped games. INCLUDE it and go.

Listing 1 is the complete library — the exact transport all three game chapters build on, in its newest revision (the games in the field carry two vintages; this one folds in the compiler-bug fixes from Chapter 3 and adds `net_write` and `fn_putnum`). Include it after your opening `GOTO`:

```
GOTO main
INCLUDE "fujinet.bas"
main:
```

Remember: never fall into an include. Chapter 3's first survival rule.

TRANSPORT

FN_WAIT_MAILBOX

Waits up to 180 frames for the mailbox magic bytes. Sets `fn_ok`. Call once at boot, before anything else. ¹

FN_TRANSACT

The engine. In: `mb_dev`, `mb_cmd`, `mb_nparam`, `#fn_txlen` (payload already at `FN_TX`), parameters staged via `fn_param`. Out: `fn_ok` on failure `#mb_err` holds the link error (0 on timeout — the RP2040 never answered). Derives SEQ from `ACKSEQ` ², polls up to 900 frames ³, and verifies the ACK ⁴.

FN_PARAM

Stages parameter `pm_i` (0-based) of size `pm_size` (1 or 2 bytes here; the mailbox allows 4) with value `#pm_val` into the parameter table.

FN_PUTSTR / FN_STRLIN

The string kit. `fn_putstr` appends `fn_len` bytes from address `#fn_src` — ROM DATA via VARPTR, or any RAM buffer — to the TX window, advancing `#fn_txlen`. `fn_strlen` measures a NUL-padded field first, so padded storage never leaks NULs into a URL (the wire sends TXLEN bytes verbatim; an embedded NUL corrupts the HTTP request downstream).

NETWORK SHORTHAND**NET_OPEN**

Opens the devicespec staged at `FN_TX` for HTTP GET (mode 12, translation 0).

NET_STATUS

STATUS with the two zero parameters; unpacks available count into `#net_avail` and the error byte into `#net_err` ⁵ — and folds “error ≠ 1” into `fn_ok`, which is what catches an HTTP error page masquerading as a good response.

NET_READ

Reads `#net_readlen` bytes; the count that actually arrived lands in `#net_gotlen` — captured before CLOSE can overwrite RXLEN.

NET_WRITE

Sends `fn_len` bytes already at `FN_TX`. New in this guide’s revision; the netcat in Appendix D is its first customer.

NET_CLOSE

Closes N1:. Unconditionally safe.

API_CALL

The one-shot HTTP GET: open → settle-poll STATUS ⁶ → clamp → read → close. Set `#net_readlen` to your reply buffer budget first.

This is the only network call the three games ever make.

APPKEYS

APPKEY_OPEN / APPKEY_READ / APPKEY_WRITE / APPKEY_CLOSE

Set `ak_creator_lo/hi`, `ak_app`, `ak_key`, `ak_mode` and open; then read into a buffer at `#fn_src` (bounded by `ls_max`, always NUL-terminated, actual length in `fn_len` — the 2-byte reply prefix is consumed for you ⁷) or write `fn_len` bytes from `#fn_src` then close.

UTILITY

FN_PUTNUM

Appends `pn_val` (0–999) to the TX window as decimal ASCII — the missing `itoa` for building URLs like `/attack/47`.

NOTE: The library owns scratch RAM \$9100–\$917F for three buffers: `SC_NAME` (player name, 9), `SC_ENDPT` (server endpoint, 64), `SC_QUERY` (48). Everything below \$9C00 and above your own use is yours to allocate; the games start theirs at \$9180.

CHAPTER 8

GAME ONE: 5 CARD STUD

Eight seats, a felt table, and every bet a URL. The founding game of the FujiNet game system, on a 159×96 screen.

5 Card Stud (Listing 2) is the pattern the other two games copy: a thin IntyBASIC client that draws whatever a stateless-feeling HTTP server says the table looks like. The client never knows the rules of poker. It knows how to draw a Game structure, and how to ask again.

THE SHAPE OF THE CLIENT



At boot (Listing 2 ¹) the client reads the shared player name from AppKey creator 1 / app 1 / key 0 — **with one retry**, because the very first transaction after power-on can catch the USB link still waking — and validates every byte to A–Z 0–9. That slot is shared by every FujiNet game on every platform, so it can hold anything; anything outside the name-entry alphabet goes straight into a URL query string unescaped, and a stray & turns into a broken request the server answers with an HTTP error page. Fall back to the letter-picker if in doubt.

Then the Lobby handoff (Chapter 11, and ² in Listing 2): AppKey creator 1 / app 1 / key 1 may hold `https://host/?table=xyz` written by the Lobby client. If it parses (`split_room_url`), the client skips straight to that table; if not, it seeds the compiled-in default endpoint and shows table select.

URLS, COMPOSED BYTE BY BYTE

IntyBASIC has no strings, so URLs are assembled by `compose_url` (3) from ASCII DATA literals and scratch buffers, directly into the TX window:

N:https://5card.carr-designs.com/state?table=r1&player=THOM&bin=1

PATH	WHEN	REPLY
tables?bin=1	table select	Tables structure
state?... &bin=1	every poll	Game structure
move/XX?... &bin=1	your move (XX = 2-char code)	Game structure
leave?... &bin=1	quit table	bye

?bin=1 asks the server for its binary serialization — fixed offsets, no JSON parser needed. Multi-byte numbers are little-endian. (There is a &be=1 big-endian option; the Intellivision client tried it and retired it after pot values kept landing on suspicious multiples of 256 — the signature of a byte-order mismatch. Little-endian is the road the whole client family exercises daily. Stay on it.)

THE TABLES STRUCTURE

OFFSET	SIZE	FIELD	NOTES
0	1	count	tables that follow
1+36i	9	table id	NUL-padded, lowercased
10+36i	21	name	
31+36i	6	players	literal text like 2 / 8

The client validates before trusting: reply length must cover $1 + \text{count} \times 36$ bytes, or the “Tables structure” was actually an error page and the poll is retried (4).

THE GAME STRUCTURE

OFFSET	SIZE	FIELD	NOTES
0	81	lastResult	one-shot message, NUL-padded
81	1	round	0 wait, 1–4 streets, 5 showdown
82	2	pot	u16 LE
84	1	activePlayer	signed; \$FF = none, 0 = you
85	1	moveTime	seconds left, server-computed
86	1	viewing	1 = you are a spectator
87	1	validMoveCount	
88	65	validMoves[5]	13 each: code[3] + name[10]
153	1	playerCount	
154	33×N	players[N]	name[9] status(1) bet(2 LE) move[8] purse(2 LE) hand[11]

Wire index 0 is **always you** — the server rotates the array per client. Hands are ASCII pairs (rank 2–9, t j q k a suit d h c s); a hidden card is ??, which the renderer converts to the card-back glyph. Status: 0 waiting, 1 playing, 2 folded, 3 left.

Validation is layered (5): length ≥ the 154-byte fixed prefix; round ≤ 5 and playerCount ≤ 8 (an HTTP error page fails this instantly); and length ≥ 154 + playerCount×33 — because a short read leaves stale bytes from a previous, larger reply in the RX window, and fixed-offset reads would render them as garbage.

DRAWING WITHOUT FLICKER

The STIC has no page-flip: BACKTAB is live every frame. The client's answer (6, and the long comment in Listing 2 is worth reading whole) is **differential rendering**: a full CLS only when the layout genuinely changed (first render, player count changed, new hand); otherwise only cells whose values changed are poked, and the redraw yields a **WAIT** between seats so no single burst outruns vblank — the cause of a half-drawn frame a screenshot once caught and made look like data corruption. Seat placement comes from the C

clients' `seatmap` table (player count → seat ring); each seat draws name, bet and its dealt cards, clicking `sound_deal` only for cards not shown last poll.

YOUR TURN

When `activePlayer` is 0 and you are not a spectator, `move_ui` (7) lists up to five server-supplied moves on the status row — **the server names the moves** the client just draws `validMoves[i].name` and submits `.move` — with the cursor defaulting to the second entry (not Fold), a countdown seeded from `moveTime` (the server already subtracted a 4-second network grace), and timeout submitting the highlighted move, exactly like the C clients. Move codes you will see: **F0** fold, **CH** check, **CA** call, **BL/BH** bet low/high, **RA** raise, **BB** post bring-in.

The one-shot `lastResult` ("fry bot won with pair, sixes") is easy to miss at bot speed, so the client hashes the field each poll; a changed, non-empty hash blacks out the bottom rows, shows the message for four seconds (8), and forces a redraw afterward.

```
CHOOSE A TABLE
>THE MAIN TABLE  2/8
  SMALL STAKES    0/8
  BOT PRACTICE    1/8
  HIGH ROLLERS    0/8
```

THE SERVER

`servers/fujinet-game-system/5cardstud` is a Go/gin service. Every route does the same four things: lock the table, apply game logic, save state, serialize the client-centric view — JSON by default, the binary layout above with `?bin=1` (that serializer is `util.go`'s `appendFixedLengthString` plus little-endian `appendUint16` every string is lowercased and gets one NUL terminator, which is why fields are "size + 1" on the wire). State lives in a `sync.Map` keyed by table, one mutex per table; bots move after 3 seconds, humans get 39, minus the 4-second grace the client shows. A hidden `test` table is reserved for client developers — polls there never register on the public Lobby.

CHAPTER 9

GAME TWO: BATTLESHIP

Four 10×10 oceans on one screen, and one shot that hits all of them. A lesson in the STIC's other graphics mode.

Battleship (Listing 6) needed something 5 Card Stud did not: four 10×10 grids visible at once. Text cards cannot do it — but the STIC's **colored-squares** mode can. Its distinctive rule: every shot you fire lands on the same coordinate of **every** opposing board simultaneously. One cursor, up to three victims.

THE COLORED-SQUARES KERNEL

A BACKTAB word with bit 12 set and bit 11 clear stops being a character and becomes four independently colored 4×4-pixel quadrants. `board.bas` (Listing 8) wraps that in three calls: `cs_fill` (whole card), `cs_plot` (read-modify-write one quadrant, ¹), and the board-level `board_cell` (quadrant of a 5×5-card board = one grid cell). Two hardware quirks are baked into its tables: the bottom-right quadrant's high color bit lives up at bit 13, not bit 11 (the pre-shifted `cs_val` table hides this); and the update mask must clear the enable bit before adding it back — an early version preserved it, the ADD doubled it into bit 13, and the card snapped back to text mode showing a stray letter (the comment at ¹ tells the story).

The screen: quadrants q1/q2 on the top row, q0 (always you) / q3 below, black divider lines, a 9-column chrome panel on the right for names and ships-left, and the prompt row at the bottom.

CLIENT FLOW AND WIRE FORMAT

Boot, name, Lobby handoff and table select are 5 Card Stud verbatim (AppKey key 5 this time). The poll loop differs in one habit: the reply's **header** decides everything. Requests add `&bin=1&v=2` — version 2 matters, because at game over v1 hides the winner's ships and the game-over screen wants to reveal them (²).

OFFSET	SIZE	FIELD	NOTES
0	1	playerCount	
1	33	prompt	server-composed status line
34	1	status	0 lobby, 1 place, 10 start, 11 miss, 12 hit, 13 sunk, 99 over
35	1	playerStatus	you: 0 playing, 1 defeated, 2 viewing, 3 ready, 10 placing
36	1	activePlayer	signed, \$FF none
37	1	moveTime	seconds

Then it branches. In the lobby (status 0): server name (21) plus 10-byte records {name[9], ready}. In play: lastAttackPos (1), myShips (10 — yours in [0..4], the winner's in [5..9] at game over), then 115-byte records {name[9], status, gamefield[100], shipsLeft[5]}. A gamefield byte is 0 unknown, 1 hit, 2 miss; a ship byte encodes **cell + 100×direction** (0 across, 1 down) — so 47 is a horizontal ship starting at column 7 row 4, and 147 the same cell going down. `validate_state` (Listing 7 ³) computes the expected length for whichever branch applies before any offset is trusted.

PLACING AND SHOOTING

Placement (⁴) is fully client-side until the last moment: five ships (5,4,3,3,2) start at random legal spots; the disc slides, **B1** rotates, the action button confirms against a local occupancy map — no server round-trip to discover an overlap. The result is five encoded bytes, comma-joined into `/place/25,113,4,167,89`. The server answers with authoritative myShips, and the renderer draws **those**, which is what makes a reconnect after a reset come back with your real fleet.

Targeting (⁵) draws one yellow cursor on every live enemy quadrant in lockstep, counts down `moveTime`, and on fire sends `/attack/47`. The status codes 11/12/13 then narrate the result — paired with `lastAttackPos` so each miss/hit/sunk sound plays once per actual event, not once per poll that repeats the status (⁶).

Rendering diffs each visible gamefield against a 100-byte shadow copy per quadrant in scratch RAM (⁷) — the same differential-drawing lesson as 5 Card Stud, with the same one-WAIT-per-board vblank discipline.

THE SERVER

Routes: `/state`, `/ready`, `/place/:ships`, `/attack/:pos`, `/leave`, `/tables`, `/view` — plus a dev-only `/debugEndGame/:winner`. Same table-mutex pattern as 5 Card Stud. Timing: bots 3 s, humans 45 s (250 s when you are alone at a table, capped so it still fits the one-byte `moveTime`), 4-second network grace, a 5-second bonus after a status change. The hard-coded rooms: two open seas and three AI tables (one, two and three bots), plus the hidden `test` room. When only bots remain, the game resets to the lobby.

GAME THREE: FUJITZEE

Five dice, thirteen rounds, sixteen scores — and a scorecard bigger than the screen.

Fujitzee (Listing 10) is the dice game you think it is, and the most UI-dense of the three: a 15-category scorecard per player simply does not fit on a 20x12 screen next to six seats and five dice. The client's answer is **one card at a time** — a permanent seat strip, dice row and prompt, with the disc paging whose scorecard fills the middle.

WIRE FORMAT

Unlike Battleship, the structure is the same shape in every round:

OFFSET	SIZE	FIELD	NOTES
0	1	playerCount	seats + spectators, up to 12
1	21	serverName	
22	41	prompt	
63	1	round	0 lobby, 1–13 play, 99 over
64	1	rollsLeft	
65	1	activePlayer	signed
66	1	moveTime	seconds
67	1	viewing	
68	6	dice	5 ASCII digits 1–6 + NUL; empty at turn start
74	6	keepRoll	5 ASCII 0/1 + NUL — 1 rerolls
80	15	validScores	signed bytes; \$FF = not selectable
95	42xN	players[N]	name[9] alias(1) scores[16] (u16 LE each)

Scores are sixteen little-endian words: six upper categories, computed upper total and bonus (indices 6, 7 — a player never picks those directly), seven lower categories (8–14, index 14 is Fujitzee itself), and the running total at 15. \$FFFF is “unset”; in the lobby, scores[0] moonlights as the ready flag (1 ready, 0 not, \$FFFE spectator).

WATCH OUT: Twelve 42-byte records plus the 95-byte header is 599 bytes — more than the mailbox’s 512-byte RX window. Up to nine player records fit; a table with more spectators than that will fail the length gate and the poll simply retries. Seats are capped at six, so play is never affected — but know where the ceiling is.

TURN MACHINERY

Your turn is a little two-mode machine (❶, Listing 10): **DICE** mode — cursor over the ROLL tile and five dice, the action button toggles a die between reroll (the default, wire 1) and keep (0), ROLL submits /roll/01100-style masks; and **SCORE** mode — cursor over the open categories, with your current roll’s would-be score already painted green in each open cell straight from validScores. Out of rolls, the client hops to SCORE mode by itself and parks the cursor on the highest-scoring open category (❷) — the same courtesy the C clients extend. Submitting sends /score/11 with the **wire** index (the display collapses the computed rows 6–7, so the cursor-to-index map adds 2 past the upper section — ❸).

Two touches keep the multiplayer legible. Whenever the active player changes, the viewed card snaps to them. And when **someone else’s** turn ends, the client diffs their scores against a shadow copy it refreshed on every poll (❹) to find which category they just filled, then holds their card on screen for three-quarters of a second with that cell highlighted — so a bot’s instant move is still a visible event.

Rolls animate on **every** card, yours or not: a change in rollsLeft (or a turn change — that covers the server’s automatic opening roll) flickers the dice the wire’s keepRoll says were thrown (❺).

THE SERVER

Routes `/state`, `/ready`, `/roll/:keep`, `/score/:index`, `/leave`, `/tables`, `/view`, with the familiar per-table locking. Timing: bots 3 s, humans 45 s — 15 for a player being penalized for a previous timeout, 250 when alone — game-start countdowns of 31/6/3 seconds, and the same 4-second grace baked into `moveTime`. Scoring — including the upper bonus at 63, the Fujitzee bonus scores and the automatic totals — is entirely server-side: the client never adds a die.

CHAPTER 11

THE LOBBY

How players find your table — and how a game remembers where it was.

The FujiNet Lobby (`lobby.fujinet.online`) is the directory of every live game room on every platform. The pieces:

- **Game servers upsert themselves.** Each server POSTs a JSON `GameServer` record — game name, region, server URL (with `?table=` suffix per room), current and max players, plus a per-platform list of client download URLs — to the Lobby whenever a room's population changes, and game results when a match ends. A QA Lobby at `qalobby.fujinet.online` receives the same traffic from development servers.
- **The Lobby client hands off through AppKeys.** When a player picks a room in the Lobby browser, the Lobby writes `https://server/?table=id` into that game's registered AppKey slot (creator 1 / app 1 / key: 1 five-card-stud, 3 fujitzee, 5 battleship) and boots the game client.
- **The game rejoins on its own.** At boot each game reads its slot, splits the URL at the `?` (`split_room_url`, validating that the query really is `table=` plus letters and digits — the value goes into rebuilt URLs unescaped), and if it parses, skips table select entirely. Picking a table by hand **writes** the same slot back, so a console reset rejoins your game; deliberately quitting **clears** it, so a reset lands on table select instead of dragging you back into a table you left.

The shared username slot (key 0) rounds out the handshake: every lobby game on every platform reads and writes the same player name. Treat both slots as hostile input — Chapters 8 through 10 all validate them byte-by-byte, and so should you.

ERROR CODES

MAILBOX LINK ERRORS (FN_ERR, \$9C0B)

VALUE	NAME	MEANING
0	OK	transaction completed (or, after a timeout, never started)
1	ENOLINK	no USB link to the ESP32 — check FN_LINK, wait, retry
2	ETIMEOUT	the ESP32 did not answer within the deadline
3	EBADFRAME	framing/checksum failure on the link
4	ETOOBIG	request or reply exceeded the buffers

NETWORK STATUS ERRORS (STATUS BYTE 3)

VALUE	NAME	MEANING
1	SUCCESS	all is well
128	WRITE ONLY	channel not open for reading
131	READ ONLY / WRITE ONLY	direction not open
132	INVALID COMMAND	
135	READ ONLY	
136	END OF FILE	
138	GENERAL TIMEOUT	
144	GENERAL ERROR	
146	NOT IMPLEMENTED	
151	FILE EXISTS	

VALUE	NAME	MEANING
162	NO SPACE ON DEVICE	
165	INVALID DEVICESPEC	the URL did not parse
166	INVALID POINT	bad SEEK
167	ACCESS DENIED	
170	FILE NOT FOUND	
200	CONNECTION REFUSED	
201	NETWORK UNREACHABLE	
202	SOCKET TIMEOUT	
203	NETWORK DOWN	
204	CONNECTION RESET	
205	CONNECTION ALREADY IN PROGRESS	
206	ADDRESS IN USE	
207	NOT CONNECTED	READ/WRITE with no OPEN
208	SERVER NOT RUNNING	
209	NO CONNECTION WAITING	TCP ACCEPT with no caller
210	SERVICE NOT AVAILABLE	
211	CONNECTION ABORTED	
212	INVALID USERNAME OR PASSWORD	
213	COULD NOT PARSE JSON	
214	CLIENT ERROR	HTTP 4xx
215	SERVER ERROR	HTTP 5xx
255	COULD NOT ALLOCATE BUFFERS	

NETWORK-BOOT ERRORS (FN_BOOT_ERR, \$9C1A)

VALUE	NAME	MEANING
1	REJECTED	ROM header malformed (bad address range)
2	TRUNCATED	Intellicart stream ended mid-segment
3	NOMAP	no .cfg, no header, and the size matches no known layout
4	MAILBOX	a segment would overlap the mailbox window
5	CFGBAD	a .cfg arrived but held no mapping line

APPENDIX B

QUICK REFERENCE

THE TRANSACTION RECIPE

1. Stage parameters: $\text{PARAM_SIZE}[i] = 1/2/4$, value LE at $\text{PARAM_VAL} + 4i$
2. Stage payload at TX, length in TXLEN (LE)
3. Write DEVICE, CMD, NPARAM
4. $\text{SEQ} = \text{ACKSEQ} + 1$ (wrap, skip 0) — **last write**
5. Poll until $\text{ACKSEQ} = \text{SEQ}$ (timeout ≈ 15 s)
6. $\text{REPLY_CMD} = \$06?$ Reply at RX, length RXLEN : else ERR

DEVICES

ID	DEVICE	NOTES
\$70	Fuji control	WiFi, hosts, mounts, AppKeys, encoders
\$71-78	Network N1:-N8:	one connection each
\$45	Clock	APETime command set
\$31-38	Disk	block devices (not used by the Inty client)

NETWORK COMMANDS (DEV \$71-\$78)

CMD	NAME	PARAMS / PAYLOAD
\$4F '0'	OPEN	p0 mode, p1 translation; payload devicespec
\$43 'C'	CLOSE	—
\$52 'R'	READ	p0 length(2) → data

APPENDIX B

CMD	NAME	PARAMS / PAYLOAD
\$57 'W'	WRITE	p0 length(2); payload data
\$53 'S'	STATUS	p0 0, p1 type → 4 bytes: avail lo/hi, conn, err
\$FC	CHANNEL MODE	p1: 0 raw, 1 JSON, 2 SGML
\$50 'P'	PARSE	—
\$51 'Q'	QUERY	payload query path
\$4D 'M'	HTTP CHANNEL	p1: 0 body, 1–3 headers, 4 POST data
\$54 'T'	TRANSLATION	p1 mode
\$4C 'L'	SET EOL	p0/p1 EOL bytes
\$25 '%'	SEEK	p0 offset(4)
\$26 '&'	TELL	→ 4-byte offset
\$30 '0'	GETCWD	→ prefix
\$2C ', '	CHDIR	payload path
\$FD	USERNAME	payload
\$FE	PASSWORD	payload
\$41 'A'	TCP ACCEPT	—
\$63 'c'	TCP CLOSE CLIENT	—
\$44 'D'	UDP DESTINATION	payload host:port
\$20/\$21	RENAME / DELETE	payload spec
\$23/\$24	LOCK / UNLOCK	payload spec
\$2A/\$2B	MKDIR / RMDIR	payload spec

FUJI COMMANDS (DEV \$70)

CMD	NAME	PARAMS / PAYLOAD
\$FF	RESET	—

CMD	NAME	PARAMS / PAYLOAD
\$FE	GET SSID	→ 97 bytes
\$FD	SCAN NETWORKS	→ count
\$FC	GET SCAN RESULT	p0 index → 34 bytes
\$FB	SET SSID	p0 any; payload 97 bytes
\$FA	GET WIFI STATUS	→ 1 byte, 3 = connected
\$F9	MOUNT HOST	p0 slot
\$F8	MOUNT IMAGE	p0 slot, p1 flags — boots the ROM on Inty
\$F7	OPEN DIRECTORY	p0 slot; payload 256 bytes path+filter
\$F6	READ DIR ENTRY	p0 maxlen, p1 flags → name / \$7F7F
\$F5	CLOSE DIRECTORY	—
\$F4	READ HOST SLOTS	→ 256 bytes
\$F3	WRITE HOST SLOTS	payload 256 bytes
\$F2/\$F1	READ / WRITE DEVICE SLOTS	304-byte table
\$EA	GET WIFI ENABLED	→ 1 byte
\$E9	UNMOUNT IMAGE	p0 slot
\$E8	GET ADAPTERCONFIG	→ 140 bytes
\$E7	NEW DISK	payload spec
\$E5/\$E4	GET / SET DIR POSITION	p0 pos(2)
\$E2	SET DEVICE FULLPATH	p0 dev, p1 host, p2 mode; payload 256
\$E1/\$E0	SET / GET HOST PREFIX	p0 slot
\$DE	WRITE APPKEY	payload value
\$DD	READ APPKEY	→ 2-byte length + value
\$DC	OPEN APPKEY	payload 6 bytes
\$DB	CLOSE APPKEY	—

APPENDIX B

CMD	NAME	PARAMS / PAYLOAD
\$DA	GET DEVICE FULLPATH	p0 slot → path
\$D9	CONFIG BOOT	p0 flag
\$D8	COPY FILE	p0 src, p1 dst; payload spec
\$D7	MOUNT ALL	—
\$D6	SET BOOT MODE	p0 mode
\$D0 - \$C9	BASE64 ENC/DEC	input / compute / length / output
\$C8 - \$C2	HASH	p0: algo 0 MD5, 1 SHA1, 2 SHA256, 3 SHA512
\$C4	ADAPTERCONFIG EXTENDED	→ 240 bytes
\$BF - \$BC	QR CODE	input / encode / length / output
\$BB	GENERATE GUID	→ guid
\$53	STATUS	p0 1 = mount times
\$00	DEVICE READY	→ 512 test bytes

APPKEY REGISTRY (CREATOR 1, APP 1)

KEY	HOLDER	CONTENTS
0	all lobby games	shared player name
1	5 Card Stud	server?table= handoff
3	Fujitzee	server?table= handoff
5	Battleship	server?table= handoff

APPENDIX C

PROGRAM LISTINGS

The complete, building source of the library and all three games, line-numbered, with the call-outs the chapters reference. Each game also includes the community-standard constants.bas (Mark Ball's, from the IntyBASIC distribution), not reprinted here.

LISTING 1 fujinet.bas — the FujiNet library (Chapter 7)

```

1 ' fujinet.bas -- FujiNet mailbox transport + network/appkey primitives.
2
3 ' Mailbox layout is the hand-synchronized copy of
4 ' fujinet-firmware/pico/intellivision/firmware/fuji_mailbox.h, exactly as
5 ' proven working by intv/fujitests.bas in that repo. Do not change these
6 ' addresses without re-checking that file.
7
8 ' MEMATTR intentionally stops at $9BFF, short of the $9C00-$9FFF mailbox
9 ' itself: on real PiT0 II hardware the RP2040 maps that whole window as
10 ' RAM unconditionally (inty_cart.c hardcodes it, independent of what this
11 ' .cfg says), so declaring less here doesn't affect real hardware or what
12 ' POKE can reach at runtime. But jzIntv's --fujinet peripheral emulation
13 ' registers its own handler for $9C00-$9FFF *after* the cart's generic
14 ' MEMATTR RAM, and its layered bus dispatch lets whichever peripheral
15 ' registered first answer a given address -- so declaring the full
16 ' $8000-$9FFF range here (as fujitests.bas does) silently shadows the
17 ' emulator's FujiNet peripheral with inert RAM, and the mailbox never
18 ' comes up under --fujinet even though it works on real hardware.
19 ' ASM MEMATTR $8000, $9BFF, "+RWN" 8
20
21 CONST FN_MAGIC0 = $9C00
22 CONST FN_MAGIC1 = $9C01
23 CONST FN_SEQ = $9C03
24 CONST FN_ACKSEQ = $9C04
25 CONST FN_DEVICE = $9C05
26 CONST FN_CMD = $9C06
27 CONST FN_NPARAM = $9C07
28 CONST FN_TXLEN_LO = $9C08
29 CONST FN_TXLEN_HI = $9C09
30 CONST FN_ERR = $9C0B
31 CONST FN_RXLEN_LO = $9C0C
32 CONST FN_RXLEN_HI = $9C0D
33 CONST FN_REPLY_CMD = $9C0E
34
35 CONST FN_PARAM_SIZE = $9C10
36 CONST FN_PARAM_VAL = $9C20
37 CONST FN_TX = $9C40
38 CONST FN_RX = $9D40
39
40 CONST FUJICMD_ACK = $06
41 CONST FUJICMD_NAK = $15
42
43 ' Fuji device (config/appkey) commands.
44 CONST FUJI_DEVICEID = $70
45 CONST FUJICMD_OPEN_APPKEY = $DC
46 CONST FUJICMD_CLOSE_APPKEY = $DB
47 CONST FUJICMD_WRITE_APPKEY = $DE
48 CONST FUJICMD_READ_APPKEY = $DD
49
50 ' Network device (N1:) commands.
51 CONST NET_DEVICEID = $71
52 CONST NETCMD_OPEN = $4F
53 CONST NETCMD_CLOSE = $43
54 CONST NETCMD_READ = $52
55 CONST NETCMD_WRITE = $57
56 CONST NETCMD_STATUS = $53
57
58 ' $0C is HTTP "GET with header access" and, on file/socket protocols,
59 ' plain READ-WRITE -- the same value serves both names.
60 CONST OPEN_MODE_HTTP_GET_H = $0C
61 CONST OPEN_MODE_RW = $0C
62 CONST OPEN_TRANS_NONE = $00
63
64 ' Scratch RAM ($9000-$97FF) -- ours, outside the mailbox proper. Holds
65 ' state that must survive across multiple mailbox transactions (the
66 ' mailbox's own TX/RX buffers get overwritten by every call).
67 CONST SC_NAME = $9100 ' playerName, 9 bytes (8 + NUL)
68 CONST SC_ENDPT = $9110 ' serverEndpoint, 64 bytes

```

```

68  CONST SC_QUERY    = $9150 ' query (?table=...&player=...), 48 bytes
69
70  ' fn_ok: 1 = last transaction produced ACK, 0 = timeout or NAK.
71  ' mb_err: FN_ERR value on failure (0 on timeout, since the RP2040 never
answered).
72  DIM fn_ok, #mb_err
73  DIM mb_dev, mb_cmd, mb_nparam, mb_seq
74  DIM #fn_txlen
75  DIM #fn_t          ' generic frame-count timeout counter
76  DIM #fn_src        ' VARPTR source for putstr/getstr
77  DIM fn_len, fn_i    ' generic length/index for putstr/getstr
78
79  '-----
80  ' fn_wait_mailbox: bounded wait for the RP2040 magic bytes at boot.
81  ' Sets fn_ok = 1 if the mailbox came up within 180 frames (3s), else 0.
82  '-----
83  fn_wait_mailbox: PROCEDURE
84  'fn_t = 0
85  WHILE ((PEEK(FN_MAGIC0) AND 255) <> 70) AND ((PEEK(FN_MAGIC1) AND 255) <>
78) AND (#fn_t < 180)
86  'fn_t = #fn_t + 1
87  WAIT
88  WEND
89  IF #fn_t >= 180 THEN
90  'fn_ok = 0
91  ELSE 1
92  'fn_ok = 1
93  END IF
94  END
95
96  '-----
97  ' fn_transact: issue the transaction described by mb_dev/mb_cmd/mb_nparam/
98  ' #fn_txlen (payload already staged at FN_TX) and block for the reply.
99  ' seq is ALWAYS derived from the RP2040's own FN_ACKSEQ, never from a local
100  ' counter -- a console reset zeroes IntyBASIC vars but not the RP2040, so a
101  ' locally incrementing seq would recompute the same value forever and never
102  ' trigger a second transaction after the first boot.
103  ' Sets fn_ok = 1 on ACK, 0 on timeout/NAK (mb_err holds the reason).
104
105  ' mb_err/net_err are deliberately 16-bit (#-prefixed), not the 8-bit plain
106  ' variables their DIM comments once declared them as: IntyBASIC v1.4.2
107  ' silently drops the AND 255 mask on any assignment of the literal shape
108  ' "plain_var = PEEK(...) AND 255" (with or without wrapping parens) when
109  ' the destination is an 8-bit variable -- confirmed by reading the
110  ' generated .lst, no ANDI instruction is emitted at all, so whatever
111  ' garbage sits in the peeked word's upper byte survives into the
112  ' variable. #-prefixed (16-bit) destinations don't hit this; the ANDI
113  ' shows up correctly. net_err in particular is compared with "< 1"
114  ' immediately after, so a stray upper byte there would misreport a good
115  ' status as a failure.
116  '-----
117  fn_transact: PROCEDURE

```

```

118  POKE (FN_DEVICE), mb_dev
119  POKE (FN_CMD), mb_cmd
120  POKE (FN_NPARAM), mb_nparam 2
121
122  POKE (FN_TXLEN_LO), #fn_txlen AND 255
123  POKE (FN_TXLEN_HI), #fn_txlen / 256
124
125  mb_seq = (PEEK(FN_ACKSEQ) AND 255) + 1
126  IF mb_seq = 0 THEN mb_seq = 1 3
127
128  POKE (FN_SEQ), mb_seq
129
130  #fn_t = 0
131  WHILE ((PEEK(FN_ACKSEQ) AND 255) <> mb_seq) AND (#fn_t < 900)
132  'fn_t = #fn_t + 1
133  WAIT
134  WEND
135
136  IF #fn_t >= 900 THEN
137  'fn_ok = 0
138  'mb_err = 0 4
139
140  RETURN
141  END IF
142
143  IF (PEEK(FN_REPLY_CMD) AND 255) <> FUJICMD_ACK THEN
144  'fn_ok = 0
145  'mb_err = (PEEK(FN_ERR) AND 255)
146  RETURN
147  END IF
148
149  fn_ok = 1
150  END
151
152  '-----
153  ' fn_param: stage transaction parameter #pm_i (0-based), #pm_size bytes
154  ' (1 or 2), value #pm_val, little-endian, into the mailbox's param table.
155  '-----
156  DIM pm_i, pm_size
157  DIM #pm_val
158  fn_param: PROCEDURE
159  POKE (FN_PARAM_SIZE + pm_i), pm_size
160  POKE (FN_PARAM_VAL + pm_i * 4), #pm_val AND 255
161  IF pm_size > 1 THEN POKE (FN_PARAM_VAL + pm_i * 4 + 1), #pm_val / 256
162  END
163
164  '-----
165  ' fn_putstr: append fn_len ASCII bytes into FN_TX, starting at the current
166  ' #fn_txlen, from the address in #fn_src -- either VARPTR of a ROM DATA
167  ' label (a literal like lit tables(0)) or a RAM address (a scratch buffer
168  ' like SC_NAME, or even FN_RX). PEEK is uniform across ROM/RAM on the
169  ' CP-1610's unified address space, so one procedure covers both. Advances
170  ' #fn_txlen.
171  '-----

```

```

169 fn_putstr: PROCEDURE
170   FOR fn_i = 0 TO fn_len - 1
171     POKE (FN_TX + #fn_txlen + fn_i), PEEK(#fn_src + fn_i) AND 255
172   NEXT fn_i
173   #fn_txlen = #fn_txlen + fn_len
174 END
175
176 '-----
177 ' fn_strlen: scan the NUL-padded field at #fn_src (max ls_max bytes) and set
178 ' fn_len to the length up to (but not including) the first NUL. Use this
179 ' before fn_putstr whenever the source is a fixed-width padded field (table
180 ' id, player name) -- putstr has no idea where the real string ends, and
181 ' blindly copying the full padded width embeds a literal $00 byte plus
182 ' whatever garbage follows it into the URL, which is sent as-is (the URL's
183 ' length is a byte count, not NUL-terminated, so nothing downstream stops
184 ' at that NUL either -- it corrupts the HTTP request on the wire).
185 '-----
186 DIM ls_max
187 fn_strlen: PROCEDURE
188   fn_len = 0
189   WHILE (fn_len < ls_max) AND ((PEEK(#fn_src + fn_len) AND 255) <> 0)
190     fn_len = fn_len + 1
191   WEND
192 END
193
194 '-----
195 ' net_open: open devicespec (ASCII bytes already staged at FN_TX, length in
196 ' #fn_txlen) for HTTP GET. Leaves fn_ok set.
197 '-----
198 net_open: PROCEDURE
199   mb_dev = NET_DEVICEID
200   mb_cmd = NETCMD_OPEN
201   mb_nparam = 2
202   pm_i = 0 : pm_size = 1 : #pm_val = OPEN_MODE_HTTP_GET_H : GOSUB fn_param
203   pm_i = 1 : pm_size = 1 : #pm_val = OPEN_TRANS_NONE : GOSUB fn_param
204   GOSUB fn_transact
205 END
206
207 '-----
208 ' net_status: query byte count available. Result in #net_avail; fn_ok as usual.
209 '-----
210 DIM #net_avail
211 ' net_err: the 4th byte of the NDeviceStatus reply (nDevStatus.t). 1 =
212 ' SUCCESS. For an HTTP-backed devicespec, the *actual* GET is deferred
213 ' until this first STATUS call (see NetworkProtocolHTTP::status_file())
214 ' http_transaction() in fujinet-firmware) and its result code lands here
215 ' -- an HTTP error response (e.g. a 400) still has a real, readable body
216 ' (the error page), so 'avail' alone can't tell it apart from a good
217 ' response. Checking net_err is what actually can.
218 DIM #net_err
219 net_status: PROCEDURE
220   mb_dev = NET_DEVICEID
221   mb_cmd = NETCMD_STATUS
222
223   mb_nparam = 2
224   pm_i = 0 : pm_size = 1 : #pm_val = 0 : GOSUB fn_param
225   pm_i = 1 : pm_size = 1 : #pm_val = 0 : GOSUB fn_param
226   #fn_txlen = 0
227
228   GOSUB fn_transact
229   IF fn_ok THEN
230     #net_avail = (PEEK(FN_RX) AND 255) + (PEEK(FN_RX + 1) AND 255) * 256
231     #net_err = (PEEK(FN_RX + 3) AND 255)
232     IF #net_err <> 1 THEN fn_ok = 0
233   ELSE
234     #net_avail = 0
235   END IF
236 END
237
238 '-----
239 ' net_read: read #net_readlen bytes into FN_RX (reply payload lands there
240 ' directly -- callers PEEK it in place, per the "never buffer in IntyBASIC
241 ' variables" rule). fn_ok as usual.
242 '-----
243 DIM #net_readlen
244 ' #net_gotlen: the actual byte count the RP2040/FujiNet peripheral reported
245 ' receiving (RXLEN), captured here because the CLOSE transaction that
246 ' follows in api_call overwrites FN_RXLEN_LO/HI with its own (always 0)
247 ' reply length -- callers need this checked before that happens.
248 DIM #net_gotlen
249 net_read: PROCEDURE
250   mb_dev = NET_DEVICEID
251   mb_cmd = NETCMD_READ
252   mb_nparam = 1
253   pm_i = 0 : pm_size = 2 : #pm_val = #net_readlen : GOSUB fn_param
254   #fn_txlen = 0
255   GOSUB fn_transact
256   IF fn_ok THEN
257     #net_gotlen = (PEEK(FN_RXLEN_LO) AND 255) + (PEEK(FN_RXLEN_HI) AND 255)
258   ELSE
259     #net_gotlen = 0
260   END IF
261 END
262
263 '-----
264 ' net_write: send fn_len bytes already staged at FN_TX out the open
265 ' channel. The byte count rides twice -- as parameter 0 (the command's
266 ' length argument) and as the transaction payload length -- and both come
267 ' from fn_len here. fn_ok as usual. The three games never write to the
268 ' network (their moves are GET query strings), but a TCP/Telnet client
269 ' needs this, so the library carries it.
270 '-----
271 net_write: PROCEDURE
272   mb_dev = NET_DEVICEID
273   mb_cmd = NETCMD_WRITE
274   mb_nparam = 1

```



```

273 pm_i = 0 : pm_size = 2 : #pm_val = fn_len : GOSUB fn_param
274 #fn_txlen = fn_len
275 GOSUB fn_transact
276 END
277
278 '-----
279 ' net_close
280 '-----
281 net_close: PROCEDURE
282   mb_dev = NET_DEVICEID
283   mb_cmd = NETCMD_CLOSE
284   mb_nparam = 0
285   #fn_txlen = 0
286   GOSUB fn_transact
287 END
288
289 '-----
290 ' api_call: full round trip -- open the URL staged at FN_TX/#fn_txlen,
291 ' check status, read up to #net_readlen bytes (caller sets this to the max
292 ' expected reply size, e.g. 418 for /state, 361 for /tables), close.
293 ' Leaves fn_ok = 1 and the reply in FN_RX on success.
294 '-----
295 ' ac_i/#ac_prev: loop counter and previous STATUS reading for api_call's
296 ' stabilization poll (see below). 6
297 DIM ac_i
298 DIM #ac_prev
299 api_call: PROCEDURE
300   GOSUB net_open
301   IF fn_ok = 0 THEN RETURN
302
303   ' The RP2040/ESP32 can report STATUS as soon as *some* bytes of the
304   ' real internet response have arrived, well before the full response
305   ' does -- open+status+read all happening within the same poll can
306   ' easily race ahead of that. Rather than guess a fixed delay, poll
307   ' STATUS repeatedly and treat two consecutive equal (nonzero)
308   ' readings as "the download has settled" before committing to a
309   ' read length. Bounded to ~20 frames (~0.33s) of polling so a
310   ' genuinely stuck connection still falls through to the normal
311   ' fn_ok=0/timout handling instead of hanging here.
312   #ac_prev = 0
313   FOR ac_i = 0 TO 19
314     WAIT
315     GOSUB net_status
316     IF fn_ok = 0 THEN RETURN
317     IF #net_avail > 0 AND #net_avail = #ac_prev THEN EXIT FOR
318     #ac_prev = #net_avail
319   NEXT ac_i
320
321   IF #net_avail < #net_readlen THEN #net_readlen = #net_avail
322
323   GOSUB net_read
324   GOSUB net_close ' close regardless of read result

```

```

325 END
326
327 '-----
328 ' AppKey. The wire struct (fujiDevice.h's 'struct appkey', packed) is 6
329 ' bytes: creator_lo, creator_hi, app, key, mode, reserved. mode: 0=read,
330 ' 1=write. Sending only 5 bytes (omitting reserved) leaves the firmware's
331 ' transaction get() blocked waiting for a byte that never arrives, which
332 ' reads back as a timeout, not a protocol error -- easy to misdiagnose.
333 ' appkey open must be followed by appkey_read or appkey_write, then
334 ' appkey_close, mirroring the C client's read_appkey()/write_appkey().
335 '-----
336 DIM ak_creator_lo, ak_creator_hi, ak_app, ak_key, ak_mode
337
338 appkey_open: PROCEDURE
339   mb_dev = FUJI_DEVICEID
340   mb_cmd = FUJICMD_OPEN_APPKEY
341   mb_nparam = 0
342   POKE (FN_TX + 0), ak_creator_lo
343   POKE (FN_TX + 1), ak_creator_hi
344   POKE (FN_TX + 2), ak_app
345   POKE (FN_TX + 3), ak_key
346   POKE (FN_TX + 4), ak_mode
347   POKE (FN_TX + 5), 0 ' reserved
348   #fn_txlen = 6
349   GOSUB fn_transact
350 END
351
352 ' Reads into SC_names buffer at address #fn_src (caller sets), up to
353 ' fn_len bytes. Actual byte count returned in fn_len after the call (from
354 ' RXLEN); callers should NUL-terminate at that offset.
355 ' Caller sets #fn_src (destination) and ls_max (destination buffer size,
356 ' including room for the NUL this always writes at fn_len). Clamps to 7
357 ' ls_max-1 bytes copied -- appkeys can be up to 64 bytes but destinations
358 ' like SC_NAME are much smaller, and always NUL-terminates at fn_len so a
359 ' short stored value doesn't leave trailing bytes undefined for callers
360 ' (like compose_url via fn_strlen) that scan for the terminator.
361 appkey_read: PROCEDURE
362   mb_dev = FUJI_DEVICEID
363   mb_cmd = FUJICMD_READ_APPKEY
364   mb_nparam = 0
365   #fn_txlen = 0
366   GOSUB fn_transact
367   IF fn_ok THEN
368     ' The rs232 transport (rs232Fuji::fujicore_read_app_key, verified
369     ' against fujinet-firmware source and a live byte dump) prepends
370     ' a 2-byte little-endian length ahead of the actual appkey bytes
371     ' -- unlike a network READ, an appkey READ takes no length
372     ' parameter, so the firmware makes the reply self-describing
373     ' instead. This is specific to appkey reads on this transport;
374     ' network reads carry no such prefix. RXLEN (FN_RXLEN_LO/HI)
375     ' covers the prefix + data together, so use the embedded prefix
376     ' itself as the real data length and skip past it.

```

```

377     fn_len = (PEEK(FN_RX) AND 255) + (PEEK(FN_RX + 1) AND 255) * 256
378     IF fn_len > ls_max - 1 THEN fn_len = ls_max - 1
379     FOR fn_i = 0 TO fn_len - 1
380         POKE (#fn_src + fn_i), PEEK(FN_RX + 2 + fn_i) AND 255
381     NEXT fn_i
382     POKE (#fn_src + fn_len), 0
383 ELSE
384     fn_len = 0
385 END IF
386 END
387
388 ' Writes fn_len bytes from #fn_src (an SC_buffer) as the appkey payload.
389 appkey_write: PROCEDURE
390     mb_dev = FUJI_DEVICEID
391     mb_cmd = FUJICMD_WRITE_APPKEY
392     mb_nparam = 0
393     FOR fn_i = 0 TO fn_len - 1
394         POKE (FN_TX + fn_i), PEEK(#fn_src + fn_i) AND 255
395     NEXT fn_i
396     #fn_txlen = fn_len
397     GOSUB fn_transact
398 END
399
400 appkey_close: PROCEDURE
401     mb_dev = FUJI_DEVICEID
402     mb_cmd = FUJICMD_CLOSE_APPKEY
403     mb_nparam = 0
404     #fn_txlen = 0
405     GOSUB fn_transact
406 END
407
408 ' -----
409 ' fn_putnum: append the decimal (no leading zeros) representation of pn_val
410 ' (0-999) into FN_TX at the current #fn_txlen. IntyBASIC has no built-in
411 ' itoa; three digits covers every value the game servers take in a URL
412 ' (board positions, score indices, ship placements).
413 ' -----
414 DIM pn_val, pn_h, pn_t, pn_o, pn_started
415 fn_putnum: PROCEDURE
416     pn_h = pn_val / 100
417     pn_t = (pn_val / 10) % 10
418     pn_o = pn_val % 10
419     pn_started = 0
420     IF pn_h > 0 THEN
421         POKE (FN_TX + #fn_txlen), pn_h + 48 : #fn_txlen = #fn_txlen + 1
422         pn_started = 1
423     END IF
424     IF pn_t > 0 OR pn_started THEN
425         POKE (FN_TX + #fn_txlen), pn_t + 48 : #fn_txlen = #fn_txlen + 1
426     END IF
427     POKE (FN_TX + #fn_txlen), pn_o + 48 : #fn_txlen = #fn_txlen + 1
428 END
429

```

LISTING 2 Scard.bas — 5 Card Stud client (Chapter 8)

```

1 ' Scard.bas -- 5 Card Stud for Intellivision, entry point + state machine.
2 '
3 ' Screen flow (simplified from the C clients' full parity, see the plan's
4 ' Risks section): name entry (disc letter picker, falls back to it whenever
5 ' the appkey read fails or comes back empty) -> table select (always shown
6 ' at boot; the server endpoint is a compiled-in default rather than an
7 ' appkey-persisted URL, since table select runs every boot anyway) -> game
8 ' loop -> in-game menu (keypad CLEAR: quit table or resume).
9 '
10 ' Rendering only fully clears the screen when the table layout actually
11 ' changes (see render_game); otherwise it updates cells in place, matching
12 ' the CoCo/MSX C clients' SINGLE_BUFFER mode -- IntyBASIC has no true
13 ' page-flip double buffering, so avoiding a per-poll CLS is what keeps the
14 ' screen from flashing blank on every state refresh.
15 '
16 ' GOTO boot_start jumps over every INCLUDE below before any of it runs.
17 ' This isn't optional: fujinet.bas/gfx.bas are libraries whose PROCEDURE
18 ' (and gfx.bas's DATA) bodies are meant to be reached only via GOSUB, per
19 ' the IntyBASIC manual's warning that falling into a PROCEDURE or DATA
20 ' block by straight-line execution corrupts the return stack. Since
21 ' INCLUDE pastes their text in at this point, and this file's own
22 ' DATA/PROCEDURE blocks (below) have the same requirement, the single
23 ' cheapest fix is to never let the CPU reach any of it except via GOSUB --
24 ' hence jumping straight to boot_start before the includes even begin.
25 '
26 ' GOTO boot_start
27 '
28 ' INCLUDE "constants.bas"
29 ' INCLUDE "fujinet.bas"
30 ' INCLUDE "gfx.bas"
31 ' INCLUDE "state.bas"
32 ' INCLUDE "sound.bas"
33 '
34 ' CONST ROWCELLS = 20
35 ' CONST STATUS_ROW = 220
36 '
37 ' CONST COL_NAME = FG_WHITE + BG_DARKGREEN
38 ' CONST COL_HILITE = FG_YELLOW + BG_DARKGREEN
39 ' CONST COL_STATUS = FG_WHITE + BG_BLACK
40 '
41 ' Seat base offsets (name row), seat 0 is always you (bottom-center).
42 ' seat_name_off:
43 ' DATA 167, 140, 80, 20, 7, 34, 94, 154
44 '
45 ' playerCountIndex: for a given playerCount (2-8, row = playerCount-2),
46 ' the seat assigned to server player index i (array position i). 255 =
47 ' unused. Ported unchanged from the C client's seat-assignment table.
48 ' seatmap:
49 ' DATA 0,4,255,255,255,255,255,255 ' 2 players
50 ' DATA 0,2,6,255,255,255,255,255 ' 3
51 ' DATA 0,2,4,6,255,255,255,255 ' 4
52 '
53 ' DATA 0,2,3,5,6,255,255,255 ' 5
54 ' DATA 0,2,3,4,5,6,255,255 ' 6
55 ' DATA 0,2,3,4,5,6,7,255 ' 7
56 ' DATA 0,1,2,3,4,5,6,7 ' 8
57 '
58 ' lit_n_colon: DATA 78,58
59 ' lit_https: DATA 104,116,116,112,115,58,47,47,53,99,97,114,100,46,99,97,114,114,
60 ' 45,100,101,115,105,103,110,115,46,99,111,109,47
61 ' lit_tables: DATA 116,97,98,108,101,115
62 ' lit_state: DATA 115,116,97,116,101
63 ' lit_move: DATA 109,111,118,101,47
64 ' lit_leave: DATA 108,101,97,118,101
65 ' "Gbe=1" (big-endian) dropped: the Go server's binary.BigEndian path is
66 ' the road much less travelled (CoCo, 6809, is the only other client that
67 ' ever requests it -- everything else in this client family is little-
68 ' endian x86/6502/Z80), and pot/bet corruption that always landed on
69 ' exact multiples of 256 (i.e. a real small value sitting in the high
70 ' byte, low byte zero) pointed straight at a byte-order mismatch on that
71 ' path specifically. Little-endian is what the rest of the ecosystem
72 ' exercises constantly.
73 ' lit_qbin: DATA 63,98,105,110,61,49
74 ' lit_abin: DATA 38,98,105,110,61,49
75 ' lit_qtable: DATA 63,116,97,98,108,101,61
76 ' lit_qlayer: DATA 38,112,108,97,121,101,114,61
77 '
78 ' CONST LEN_HTTPS = 31
79 '
80 ' Lobby appkey: creator 1 / app 1, key = this game's registered slot.
81 ' See fujinet-Scardstud/src/misc.h AK_LOBBY_KEY_SERVER.
82 ' CONST AK_LOBBY_KEY_SERVER = 1
83 '
84 ' Selected table id, 9 bytes (8 + NUL). SC_ENDPT used to double as this
85 ' buffer, but it's now needed for the full lobby-supplied endpoint, so
86 ' the table id gets its own slot -- free RAM just past fujinet.bas's
87 ' own SC.* buffers (which stop at SC_QUERY $9150 + 48 = $9180) and
88 ' well below the $9C00 mailbox.
89 ' CONST SC_TABLE = $9180
90 '
91 ' Globals
92 '
93 ' DIM gs_i, gs_j, #gs_c
94 ' DIM ne_cur, ne_len
95 ' DIM ne_buf(8)
96 ' DIM tbl_count, tbl_sel
97 ' DIM sel_seat, sel_i
98 ' DIM poll_wait
99 ' DIM mv_sel, mv_count
100 ' DIM mv_col(5)
101 ' DIM inp_lock

```

```

100 DIM mvcode_a, mvcode_b
101 DIM has_move
102 DIM #tmp_addr
103 DIM #tmp_num
104 DIM #prev_pot
105 DIM #prev_purse
106 DIM prev_bet(8)
107 DIM prev_cards(8)
108 DIM deal_count
109 DIM #tmp_expect
110 DIM #tmp_hash
111 DIM #prev_result_hash
112 DIM first_poll
113
114 DIM df_pos, df_len, #df_color, df_i, df_c, df_stop
115 DIM #df_src
116
117 ' split_room_url locals (see procedure below)
118 DIM sp_i, sp_found, sp_ok, sp_j, sp_k, sp_c, sp_valid, sp_m
119
120 ' -----
121 ' draw_field: render df_len ASCII bytes from #df_src onto the screen at
122 ' BACKTAB offset df_pos, in color #df_color. Uppercases (MODE 1 has no
123 ' lowercase GROM; the wire is lowercase anyway), stops at a NUL and pads
124 ' the rest of the field with spaces, and clamps anything outside the
125 ' printable GROM range (32-95) to a space rather than drawing garbage.
126 ' -----
127
128 ' fill_bg: felt-table background -- green everywhere except the status row
129 ' (which stays black). Call right after every CLS.
130 ' -----
131 fill_bg: PROCEDURE
132   FOR gs_i = 0 TO STATUS_ROW - 1
133     #BACKTAB(gs_i) = COL_NAME
134   NEXT gs_i
135 END
136
137 draw_field: PROCEDURE
138   df_stop = 0
139   FOR df_i = 0 TO df_len - 1
140     df_c = PEEK(#df_src + df_i) AND 255
141     IF df_c = 0 THEN df_stop = 1
142     IF df_stop THEN
143       df_c = 32
144     ELSEIF df_c >= 97 AND df_c <= 122 THEN
145       df_c = df_c - 32
146     END IF
147     IF df_c < 32 OR df_c > 95 THEN df_c = 32
148     #BACKTAB(df_pos + df_i) = (df_c - 32) * 8 + #df_color
149   NEXT df_i
150 END
151
152 ' -----

```

```

153 ' compose_url: build "N:<endpoint><path...><suffix>" directly into FN_TX.
154 ' Callers set gs_path (0=tables 1=state 2=move 3=leave) and, for move,
155 ' mvcode_a/mvcode_b (the 2-char move code) beforehand.
156 ' -----
157   DIM gs_path
158   compose_url: PROCEDURE 3
159     #fn_txlen = 0
160     #fn_src = VARPTR lit_n_colon(0) : fn_len = 2 : GOSUB fn_putstr
161     #fn_src = SC_ENDPT : ls_max = 65 : GOSUB fn_strlen : GOSUB fn_putstr
162
163     IF gs_path = 0 THEN
164       #fn_src = VARPTR lit_tables(0) : fn_len = 6 : GOSUB fn_putstr
165       #fn_src = VARPTR lit_qbin(0) : fn_len = 6 : GOSUB fn_putstr
166     ELSE
167       IF gs_path = 1 THEN
168         #fn_src = VARPTR lit_state(0) : fn_len = 5 : GOSUB fn_putstr
169       ELSEIF gs_path = 2 THEN
170         #fn_src = VARPTR lit_move(0) : fn_len = 5 : GOSUB fn_putstr
171         POKE (FN_TX + #fn_txlen), mvcode_a : #fn_txlen = #fn_txlen + 1
172         POKE (FN_TX + #fn_txlen), mvcode_b : #fn_txlen = #fn_txlen + 1
173       ELSE
174         #fn_src = VARPTR lit_leave(0) : fn_len = 5 : GOSUB fn_putstr
175       END IF
176
177       #fn_src = VARPTR lit_qtable(0) : fn_len = 7 : GOSUB fn_putstr
178       #fn_src = SC_TABLE : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr
179       table id, NUL-terminated within its 9-byte field
180       #fn_src = VARPTR lit_qlayer(0) : fn_len = 8 : GOSUB fn_putstr
181       #fn_src = SC_NAME : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr
182       player name, likewise
183       #fn_src = VARPTR lit_abin(0) : fn_len = 6 : GOSUB fn_putstr
184     END IF
185     ' -----
186     ' split_room_url: given a lobby-supplied "https://host/?table=xyz" value
187     ' already sitting in SC_ENDPT with its length in fn_len (as left by
188     ' appkey_read), split it into the bare endpoint (SC_ENDPT, truncated at
189     ' the '?') and the table id (SC_TABLE). Mirrors the C clients'
190     ' welcomeActionVerifyServerDetails() '?' scan. Leaves SC_TABLE empty (a
191     ' leading NUL) if there's no '?', the query isn't "table=...", or the id
192     ' contains anything other than A-Z/a-z/0-9 -- it goes straight into a
193     ' rebuilt query string unescaped, same reasoning as the username check
194     ' above.
195     ' -----
196     split_room_url: PROCEDURE
197       POKE SC_TABLE, 0
198
199       sp_found = 255 ' sentinel: not found (mirrors st_boot.bas's bt_slash
200       convention)
201       sp_i = 0
202       WHILE sp_i < fn_len

```

```

202     IF (PEEK(SC_ENDPT + sp_i) AND 255) = 63 THEN ' '?
203         sp_found = sp_i
204         EXIT WHILE
205     END IF
206     sp_i = sp_i + 1
207 WEND
208 IF sp_found = 255 THEN RETURN
209
210 ' Truncate the endpoint at the '?' regardless of what follows.
211 POKE (SC_ENDPT + sp_found), 0
212
213 ' Require "table=" (6 bytes) right after the '?'.
214 sp_ok = 0
215 IF sp_found + 7 <= fn_len THEN
216     sp_ok = 1
217     IF (PEEK(SC_ENDPT + sp_found + 1) AND 255) <> 116 THEN sp_ok = 0 ' t
218     IF (PEEK(SC_ENDPT + sp_found + 2) AND 255) <> 97 THEN sp_ok = 0 ' a
219     IF (PEEK(SC_ENDPT + sp_found + 3) AND 255) <> 98 THEN sp_ok = 0 ' b
220     IF (PEEK(SC_ENDPT + sp_found + 4) AND 255) <> 108 THEN sp_ok = 0 ' l
221     IF (PEEK(SC_ENDPT + sp_found + 5) AND 255) <> 101 THEN sp_ok = 0 ' e
222     IF (PEEK(SC_ENDPT + sp_found + 6) AND 255) <> 61 THEN sp_ok = 0 ' =
223 END IF
224 IF sp_ok = 0 THEN RETURN
225
226 ' Copy the id: up to 8 bytes, stopping at NUL or '&'.
227 sp_j = sp_found + 7
228 sp_k = 0
229 WHILE sp_k < 8 AND sp_j < fn_len
230     sp_c = PEEK(SC_ENDPT + sp_j) AND 255
231     IF sp_c = 0 OR sp_c = 38 THEN EXIT WHILE ' NUL or '&'
232     POKE (SC_TABLE + sp_k), sp_c
233     sp_k = sp_k + 1
234     sp_j = sp_j + 1
235 WEND
236 POKE (SC_TABLE + sp_k), 0
237
238 ' Validate: A-Z/a-z/0-9 only.
239 sp_valid = 1
240 FOR sp_m = 0 TO sp_k - 1
241     sp_c = PEEK(SC_TABLE + sp_m) AND 255
242     IF (sp_c < 65 OR sp_c > 90) AND (sp_c < 97 OR sp_c > 122) AND (sp_c < 48
OR sp_c > 57) THEN sp_valid = 0
243 NEXT sp_m
244 IF sp_valid = 0 THEN POKE SC_TABLE, 0
245 END
246
247 ' -----
248 ' write_room_appkey: persist SC_ENDPT + "?table=" + SC_TABLE back to the
249 ' lobby server appkey slot, so a reboot without going back through the
250 ' lobby rejoins the same room (mirrors the C clients' "Update server app
251 ' key in case of reboot"). Builds the payload directly in FN_TX and writes
252 ' it from there -- appkey_write's byte-by-byte copy from #fn_src into
253 ' FN_TX is a safe no-op when #fn_src is FN_TX itself.

```

```

254 ' -----
255 write_room_appkey: PROCEDURE
256     #fn_txlen = 0
257     #fn_src = SC_ENDPT : ls_max = 65 : GOSUB fn_strlen : GOSUB fn_putstr
258     #fn_src = VARPTR lit_qtable(0) : fn_len = 7 : GOSUB fn_putstr
259     #fn_src = SC_TABLE : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr
260
261     ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
262     ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 1
263     GOSUB appkey_open
264     IF fn_ok THEN
265         fn_len = #fn_txlen : #fn_src = FN_TX
266         GOSUB appkey_write
267         GOSUB appkey_close
268     END IF
269 END
270
271 ' -----
272 ' clear_room_appkey: blank the lobby server appkey slot on a deliberate
273 ' leave, so a later reboot lands on table select instead of silently
274 ' rejoining a table the player just quit.
275 ' -----
276 clear_room_appkey: PROCEDURE
277     ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
278     ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 1
279     GOSUB appkey_open
280     IF fn_ok THEN
281         fn_len = 0 : #fn_src = FN_TX
282         GOSUB appkey_write
283         GOSUB appkey_close
284     END IF
285     POKE SC_TABLE, 0
286 END
287
288 ' =====
289 ' Boot
290 ' =====
291 boot_start:
292     MODE 1
293     CLS
294     GOSUB fill_bg
295     GOSUB gfx_init
296
297     PRINT AT 0 COLOR COL_STATUS, "5 CARD STUD"
298     PRINT AT 20, "IN GAME, PRESS KEYPAD"
299     PRINT AT 40, "CLEAR FOR THE MENU"
300     PRINT AT 60, "CONNECTING TO FUJINET"
301     GOSUB fn_wait_mailbox
302     IF fn_ok = 0 THEN
303         PRINT AT 60, "NO CARTRIDGE MAILBOX"
304         GOTO halt
305     END IF
306

```

```

307 '-----
308 Name: try the lobby appkey first; fall back to the disc letter picker if
309 the transaction fails or the stored name is empty.
310 '-----
311 ' Every other network call in this game gets an implicit retry via the
312 ' outer poll loop; this one-shot boot-time appkey read didn't, so a
313 ' single cold-connection timeout (the RP2040/ESP32 link needing a
314 ' moment right after the mailbox first comes up -- the same thing
315 ' that made the very first transaction after boot occasionally time
316 ' out earlier in testing) meant falling back to manual name entry for
317 ' the whole session even though a name was genuinely stored. Retry
318 ' once before giving up.
319 gs_i = 0
320 FOR ak_try = 0 TO 1 1
321     ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1 : ak_key = 0 :
ak_mode = 0
322     GOSUB appkey_open
323     IF fn_ok THEN EXIT FOR
324     NEXT ak_try
325     IF fn_ok THEN
326         #fn_src = SC_NAME : ls_max = 9 : GOSUB appkey_read
327         GOSUB appkey_close
328         IF fn_len > 0 THEN
329             ' Validate every character -- the appkey slot (creator=1,
330             ' app=1, key=0) is the *shared* lobby username used by every
331             ' FujiNet client on this server, so it can hold stale or
332             ' outright incompatible data left by other testing (this
333             ' repo's own earlier test runs included). Our name-entry
334             ' screen can only ever produce A-Z/0-9, so anything outside
335             ' that -- a space, punctuation, an '&' or '?' -- is untrusted:
336             ' it goes straight into the /state URL's query string
337             ' unescaped, and a stray reserved character there is exactly
338             ' the kind of thing that gets the request rejected by the
339             ' server (seen as an HTTP 400 page landing in FN_RX, which
340             ' render_game then tries to draw as if it were game data).
341             gs_i = 1
342             FOR gs_j = 0 TO fn_len - 1
343                 gs_char = PEEK(SC_NAME + gs_j) AND 255
344                 IF gs_char >= 97 AND gs_char <= 122 THEN gs_char = gs_char - 32
345             ' fold to uppercase for the check
346             IF (gs_char < 65 OR gs_char > 90) AND (gs_char < 48 OR gs_char >
57) THEN gs_i = 0
347             NEXT gs_j
348             END IF
349             IF gs_i = 0 THEN GOSUB name_entry_screen
350
351 '-----
352 ' Room: check the lobby server appkey (creator 1 / app 1 / key
353 ' AK_LOBBY_KEY_SERVER). If the lobby wrote a room there, split it into
354 ' SC_ENDPT/SC_TABLE and skip straight past table select. Seed the
355 ' compiled-in default endpoint first so SC_ENDPT is always valid even if

```

```

356 ' the appkey is empty or the read fails -- compose_url relies on it from
357 ' here on for every request, not just /tables.
358 '-----
359     FOR gs_i = 0 TO LEN_HTTPS - 1
360         POKE (SC_ENDPT + gs_i), PEEK(VARPTR lit_https(0) + gs_i) AND 255
361     NEXT gs_i
362     POKE (SC_ENDPT + LEN_HTTPS), 0
363     POKE SC_TABLE, 0
364
365     FOR ak_try = 0 TO 1 2
366         ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
367         ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 0
368         GOSUB appkey_open
369         IF fn_ok THEN EXIT FOR
370     NEXT ak_try
371     IF fn_ok THEN
372         #fn_src = SC_ENDPT : ls_max = 65 : GOSUB appkey_read
373         GOSUB appkey_close
374         IF fn_len > 0 THEN
375             GOSUB split_room_url
376         ELSE
377             ' appkey_read always NUL-terminates at fn_len, even when
378             ' empty, which would otherwise wipe out the default just
379             ' seeded above -- restore it.
380             FOR gs_i = 0 TO LEN_HTTPS - 1
381                 POKE (SC_ENDPT + gs_i), PEEK(VARPTR lit_https(0) + gs_i) AND 255
382             NEXT gs_i
383             POKE (SC_ENDPT + LEN_HTTPS), 0
384         END IF
385     END IF
386
387     IF (PEEK(SC_TABLE) AND 255) <> 0 THEN GOTO table_joined
388
389 ' =====
390 ' Table select (re-entered after leaving a table)
391 ' =====
392 table_select:
393     CLS
394     GOSUB fill_bg
395     PRINT AT 0 COLOR COL_STATUS, "CHOOSE A TABLE"
396     PRINT AT 40, "REFRESHING..."
397
398     gs_path = 0 : GOSUB compose_url
399     #net_readlen = TABLES_MAXLEN
400     GOSUB api_call
401
402     ' As with /state: the mailbox transaction can report success (the
403     ' RP2040 relayed *a* reply) even when the HTTP request itself was
404     ' rejected -- an error page's bytes would otherwise get misread as a
405     ' Tables struct. Validate the response length against what the
406     ' reported count actually implies before trusting it.

```

```

407 IF fn_ok = 1 THEN 4
408     #tmp_expect = 1 + (PEEK(FN_RX) AND 255) * TABLE_STRIDE
409     IF #net_gotlen < #tmp_expect THEN fn_ok = 0
410 END IF
411
412 IF fn_ok = 0 THEN
413     PRINT AT 40, "SERVER UNREACHABLE "
414     ' TEMP DEBUG: show the exact outgoing URL so a repeat failure is
415     ' diagnosable from a screenshot instead of needing another round trip.
416     #df_src = FN_TX : df_pos = 60 : df_len = 20 : #df_color = COL_STATUS
417     GOSUB draw_field
418     #df_src = FN_TX + 20 : df_pos = 80 : df_len = 20 : #df_color =
COL_STATUS
419     GOSUB draw_field
420     #df_src = FN_TX + 40 : df_pos = 100 : df_len = 20 : #df_color =
COL_STATUS
421     GOSUB draw_field
422     poll_wait = 90
423     WHILE poll_wait > 0
424         poll_wait = poll_wait - 1
425     WAIT
426     WEND
427     GOTO table_select
428 END IF
429
430 tbl_count = PEEK(FN_RX) AND 255
431 IF tbl_count > 6 THEN tbl_count = 6
432 IF tbl_count = 0 THEN
433     PRINT AT 40, "NO TABLES AVAILABLE "
434     poll_wait = 90
435     WHILE poll_wait > 0
436         poll_wait = poll_wait - 1
437     WAIT
438     WEND
439     GOTO table_select
440 END IF
441
442 CLS
443 GOSUB fill_bg
444 PRINT AT 0 COLOR COL_STATUS, "CHOOSE A TABLE"
445 FOR gs_i = 0 TO tbl_count - 1
446     #tmp_addr = table_addr(gs_i)
447     #df_src = #tmp_addr + TBL_NAME : df_pos = 20 + gs_i * 20 + 1 : df_len =
14 : #df_color = COL_NAME
448     GOSUB draw_field
449     #df_src = #tmp_addr + TBL_PLAYERS : df_pos = 20 + gs_i * 20 + 15 :
df_len = 5 : #df_color = COL_NAME
450     GOSUB draw_field
451     NEXT gs_i
452
453     tbl_sel = 0
454     inp_lock = 0
455     tbl_input:
456     WAIT
457     FOR gs_i = 0 TO tbl_count - 1
458         #gs_c = COL_NAME
459         IF gs_i = tbl_sel THEN #gs_c = COL_HILITE
460             #BACKTAB(20 + gs_i * 20) = (62 - 32) * 8 + #gs_c ' '>' cursor glyph
461     NEXT gs_i
462
463     IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO tbl_input
464
465     IF CONT1.DOWN THEN
466         tbl_sel = tbl_sel + 1
467         IF tbl_sel >= tbl_count THEN tbl_sel = 0
468         inp_lock = 8
469         GOSUB sound_cursor
470         GOTO tbl_input
471     END IF
472     IF CONT1.UP THEN
473         IF tbl_sel = 0 THEN tbl_sel = tbl_count
474         tbl_sel = tbl_sel - 1
475         inp_lock = 8
476         GOSUB sound_cursor
477         GOTO tbl_input
478     END IF
479     IF CONT1.BUTTON = 0 THEN GOTO tbl_input
480     GOSUB sound_select
481
482     #tmp_addr = table_addr(tbl_sel)
483     FOR gs_i = 0 TO 8
484         POKE (SC_TABLE + gs_i), PEEK(#tmp_addr + TBL_ID + gs_i) AND 255
485     NEXT gs_i
486
487     ' Update the lobby server appkey so a reboot without going back
488     ' through the lobby rejoins this table.
489     GOSUB write_room_appkey
490
491     table_joined:
492     ' =====
493     ' Main game loop
494     ' =====
495     GOSUB sound_join
496     has_move = 0
497     poll_wait = 0
498     force_redraw = 0
499     prev_round = 255 ' sentinel: forces a full CLS on the first render_game
500     prev_playercount = 255
501     #prev_pot = 65535
502     #prev_purse = 65535
503     #prev_result_hash = 65535 ' sentinel: max possible real hash is 20*255=5100
504     first_poll = 1 ' arm the baseline on the first poll instead of showing it --
505     ' lastResult can already hold a message from a hand that
506     ' finished before we joined/sat down, which isn't "new"
507     FOR gs_i = 0 TO 7

```

```

508     prev_bet(gs_i) = 255
509     prev_cards(gs_i) = 0
510     NEXT gs_i
511
512 game_loop:
513     WAIT
514
515     IF poll_wait > 0 THEN
516         poll_wait = poll_wait - 1
517         GOTO input_check
518     END IF
519
520     IF has_move THEN
521         gs_path = 2
522     ELSE
523         gs_path = 1
524     END IF
525     GOSUB compose_url
526     #net_readlen = GAME_MAXLEN
527     GOSUB api_call
528     has_move = 0
529
530     ' A short read means FN_RX beyond #net_gotten still holds stale bytes
531     ' from a previous, larger response -- render_game trusts fixed offsets
532     ' deep into the buffer, so rendering it would show corrupted-looking
533     ' data (a garbage pot/bet value, or a stray card glyph) instead of
534     ' just skipping this poll. GAME_MINLEN alone isn't tight enough: it's
535     ' only the 0-player floor, so a 7-8 player game's response could land
536     ' anywhere from 154 bytes up to its true ~385+ bytes and still pass
537     ' that check while genuinely truncated partway through the player
538     ' array. playerCount (offset 153) is read from before where any
539     ' truncation could have happened, so it's safe to use here to compute
540     ' the *exact* expected length for this specific response.
541     ' A malformed request (e.g. a stray character somewhere it shouldn't
542     ' be) can get rejected by the server with an HTTP error page instead
543     ' of a Game struct -- our mailbox transaction still reports "OK" (the
544     ' RP2040 got *a* reply, just not the one we wanted), so round/
545     ' playerCount are the cheapest smoke test: neither should ever be
546     ' implausible for a real response, and checking them here means a
547     ' bogus response gets treated as a failed poll instead of being
548     ' handed to render_game as if it were valid.
549     IF fn_ok = 1 AND #net_gotten >= GAME_MINLEN THEN 5
550         IF (PEEK(FN_RX + GAME_ROUND) AND 255) > 5 OR (PEEK(FN_RX +
551             GAME_PLAYERCOUNT) AND 255) > 8 THEN fn_ok = 0
552         END IF
553
554         IF fn_ok = 1 AND #net_gotten >= GAME_MINLEN THEN
555             #tmp_expect = GAME_PLAYERS + (PEEK(FN_RX + GAME_PLAYERCOUNT) AND 255) *
556             PLAYER_STRIDE
557             IF #net_gotten < #tmp_expect THEN fn_ok = 0
558             END IF

```

```

558     IF fn_ok = 0 OR #net_gotten < GAME_MINLEN THEN
559         poll_wait = 30
560         GOTO input_check
561     END IF
562
563     ' Render *before* checking for a new result message: this is the one
564     ' poll whose response actually carries the showdown's revealed hands
565     ' (folded/hidden "??" flips to the real cards only for this response --
566     ' by the next poll the server has usually already started a new hand
567     ' and reset them). Rendering first means the reveal lands on screen
568     ' before the message overlay below covers the bottom two rows; doing
569     ' it the other way around (as before) meant the reveal was drawn *at
570     ' the earliest* 4 seconds later, by which point it was already gone,
571     ' so it never actually appeared.
572     GOSUB render_game
573
574     ' The server sends a one-shot message (e.g. "Fry BOT won with Pair,
575     ' Sixes") in lastResult at the end of a round/hand, but it's normally
576     ' only shown in the single-line status row -- easy to miss entirely
577     ' since bots move fast and the round can already have advanced past
578     ' it by the next poll. Detect a new (changed, non-empty) message via
579     ' a cheap byte-sum "hash" and, when one shows up, black out the
580     ' bottom two rows to display it prominently and hold it there for a
581     ' few seconds before resuming play.
582     #tmp_hash = 0
583     FOR gs_i = 0 TO 19
584         #tmp_hash = #tmp_hash + (PEEK(FN_RX + GAME_LASTRESULT + gs_i) AND 255)
585     NEXT gs_i
586     IF first_poll THEN
587         first_poll = 0
588         #prev_result_hash = #tmp_hash ' arm the baseline silently; whatever's
589                                     ' already there predates us sitting down
590     ELSEIF (PEEK(FN_RX + GAME_LASTRESULT) AND 255) <> 0 AND #tmp_hash <>
591         #prev_result_hash THEN 8
592         #prev_result_hash = #tmp_hash
593         FOR gs_i = STATUS_ROW - ROWCELLS TO STATUS_ROW + ROWCELLS - 1
594             #BACKTAB(gs_i) = COL_STATUS
595         NEXT gs_i
596         #df_src = FN_RX + GAME_LASTRESULT : df_pos = STATUS_ROW - ROWCELLS :
597         df_len = 2 * ROWCELLS : #df_color = COL_STATUS
598         GOSUB draw_field
599         GOSUB sound_gamedone
600         force_redraw = 1 ' restore the felt background under row 10 once play
601         resumes
602         poll_wait = 240
603         GOTO input_check
604     END IF
605     poll_wait = 20
606
607     IF active_player = 0 AND (PEEK(FN_RX + GAME_VIEWING) AND 255) = 0 THEN
608         GOSUB move_ui
609         IF has_move THEN poll_wait = 0

```



```

607 END IF
608
609 input_check:
610 IF CONT1.KEY = 10 THEN GOSUB ingame_menu
611 IF CONT1.KEY = 11 THEN GOSUB show_purses
612 IF want_leave THEN
613   want_leave = 0
614   GOTO table_select
615 END IF
616
617 GOTO game_loop
618
619 ' =====
620 ' render_game: full redraw of the table from the Game struct in FN_RX.
621 ' =====
622 ' -----
623 ' render_game: redraw the table from the Game struct in FN_RX.
624 ' -----
625 ' IntyBASIC/the STIC have no true page-flip (unlike the C clients' Apple2/
626 ' C64 double buffer or CoCo/MSX's SINGLE_BUFFER differential-only mode) --
627 ' BACKTAB is read live every frame, so a CLS visibly blanks the screen for
628 ' a frame before the redraw lands. Since state only changes ~once per poll
629 ' (not per frame), the fix that matters is simply not clearing the whole
630 ' screen on every poll: a full CLS only happens on the first render after
631 ' entering the game, or when the player count or hand round regresses
632 ' (i.e. a new hand started) -- both mean the previous frame's layout is no
633 ' longer valid. Every other poll only touches the specific cells whose
634 ' values can actually change (pot digits, bet digits, name/card cells,
635 ' status row), matching the CoCo/MSX approach since we're in the same
636 ' no-real-double-buffer boat they were.
637 ' -----
638 render_game: PROCEDURE 6
639   round = PEEK(FN_RX + GAME_ROUND) AND 255
640   tmp_pc = PEEK(FN_RX + GAME_PLAYERCOUNT) AND 255
641
642   IF (tmp_pc <> prev_playercount) OR (round < prev_round) OR force_redraw THEN
643     CLS
644     GOSUB fill_bg
645     ' Center block, rows 5-6: pot ("s" + value) and your own purse ("P"
646     ' + value) stacked directly beneath it. Both stay within cols 8-12
647     ' -- the one column range no seat's card art ever reaches at any
648     ' player count (the nearest cards, seat 2/6's, start at col 0/14).
649     PRINT AT 108 COLOR COL_NAME, "s"
650     PRINT AT 128 COLOR COL_NAME, "P"
651     #prev_pot = 65535 ' force the pot to redraw too on a full layout reset
652     #prev_purse = 65535 ' likewise for purse
653     force_redraw = 0
654     ' A new hand means every seat's hand is starting over -- without
655     ' this, a seat that had (say) 3 cards last hand would treat this
656     ' hand's first 3 cards as "already dealt" and skip animating them.
657     IF round < prev_round THEN
658       FOR gs_i = 0 TO 7

```

```

659       prev_cards(gs_i) = 0
660       NEXT gs_i
661     END IF
662   END IF
663
664   ' prev_playercount=255 is the boot sentinel (first render ever) -- skip
665   ' the join/leave cue then, there's nothing to compare against yet.
666   IF prev_playercount <> 255 THEN
667     IF tmp_pc > prev_playercount THEN GOSUB sound_player_join
668     IF tmp_pc < prev_playercount THEN GOSUB sound_player_left
669   END IF
670   IF round < prev_round THEN GOSUB sound_deal
671
672   prev_playercount = tmp_pc
673   prev_round = round
674
675   ' Real hardware/accurate emulation only guarantees a BACKTAB write is
676   ' tear-free if it lands within the vblank window; a full-table redraw
677   ' (dozens of cells plus several PRINT calls, each involving a slow
678   ' division loop for the digits) can run long enough to spill into
679   ' active display, and a live screenshot can catch that half-written
680   ' state -- seen as a stray non-digit glyph where only a digit could
681   ' ever legitimately be. Skipping the PRINT entirely when the value
682   ' hasn't changed since the last poll is what keeps the common-case
683   ' redraw (most fields static from one poll to the next) small enough
684   ' to actually fit in one vblank, rather than trying to chase tearing
685   ' after the fact.
686   #tmp_num = u16be(FN_RX + GAME_POT)
687   IF #tmp_num <> #prev_pot THEN
688     IF #tmp_num > #prev_pot AND #prev_pot <> 65535 THEN GOSUB sound_chip
689     PRINT AT 109 COLOR COL_NAME, <.4>#tmp_num
690     #prev_pot = #tmp_num
691   END IF
692
693   ' Your own purse (wire index 0 is always you, per the server's rotation
694   ' -- see state.bas), directly under the pot now that collapsing POT to
695   ' one line freed row 6 for it.
696   #tmp_num = u16be(player_addr(0) + PL_PURSE)
697   IF #tmp_num <> #prev_purse THEN
698     PRINT AT 129 COLOR COL_NAME, <.4>#tmp_num
699     #prev_purse = #tmp_num
700   END IF
701
702   gs_j = tmp_pc - 2
703   IF gs_j < 0 THEN gs_j = 0
704   IF gs_j > 6 THEN gs_j = 6
705
706   FOR gs_i = 0 TO tmp_pc - 1
707     sel_seat = PEEK(VARPTR seatmap(0) + gs_j * 8 + gs_i) AND 255
708     IF sel_seat <> 255 THEN
709       sel_i = PEEK(VARPTR seat_name_off(0) + sel_seat) AND 255
710
711

```

```

712 #gs_c = COL_NAME
713 IF gs_i = active_player THEN #gs_c = COL_HILITE
714
715 #tmp_addr = player_addr(gs_i)
716 #df_src = #tmp_addr + PL_NAME : df_pos = sel_i : df_len = 4 :
#df_color = #gs_c
717 GOSUB draw_field
718
719 #tmp_num = ul6be(#tmp_addr + PL_BET)
720 IF #tmp_num > 255 THEN #tmp_num = 255 ' clamp: prev_bet is an 8-bit
array
721 IF #tmp_num <> prev_bet(sel_seat) THEN
722 PRINT AT sel_i + 4 COLOR #gs_c, <2>#tmp_num
723 prev_bet(sel_seat) = #tmp_num
724 END IF
725
726 ' Cards fill each hand contiguously from index 0 (never a gap),
727 ' so the count currently dealt is just 1 + the highest nonzero
728 ' index. Comparing that against what this seat showed last
729 ' poll (prev_cards) is what tells a genuinely new card apart
730 ' from one we've already drawn on every previous poll.
731 deal_count = 0
732 FOR mv_sel = 0 TO 4
733 card = PEEK(#tmp_addr + PL_HAND + mv_sel * 2) AND 255
734 IF card <> 0 THEN deal_count = mv_sel + 1
735 NEXT mv_sel
736
737 FOR mv_sel = 0 TO deal_count - 1
738 card = PEEK(#tmp_addr + PL_HAND + mv_sel * 2) AND 255
739 suit = PEEK(#tmp_addr + PL_HAND + mv_sel * 2 + 1) AND 255
740 ' A card index this seat hasn't shown before is a fresh
741 ' deal -- the click (and its own built-in pause) stands in
742 ' for the "card landing on the table" beat, instead of
743 ' every new card just popping in silently and instantly
744 ' like the C clients' animated deal never happens here.
745 IF mv_sel >= prev_cards(sel_seat) THEN GOSUB sound_deal
746 ' card=0/suit=0 after conversion (an unrecognized wire
747 ' char, e.g. "??" for a folded/hidden hand) is itself a
748 ' valid "hidden" input to print_card -- draws the card-
749 ' back glyph, which is what keeps a folded/hidden card
750 ' from leaving stale rank art on screen instead of just
751 ' going quiet.
752 conv_in = card : GOSUB card_from_ascii : card = conv_out
753 conv_in = suit : GOSUB suit_from_ascii : suit = conv_out
754 p = sel_i + 20 + mv_sel
755 GOSUB print_card
756 NEXT mv_sel
757 prev_cards(sel_seat) = deal_count
758
759 ' One seat's worth of pokes (name + bet digits + up to 5 cards)
760 ' is small enough to reliably land within a single vblank; the
761 ' full ~8-seat redraw as one unbroken burst isn't, and that's
762 ' what let a screenshot catch a genuinely half-written frame
763
764 ' (garbage that looked like data corruption but wasn't -- nothing
765 ' in FN_RX was wrong, the *display* was mid-update). Yielding a
766 ' frame between seats bounds the tear to at most one seat's
767 ' fields at a time instead of the whole table, at the cost of
768 ' a full new-hand redraw taking up to ~8 extra frames (moot;
769 ' that only happens once per hand, not every poll).
770 WAIT
771 END IF
772 NEXT gs_i
773 GOSUB draw_status
774 END
775
776 -----
777 ' show_purses: while KEYPAD ENTER is held, overwrite every seated player's
778 ' name cells with their purse value instead. Reuses tmp_pc/seatmap/
779 ' seat_name_off exactly as render_game's own seat loop does -- tmp_pc is a
780 ' global left holding the last poll's player count, so this works between
781 ' polls without needing a fresh network round-trip. Releasing the key just
782 ' asks for a full redraw on the next poll (the same trick the win-message
783 ' overlay uses) rather than restoring each name field by hand here.
784 -----
785 show_purses: PROCEDURE
786 gs_j = tmp_pc - 2
787 IF gs_j < 0 THEN gs_j = 0
788 IF gs_j > 6 THEN gs_j = 6
789
790 FOR gs_i = 0 TO tmp_pc - 1
791 sel_seat = PEEK(VARPTR seatmap(0) + gs_j * 8 + gs_i) AND 255
792 IF sel_seat <> 255 THEN
793 sel_i = PEEK(VARPTR seat_name_off(0) + sel_seat) AND 255
794 #tmp_addr = player_addr(gs_i)
795 #tmp_num = ul6be(#tmp_addr + PL_PURSE)
796 IF #tmp_num > 9999 THEN #tmp_num = 9999
797 PRINT AT sel_i COLOR COL_HILITE, <.4>#tmp_num
798 END IF
799 NEXT gs_i
800
801 sp_wait:
802 WAIT
803 IF CONT1.KEY = 11 THEN GOTO sp_wait
804
805 force_redraw = 1 ' restore names (and everything else) on the next poll
806 END
807
808 -----
809 ' card_from_ascii / suit_from_ascii: translate the wire's lowercase ASCII
810 ' rank/suit chars (conv_in) into print_card's expected numeric codes
811 ' (conv_out; 0 = unrecognized/blank).
812 -----
813 card_from_ascii: PROCEDURE
814 IF conv_in = 50 THEN
815 conv_out = 2

```

```

816 ELSEIF conv_in = 51 THEN
817     conv_out = 3
818 ELSEIF conv_in = 52 THEN
819     conv_out = 4
820 ELSEIF conv_in = 53 THEN
821     conv_out = 5
822 ELSEIF conv_in = 54 THEN
823     conv_out = 6
824 ELSEIF conv_in = 55 THEN
825     conv_out = 7
826 ELSEIF conv_in = 56 THEN
827     conv_out = 8
828 ELSEIF conv_in = 57 THEN
829     conv_out = 9
830 ELSEIF conv_in = 116 THEN
831     conv_out = 10
832 ELSEIF conv_in = 106 THEN
833     conv_out = 11
834 ELSEIF conv_in = 113 THEN
835     conv_out = 12
836 ELSEIF conv_in = 107 THEN
837     conv_out = 13
838 ELSEIF conv_in = 97 THEN
839     conv_out = 14
840 ELSE
841     conv_out = 0
842 END IF
843 END
844
845 suit_from_ascii: PROCEDURE
846     IF conv_in = 100 THEN
847         conv_out = DIAMONDS
848     ELSEIF conv_in = 104 THEN
849         conv_out = HEARTS
850     ELSEIF conv_in = 99 THEN
851         conv_out = CLUBS
852     ELSEIF conv_in = 115 THEN
853         conv_out = SPADES
854     ELSE
855         conv_out = 0
856     END IF
857 END
858
859 '-----
860 ' draw_status: status row (row 11). Your turn is handled separately by
861 ' move_ui (which overwrites this row); otherwise show "WAIT <name>" while
862 ' someone else acts, or the truncated lastResult otherwise.
863 '-----
864 draw_status: PROCEDURE
865     IF active_player > 0 THEN
866         PRINT AT STATUS_ROW COLOR COL_STATUS, "WAIT "
867         #tmp_addr = player_addr(active_player)
868         #df_src = #tmp_addr + PL_NAME : df_pos = STATUS_ROW + 5 : df_len = 4 :

```

```

#df_color = COL_STATUS
869     GOSUB draw_field
870     ' "WAIT <name>" only fills 9 of the row's 20 cells -- without a
871     ' full CLS between polls, blank out the rest so a previous,
872     ' longer status line (the lastResult branch below, or a move
873     ' menu) can't leave stale characters past column 8.
874     FOR gs_i = STATUS_ROW + 9 TO STATUS_ROW + ROWCELLS - 1
875         #BACKTAB(gs_i) = COL_STATUS
876     NEXT gs_i
877 ELSE
878     #df_src = FN_RX + GAME_LASTRESULT : df_pos = STATUS_ROW : df_len =
ROWCELLS : #df_color = COL_STATUS
879     GOSUB draw_field
880     END IF
881 END
882
883 ' =====
884 ' move_ui: your turn. List valid moves on the status row, disc left/right
885 ' to choose, action button to submit. Sets has_move + mvcode_a/mvcode_b.
886 ' =====
887 move_ui: PROCEDURE
888     mv_count = PEEK(FN_RX + GAME_VALIDMOVECOUNT) AND 255
889     IF mv_count > 5 THEN mv_count = 5
890     IF mv_count = 0 THEN RETURN
891
892     mv_sel = 0
893     IF mv_count > 1 THEN mv_sel = 1 ' default to the second move (not Fold),
matching the C client
894     GOSUB sound_myturn
895
896     ' Without a per-poll CLS, the status row can be carrying over a
897     ' previous, wider draw_status message (lastResult fills all 20
898     ' cells) -- move_ui only pokes the exact columns its move names
899     ' occupy, so blank the whole row once up front rather than leaving
900     ' stale text past the last move.
901     FOR gs_i = STATUS_ROW TO STATUS_ROW + ROWCELLS - 1
902         #BACKTAB(gs_i) = COL_STATUS
903     NEXT gs_i
904     ' Stopwatch icon (cardbot's 7th/last glyph, screen code 276 -- the
905     ' only one of that set that's a clock face) in the last 3 columns of
906     ' the move row, poked directly rather than through print_card since
907     ' this isn't a playing card.
908     #BACKTAB(STATUS_ROW + 17) = 276 * 8 + COL_STATUS
909
910     ' moveTime (server-computed, already net of a network round-trip
911     ' grace period -- see gameLogic.go) is our countdown's starting
912     ' point; we count it down locally frame-by-frame rather than
913     ' re-polling the server every second.
914     mv_timeleft = PEEK(FN_RX + GAME_MOVEITIME) AND 255
915     mv_framecount = 0
916     PRINT AT STATUS_ROW + 18 COLOR COL_STATUS, <2>mv_timeleft
917

```

```

918   inp_lock = 0
919 mu_loop:
920   gs_j = 0
921   FOR gs_i = 0 TO mv_count - 1
922     mv_col(gs_i) = gs_j
923     #gs_c = COL_STATUS
924     IF gs_i = mv_sel THEN #gs_c = COL_HILITE
925     #tmp_addr = move_addr(gs_i)
926     #df_src = #tmp_addr + MOVE_NAME : df_pos = STATUS_ROW + gs_j : df_len =
4 : #df_color = #gs_c
927     GOSUB draw_field
928     gs_j = gs_j + 5
929     IF gs_j > 12 THEN gs_j = 12 ' clamp so the last move name never reaches
the stopwatch/timer at cols 17-19
930   NEXT gs_i
931
932   WAIT
933
934   mv_framecount = mv_framecount + 1
935   IF mv_framecount >= 60 THEN
936     mv_framecount = 0
937     IF mv_timeleft > 0 THEN mv_timeleft = mv_timeleft - 1
938     PRINT AT STATUS_ROW + 18 COLOR COL_STATUS, <2>mv_timeleft
939     ' Timed out -- submit whatever's currently highlighted (the
940     ' default move if the player never touched the disc, matching
941     ' the C clients' timeout behavior of submitting the highlighted
942     ' selection rather than always folding).
943     IF mv_timeleft = 0 THEN GOTO mu_confirm
944   END IF
945
946   IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO mu_loop
947
948   IF CONT1.RIGHT THEN
949     mv_sel = mv_sel + 1
950     IF mv_sel >= mv_count THEN mv_sel = 0
951     inp_lock = 8
952     GOSUB sound_cursor
953     GOTO mu_loop
954   END IF
955   IF CONT1.LEFT THEN
956     IF mv_sel = 0 THEN mv_sel = mv_count
957     mv_sel = mv_sel - 1
958     inp_lock = 8
959     GOSUB sound_cursor
960     GOTO mu_loop
961   END IF
962   IF CONT1.KEY = 10 THEN RETURN ' bail to in-game menu without moving
963   IF CONT1.BUTTON = 0 THEN GOTO mu_loop
964   GOSUB sound_select
965
966 mu_confirm:
967   #tmp_addr = move_addr(mv_sel)
968   mvcode_a = PEEK(#tmp_addr + MOVE_CODE) AND 255

```

```

969   mvcode_b = PEEK(#tmp_addr + MOVE_CODE + 1) AND 255
970   has_move = 1
971 END
972
973 ' =====
974 ' ingame_menu: keypad CLEAR overlay. Disc up/down to choose, action button
975 ' to confirm, matching every other screen's controls (Clear itself also
976 ' cancels, for a quick way back in without touching the disc). Defaults
977 ' to RESUME so an accidental Clear press can't quit a hand by surprise.
978 ' =====
979 ingame_menu: PROCEDURE
980   im_sel = 0
981   inp_lock = 0
982   im_loop:
983     PRINT AT 80 COLOR COL_STATUS, " "
984     PRINT AT 100 COLOR COL_STATUS, " "
985     PRINT AT 120 COLOR COL_STATUS, " "
986     PRINT AT 86 COLOR COL_STATUS, "TABLE MENU"
987     #gs_c = COL_STATUS
988     IF im_sel = 0 THEN #gs_c = COL_HILITE
989     PRINT AT 106 COLOR #gs_c, "RESUME"
990     #gs_c = COL_STATUS
991     IF im_sel = 1 THEN #gs_c = COL_HILITE
992     PRINT AT 126 COLOR #gs_c, "QUIT TABLE"
993
994     WAIT
995     IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO im_loop
996
997     IF CONT1.UP OR CONT1.DOWN THEN
998       im_sel = 1 - im_sel
999       inp_lock = 10
1000       GOSUB sound_cursor
1001       GOTO im_loop
1002     END IF
1003     IF CONT1.KEY = 10 THEN GOTO im_done ' Clear again cancels straight back to
RESUME
1004     IF CONT1.BUTTON = 0 THEN GOTO im_loop
1005     GOSUB sound_select
1006
1007     IF im_sel = 1 THEN
1008       gs_path = 3 : GOSUB compose_url
1009       #net_readlen = 8
1010       GOSUB api_call
1011       GOSUB clear_room_appkey
1012       want_leave = 1
1013     END IF
1014   im_done:
1015     force_redraw = 1
1016 END
1017
1018 ' =====
1019 ' name_entry_screen: disc letter picker, max 8 chars. Disc up/down cycles
1020 ' the character under the cursor through A-Z, 0-9, space; left/right moves

```

```

1021 ' the cursor; the action button accepts (at least 1 non-space char).
1022 ' =====
1023 name_entry_screen: PROCEDURE
1024   CLS
1025   GOSUB fill_bg
1026   PRINT AT 0 COLOR COL_STATUS, "ENTER YOUR NAME"
1027   FOR ne_i = 0 TO 7
1028     ne_buf(ne_i) = 36 ' space
1029   NEXT ne_i
1030   ne_cur = 0
1031   inp_lock = 0
1032
1033 ne_loop:
1034   FOR ne_i = 0 TO 7
1035     #gs_c = COL_NAME
1036     IF ne_i = ne_cur THEN #gs_c = COL_HILITE
1037     ne_j = ne_buf(ne_i)
1038     IF ne_j < 26 THEN
1039       gs_j = 65 + ne_j
1040     ELSEIF ne_j < 36 THEN
1041       gs_j = 48 + ne_j - 26
1042     ELSE
1043       gs_j = 95 ' underscore stands in for a visible blank
1044     END IF
1045     #BACKTAB(60 + 6 + ne_i) = (gs_j - 32) * 8 + #gs_c
1046   NEXT ne_i
1047
1048   WAIT
1049   IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO ne_loop
1050
1051   IF CONT1.RIGHT THEN
1052     ne_cur = ne_cur + 1
1053     IF ne_cur > 7 THEN ne_cur = 0
1054     inp_lock = 8
1055     GOSUB sound_cursor
1056     GOTO ne_loop
1057   END IF
1058   IF CONT1.LEFT THEN
1059     IF ne_cur = 0 THEN ne_cur = 8
1060     ne_cur = ne_cur - 1
1061     inp_lock = 8
1062     GOSUB sound_cursor
1063     GOTO ne_loop
1064   END IF
1065   IF CONT1.UP THEN
1066     ne_buf(ne_cur) = ne_buf(ne_cur) + 1
1067     IF ne_buf(ne_cur) > 36 THEN ne_buf(ne_cur) = 0
1068     inp_lock = 6
1069     GOSUB sound_cursor
1070     GOTO ne_loop
1071   END IF
1072   IF CONT1.DOWN THEN
1073     IF ne_buf(ne_cur) = 0 THEN ne_buf(ne_cur) = 37

```

```

1074     ne_buf(ne_cur) = ne_buf(ne_cur) - 1
1075     inp_lock = 6
1076     GOSUB sound_cursor
1077     GOTO ne_loop
1078   END IF
1079   IF CONT1.BUTTON = 0 THEN GOTO ne_loop
1080   GOSUB sound_select
1081
1082   ne_len = 8
1083   WHILE ne_len > 0 AND ne_buf(ne_len - 1) = 36
1084     ne_len = ne_len - 1
1085   WEND
1086   IF ne_len = 0 THEN GOTO ne_loop
1087
1088   FOR ne_i = 0 TO ne_len - 1
1089     ne_j = ne_buf(ne_i)
1090     IF ne_j < 26 THEN
1091       gs_j = 65 + ne_j
1092     ELSE
1093       gs_j = 48 + ne_j - 26
1094     END IF
1095     POKE (SC_NAME + ne_i), gs_j
1096   NEXT ne_i
1097   POKE (SC_NAME + ne_len), 0
1098
1099   ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1 : ak_key = 0 : ak_mode =
1100   1
1101   GOSUB appkey_open
1102   IF fn_ok THEN
1103     #fn_src = SC_NAME : fn_len = ne_len
1104     GOSUB appkey_write
1105     ' A failed write used to be silent -- looked identical to success
1106     ' from here, even though the name would never come back on the
1107     ' next boot (e.g. if the backend has no SD/appkey storage mounted).
1108     IF fn_ok = 0 THEN
1109       PRINT AT 100 COLOR COL_STATUS, "NAME NOT SAVED FOR "
1110       PRINT AT 120 COLOR COL_STATUS, "NEXT TIME "
1111       poll_wait = 90
1112       WHILE poll_wait > 0
1113         poll_wait = poll_wait - 1
1114       WAIT
1115     WEND
1116     END IF
1117     GOSUB appkey_close
1118   END IF
1119
1120 halt:
1121   WAIT
1122   GOTO halt
1123

```

LISTING 3 state.bas — 5 Card Stud wire format

```

1 ' state.bas -- binary Game/Tables wire format, read in place out of FN_RX.
2
3 ' Per the plan: never copy the reply into IntyBASIC variables. These are
4 ' just fixed offsets (matching server/util.go's appendFixedLengthString
5 ' binary layout, requested with ?bin=1 -- uint16 fields are little-endian,
6 ' the server's default; &be=1 was dropped after pot/bet started showing
7 ' up as an exact multiple of 256, the signature of a byte-order mismatch
8 ' on the server's binary.BigEndian path, which is exercised by only one
9 ' other client in the whole family (CoCo/6809) versus everything else
10 ' being little-endian x86/6502/Z80)
11 ' plus small DEF FN accessors that PEEK straight out of FN_RX.
12
13 '/state, /move/XX, /leave -> Game struct, up to 418 bytes:
14 ' 0 lastResult[81]
15 ' 81 round (1) 0=waiting 1-4=streets 5=showdown
16 ' 82 pot (u16 LE)
17 ' 84 activePlayer (signed, $FF = -1)
18 ' 85 moveTime (1, seconds)
19 ' 86 viewing (1, 0/1)
20 ' 87 validMoveCount (1)
21 ' 88 validMoves[5] -- always 5 slots, 13 bytes each: move[3] + name[10]
22 ' 153 playerCount (1)
23 ' 154 players[N] -- N = playerCount, 33 bytes each:
24 ' name[9] status(1) bet(u16 LE) move[8] purse(u16 LE) hand[11]
25
26 '/tables -> Tables struct:
27 ' 0 count (1)
28 ' then N x 36 bytes: table[9] name[21] players[6] (literal "cur / max")
29
30 CONST GAME_LASTRESULT = 0
31 CONST GAME_ROUND = 81
32 CONST GAME_POT = 82
33 CONST GAME_ACTIVEPLAYER = 84
34 CONST GAME_MOVETIME = 85
35 CONST GAME_VIEWING = 86
36 CONST GAME_VALIDMOVECOUNT = 87
37 CONST GAME_VALIDMOVES = 88
38 CONST GAME_PLAYERCOUNT = 153
39 CONST GAME_PLAYERS = 154
40
41 CONST MOVE_STRIDE = 13
42 CONST MOVE_CODE = 0 ' 3 bytes
43 CONST MOVE_NAME = 3 ' 10 bytes
44
45 CONST PLAYER_STRIDE = 33
46 CONST PL_NAME = 0 ' 9 bytes
47 CONST PL_STATUS = 9 ' 1 byte: 0 waiting, 1 playing, 2 folded, 3 left
48 CONST PL_BET = 10 ' u16 LE
49 CONST PL_MOVE = 12 ' 8 bytes
50 CONST PL_PURSE = 20 ' u16 LE

```

```

51 CONST PL_HAND = 22 ' 11 bytes (5 cards x 2 chars + NUL)
52
53 CONST TABLE_STRIDE = 36
54 CONST TBL_ID = 0 ' 9 bytes
55 CONST TBL_NAME = 9 ' 21 bytes
56 CONST TBL_PLAYERS = 30 ' 6 bytes, literal "cur / max"
57
58 CONST GAME_MAXLEN = 418
59 CONST GAME_MINLEN = 154 ' fixed prefix through playerCount, 0 players
60 CONST TABLES_MAXLEN = 361
61
62 ' u16be: read a little-endian uint16 at RAM address `addr` (see header note
63 ' on why this is LE despite the name -- kept as u16be to avoid renaming
64 ' every call site for what's ultimately a one-line fix).
65 ' DEF FN u16be(addr) = (PEEK(addr + 1) AND 255) * 256 + (PEEK(addr) AND 255)
66
67 ' player_addr(i): address of player i's 33-byte record in FN_RX.
68 ' DEF FN player_addr(i) = FN_RX + GAME_PLAYERS + i * PLAYER_STRIDE
69
70 ' move_addr(i): address of valid-move slot i's 13-byte record in FN_RX.
71 ' DEF FN move_addr(i) = FN_RX + GAME_VALIDMOVES + i * MOVE_STRIDE
72
73 ' table_addr(i): address of table i's 36-byte record in FN_RX (i is 0-based,
74 ' the record right after the leading count byte).
75 ' DEF FN table_addr(i) = FN_RX + 1 + i * TABLE_STRIDE
76
77 ' active_player: activePlayer as a signed value (-1..7), converting the
78 ' wire's $FF sentinel. Called as plain `active_player` (no parens -- DEF FN
79 ' with no arguments is defined and invoked without them).
80 ' DEF FN active_player = ((PEEK(FN_RX + GAME_ACTIVEPLAYER) AND 255) *
81 '-256 + (PEEK(FN_RX + GAME_ACTIVEPLAYER) AND 255))
82
83 ' draw_field is in 5card.bas -- it renders a field straight from FN_RX (or
84 ' any RAM address) onto the screen in one pass (uppercase, NUL/pad-stop,
85 ' ASCII->card-code), so there's no need to stage a separate ASCII copy here.

```

LISTING 4 gfx.bas — 5 Card Stud card art

```

1 ' gfx.bas -- card GRAM bitmaps and draw routines.
2 '
3 ' Card faces and the print_card/foreground_color rendering logic are ported
4 ' verbatim from carlsson's intv/5card-test-display.bas mock. cardtop holds
5 ' 14 GRAM images (index 0 = card back/blank, 1-13 = ranks 2..A), cardbot
6 ' holds 7 (index 0 = blank, 1-4 = suit bottoms for DIAMONDS/HEARTS/CLUBS/
7 ' SPADES, plus 2 more finishing the set) -- see print_card for the exact
8 ' index math. DEFINE loads these into GRAM slots 0-13 (screen codes 256-269)
9 ' and 14-20 (screen codes 270-276) respectively.
10
11     CONST DIAMONDS = 1
12     CONST HEARTS = 2
13     CONST CLUBS = 3
14     CONST SPADES = 4
15
16 '-----
17 ' gfx_init: load both GRAM sets. Must run once at startup, each DEFINE
18 ' followed by a WAIT (GRAM loads take effect on the next video frame; a
19 ' second DEFINE in the same frame would silently overwrite the first).
20 '-----
21 gfx_init: PROCEDURE
22     DEFINE 0, 14, cardtop : WAIT
23     DEFINE 14, 7, cardbot : WAIT
24 END
25
26 '-----
27 ' print_card: draw one card at screen cell p (top half) / p+20 (bottom
28 ' half). Inputs: p (top-left cell), card (1-13, 0 = hidden/back), suit
29 ' (1-4, DIAMONDS/HEARTS/CLUBS/SPADES).
30 '-----
31 print_card: PROCEDURE
32     #col = BG_WHITE
33     GOSUB foreground_color
34     #BACKTAB(p) = #col + (card + 256) * 8
35     #BACKTAB(p + 20) = #col + (suit + 270) * 8
36 END
37
38 '-----
39 ' foreground_color: pick #col's foreground bits for the card being drawn
40 ' (red for D/H, black for C/S, blue for a hidden card), and adjusts `card`
41 ' from its wire value (2-14) down to the 1-13 GRAM index.
42 '-----
43 foreground_color: PROCEDURE
44     IF card > 0 THEN
45         card = card - 1 ' compensate for suit runs 23456789TJQKA
46         IF suit = DIAMONDS OR suit = HEARTS THEN
47             #col = #col + FG_RED
48         ELSE
49             #col = #col + FG_BLACK
50         END IF
51     ELSE
52         #col = #col + FG_BLUE
53     END IF
54 END
55
56 cardtop:
57     BITMAP "00000000"
58     BITMAP "00..0..0"
59     BITMAP "0..0..0.."
60     BITMAP "0..0..0.."
61     BITMAP "00..0..0"
62     BITMAP "0..0..0.."
63     BITMAP "0..0..0.."
64     BITMAP "00..0..0"
65
66     BITMAP "00000000"
67     BITMAP "0....."
68     BITMAP "0.0000.."
69     BITMAP "0.....0."
70     BITMAP "0..000.."
71     BITMAP "0....."
72     BITMAP "0.0000.."
73     BITMAP "0....."
74
75     BITMAP "00000000"
76     BITMAP "0....."
77     BITMAP "0.0000.."
78     BITMAP "0.....0."
79     BITMAP "0..000.."
80     BITMAP "0.....0."
81     BITMAP "0.0000.."
82     BITMAP "0....."
83
84     BITMAP "00000000"
85     BITMAP "0....."
86     BITMAP "0..0..0.."
87     BITMAP "0..0..0.."
88     BITMAP "0.0000.."
89     BITMAP "0.....0."
90     BITMAP "0.....0."
91     BITMAP "0....."
92
93     BITMAP "00000000"
94     BITMAP "0....."
95     BITMAP "0.0000.."
96     BITMAP "0....."
97     BITMAP "0.0000.."
98     BITMAP "0.....0."
99     BITMAP "0.0000.."
100    BITMAP "0....."

```

```

101 BITMAP "00000000"
102 BITMAP "0....."
103 BITMAP "0...00..."
104 BITMAP "0..0...."
105 BITMAP "0.0000..."
106 BITMAP "0.0...0."
107 BITMAP "0..000.."
108 BITMAP "0....."
109 BITMAP "0....."
110
111 BITMAP "00000000"
112 BITMAP "0....."
113 BITMAP "0.00000.."
114 BITMAP "0.....0."
115 BITMAP "0....0.."
116 BITMAP "0...0..."
117 BITMAP "0...0..."
118 BITMAP "0....."
119
120 BITMAP "00000000"
121 BITMAP "0....."
122 BITMAP "0..000..."
123 BITMAP "0.0...0."
124 BITMAP "0..000..."
125 BITMAP "0.0...0."
126 BITMAP "0..000..."
127 BITMAP "0....."
128
129 BITMAP "00000000"
130 BITMAP "0....."
131 BITMAP "0..000..."
132 BITMAP "0.0...0."
133 BITMAP "0..0000..."
134 BITMAP "0.....0."
135 BITMAP "0..000..."
136 BITMAP "0....."
137
138 BITMAP "00000000"
139 BITMAP "0....."
140 BITMAP "0.0...0..."
141 BITMAP "0.0.0.0..."
142 BITMAP "0.0.0.0..."
143 BITMAP "0.0.0.0..."
144 BITMAP "0.0...0..."
145 BITMAP "0....."
146
147 BITMAP "00000000"
148 BITMAP "0....."
149 BITMAP "0.....0."
150 BITMAP "0.....0."
151 BITMAP "0.....0."
152 BITMAP "0.0...0..."
153 BITMAP "0..000..."
154 BITMAP "0....."
155
156 BITMAP "00000000"
157 BITMAP "0....."
158 BITMAP "0..000..."
159 BITMAP "0.0...0..."
160 BITMAP "0.0.0.0..."
161 BITMAP "0.0...0..."
162 BITMAP "0..00.0..."
163 BITMAP "0....."
164
165 BITMAP "00000000"
166 BITMAP "0....."
167 BITMAP "0.0...0..."
168 BITMAP "0.0...0..."
169 BITMAP "0.000..."
170 BITMAP "0.0...0..."
171 BITMAP "0.0...0..."
172 BITMAP "0....."
173
174 BITMAP "00000000"
175 BITMAP "0....."
176 BITMAP "0..000..."
177 BITMAP "0.0...0..."
178 BITMAP "0.00000..."
179 BITMAP "0.0...0..."
180 BITMAP "0.0...0..."
181 BITMAP "0....."
182
183 cardbot:
184 BITMAP "0.0...0..."
185 BITMAP "0.0...0..."
186 BITMAP "00..0...0"
187 BITMAP "0.0...0..."
188 BITMAP "0.0...0..."
189 BITMAP "00..0...0"
190 BITMAP "0.0...0..."
191 BITMAP "00000000"
192
193 BITMAP "0....."
194 BITMAP "0...0..."
195 BITMAP "0..000..."
196 BITMAP "0.00000..."
197 BITMAP "0..000..."
198 BITMAP "0...0..."
199 BITMAP "0....."
200 BITMAP "00000000"
201
202 BITMAP "0....."
203 BITMAP "0..0...0..."
204 BITMAP "0.00000..."
205 BITMAP "0.00000..."
206 BITMAP "0..000..."

```


APPENDIX C

```

207 BITMAP "0...0..."
208 BITMAP "0....."
209 BITMAP "00000000"
210
211 BITMAP "0....."
212 BITMAP "0..000.."
213 BITMAP "0.0.0.0."
214 BITMAP "0.00000."
215 BITMAP "0.0.0.0."
216 BITMAP "0...0..."
217 BITMAP "0....."
218 BITMAP "00000000"
219
220 BITMAP "0....."
221 BITMAP "0..000.."
222 BITMAP "0.00000."
223 BITMAP "0.00000."
224 BITMAP "0...0..."
225 BITMAP "0...0..."
226 BITMAP "0....."
227 BITMAP "00000000"
228
229 BITMAP "0....."
230 BITMAP "0....."
231 BITMAP "0....."
232 BITMAP "0....."
233 BITMAP "0....."
234 BITMAP "0....."
235 BITMAP "0....."
236 BITMAP "0....."
237
238 BITMAP ".00000.."
239 BITMAP "000.000."
240 BITMAP "000.000."
241 BITMAP "000...0."
242 BITMAP "0000000."
243 BITMAP "0000000."
244 BITMAP ".00000.."
245 BITMAP "....."
246
247

```

LISTING 5 sound.bas — 5 Card Stud effects

```

1 ' sound.bas -- sound effects, modeled on the C clients' platform-specific
2 ' sound.c implementations (src/msx/sound.c and src/plus4/sound.c gave the
3 ' clearest Hz-based reference; other platforms use raw PSG period/noise
4 ' values that amount to the same tones). Every effect there is built from
5 ' plain square-wave tones (SOUND channel 0-2, no envelope/noise), which is
6 ' exactly IntyBASIC's SOUND statement, so this is a direct transcription
7 ' rather than a reinterpretation.

```

```

8 '
9 ' All effects play on PSG channel 0 -- the game is turn-based and effects
10 ' are triggered at distinct, non-overlapping UI events, so there's no need
11 ' for polyphony (and SOUND's channel argument must be a compile-time
12 ' constant anyway, which would rule out a single generic multi-channel
13 ' helper).
14 '

```

```

15 ' SOUND's frequency argument is a 12-bit PSG divisor, computed as
16 ' round(3579545/32/hz) for NTSC (the manual's documented formula;
17 ' IntyBASIC exposes no runtime PAL/NTSC query outside the MUSIC/PLAY
18 ' engine, and this project has only ever been built/tested NTSC). These
19 ' are precomputed here rather than divided at runtime: 3579545/32 alone
20 ' is ~11861, which doesn't fit IntyBASIC's 16-bit variables (max 65535),
21 ' so it only works as a compile-time-folded literal division (a fixed Hz
22 ' baked into the expression) -- not as CONST-divided-by-a-runtime-variable,
23 ' which is what a single reusable "play Hz X" helper would need. Since
24 ' every effect here uses fixed, known-in-advance frequencies anyway,
25 ' precomputing sidesteps the overflow entirely and is cheaper besides.
26 ' 50Hz->2237 60Hz->1864 70Hz->1598 80Hz->1398 100Hz->1119
27 ' 150Hz->746 300Hz->373 311Hz->360 330Hz->339 340Hz->329
28 ' 350Hz->320 392Hz->285 415Hz->270 430Hz->260 500Hz->224
29 ' CONST SND_VOL = 12
30
31 ' DIM #snd_val, snd_gate, snd_post, snd_i
32
33 '-----
34 ' play_tone: one square-wave note on channel 0. #snd_val = precomputed PSG
35 ' divisor (see table above), snd_gate = frames the tone sounds, snd_post =
36 ' frames of silence after (both "vblank" counts, taken directly from the
37 ' reference implementations since they're already frame units at ~60Hz).
38 '-----
39 play_tone: PROCEDURE
40   SOUND 0, #snd_val, SND_VOL
41   FOR snd_i = 1 TO snd_gate
42     WAIT
43   NEXT snd_i
44   SOUND 0, 0, 0
45   FOR snd_i = 1 TO snd_post
46     WAIT
47   NEXT snd_i
48 END
49
50 ' sound_join: sit down at a table (3-tone rising figure: 430,340,500 Hz).
51 sound_join: PROCEDURE
52   #snd_val = 260 : snd_gate = 5 : snd_post = 8 : GOSUB play_tone
53   #snd_val = 329 : snd_gate = 5 : snd_post = 0 : GOSUB play_tone
54   #snd_val = 224 : snd_gate = 5 : snd_post = 0 : GOSUB play_tone
55 END
56
57 ' sound_myturn: it's your turn to move (double beep, 430,430 Hz).
58 sound_myturn: PROCEDURE
59   #snd_val = 260 : snd_gate = 4 : snd_post = 2 : GOSUB play_tone
60   #snd_val = 260 : snd_gate = 4 : snd_post = 2 : GOSUB play_tone
61 END
62
63 ' sound_gamedone: round/showdown result just landed (4-tone rising
64 ' fanfare: 311,330,392,415 Hz).
65 sound_gamedone: PROCEDURE
66   #snd_val = 360 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
67   #snd_val = 339 : snd_gate = 20 : snd_post = 0 : GOSUB play_tone
68
69   #snd_val = 285 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
70   #snd_val = 270 : snd_gate = 20 : snd_post = 0 : GOSUB play_tone
71 END
72
73 ' sound_deal: a new hand's cards have arrived (150 Hz click). The C clients
74 ' click once per card as each is dealt; we don't track per-card deal
75 ' state (cards just render as soon as they're in the fetched hand), so
76 ' this plays once per new deal instead of per card.
77 sound_deal: PROCEDURE
78   #snd_val = 746 : snd_gate = 1 : snd_post = 5 : GOSUB play_tone
79 END
80
81 ' sound_player_join: another player sits down mid-game (5-tone rising
82 ' sweep: 50,60,70,80 Hz).
83 sound_player_join: PROCEDURE
84   #snd_val = 2237 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
85   #snd_val = 1864 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
86   #snd_val = 1598 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
87   #snd_val = 1398 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
88 END
89
90 ' sound_player_left: a player leaves mid-game (falling sweep, mirror of join).
91 sound_player_left: PROCEDURE
92   #snd_val = 1398 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
93   #snd_val = 1598 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
94   #snd_val = 1864 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
95   #snd_val = 2237 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
96 END
97
98 ' sound_select: a move or table selection was confirmed (2-tone rising
99 ' chirp: 300,350 Hz).
100 sound_select: PROCEDURE
101   #snd_val = 373 : snd_gate = 3 : snd_post = 1 : GOSUB play_tone
102   #snd_val = 320 : snd_gate = 3 : snd_post = 0 : GOSUB play_tone
103 END
104
105 ' sound_cursor: menu/move cursor moved one position (300 Hz).
106 sound_cursor: PROCEDURE
107   #snd_val = 373 : snd_gate = 2 : snd_post = 0 : GOSUB play_tone
108 END
109
110 ' sound_chip: pot value increased (a bet/call/raise landed, 50 Hz). The C
111 ' clients play one ascending tone per player as chips sweep into the pot
112 ' during a dedicated collection animation; we don't animate that (pot just
113 ' updates), so this plays their first-player tone once per pot increase
114 ' as a stand-in "chip clink."
115 sound_chip: PROCEDURE
116   #snd_val = 2237 : snd_gate = 2 : snd_post = 2 : GOSUB play_tone
117 END

```

LISTING 6 battleship.bas — Battleship client (Chapter 9)

```

1 ' battleship.bas -- FujiNet Battleship for Intellivision, entry point +
2 ' state machine. Follows the pattern established by fujinet-5cardstud/intv/
3 ' 5card.bas (name entry -> table select -> game loop -> in-game menu), but
4 ' the board itself is rendered with board.bas's colored-squares kernel
5 ' instead of GRAM card art, since four 10x10 grids need to be on screen at
6 ' once (Battleship's core multiplayer rule is that one shot hits every
7 ' opposing board simultaneously).
8
9 ' GOTO boot_start jumps over every INCLUDE below before any of it runs --
10 ' see fujinet.bas/5card.bas for why: falling into a PROCEDURE or DATA block
11 ' by straight-line execution corrupts the return stack.
12 ' GOTO boot_start
13
14 ' INCLUDE "constants.bas"
15 ' INCLUDE "fujinet.bas"
16 ' INCLUDE "state.bas"
17 ' INCLUDE "board.bas"
18 ' INCLUDE "sound.bas"
19
20 ' CONST ROWCELLS = 20
21 ' CONST STATUS_ROW = 220 ' row 11 (screenpos(0,11))
22 ' CONST PANEL_COL = 11 ' right-hand chrome panel starts at column 11
23
24 ' CONST COL_TEXT = CS_WHITE
25 ' CONST COL_HILITE = CS_YELLOW
26
27 ' Scratch RAM, ours, below fujinet.bas's own SC_* buffers (which stop at
28 ' SC_QUERY $9150 + 48 = $9180) and well below the $9C00 mailbox.
29 ' CONST SC_TABLE = $9180 ' selected table id, 9 bytes (8 + NUL)
30 ' CONST SC_MYSHIPS = $9190 ' our 5 placed ship values (cell+100*dir), 1
byte each
31 ' CONST SC_OCCUPY = $91A0 ' 100 bytes, placement-time occupancy map (0/1)
32 ' CONST SC_PREVFIELD = $9200 ' 4 x 100 bytes, shadow gamefields for diffing
33
34 ' -----
35 ' URL literals (ASCII DATA). url_path selects the endpoint:
36 ' 0 tables 1 state 2 ready 3 place 4 attack 5 leave
37 ' -----
38 lit_n_colon: DATA 78,58
39 lit_https: DATA 104,116,116,112,115,58,47,47,98,97,116,116,108,101,115,104,105,
112,46,99,97,114,114,45,100,101,115,105,103,110,115,46,99,111,109,47
40 lit_tables: DATA 116,97,98,108,101,115
41 lit_qbin: DATA 63,98,105,110,61,49
42 lit_state: DATA 115,116,97,116,101
43 lit_ready: DATA 114,101,97,100,121
44 lit_place: DATA 112,108,97,99,101,47
45 lit_attack: DATA 97,116,116,97,99,107,47
46 lit_leave: DATA 108,101,97,118,101
47 lit_qtable: DATA 63,116,97,98,108,101,61
48 lit_aphayer: DATA 38,112,108,97,121,101,114,61
49 lit_abinv2: DATA 38,98,105,110,61,49,38,118,118,61,50 2
50 lit_comma: DATA 44
51
52 ' CONST LEN_HTTPS = 36
53
54 ' Lobby appkey: creator 1 / app 1, key = this game's registered slot.
55 ' See fujinet-battleship/src/misc.h AK_LOBBY_KEY_SERVER.
56 ' CONST AK_LOBBY_KEY_SERVER = 5
57
58 ' -----
59 ' Globals
60 ' -----
61 DIM gs_i, gs_j, gs_char, #gs_c
62 DIM ne_i, ne_j, ne_cur, ne_len
63 DIM ne_buf(8)
64 DIM ak_try
65 DIM tbl_count, tbl_sel
66 DIM inp_lock
67 DIM want_leave, im_sel
68 DIM url_path
69 DIM #tmp_addr, #tmp_field
70
71 DIM df_pos, df_len, #df_color, df_i, df_c, df_stop
72 DIM #df_src
73
74 ' split_room_url locals (see procedure below)
75 DIM sp_i, sp_found, sp_ok, sp_j, sp_k, sp_c, sp_valid, sp_m
76
77 DIM cu_i
78 DIM pn_val, pn_h, pn_t, pn_o, pn_started
79
80 DIM cur_status, cur_pc, cur_pstatus, prev_status, ships_drawn, has_action
81 DIM atk_pos, has_targeted
82 DIM prev_seen_pc, prev_result_status, prev_result_pos, rb_lastattack
83
84 DIM pc_x, pc_y, pc_dir, pc_size, pc_i, pc_ok, pc_moved
85 DIM pc_prevx, pc_prevy, pc_prevdir, pc_cx, pc_cy, pc_mx, pc_my
86 DIM ship_idx, ph_shipnum
87
88 DIM rb_i, rb_j, rb_c, rb_state, rb_pstatus, rb_ch
89 DIM #rb_field
90 DIM rb_ship, rb_sx, rb_sy, rb_sdir, rb_ssize
91 DIM ds_bq, ds_base, ds_color
92
93 DIM tg_x, tg_y, tg_newx, tg_newy, tg_moved, tg_timeleft, tg_framecount
94 DIM tc_q, tc_state, tc_color, tc_field
95
96 ' -----
97 ' draw_field: render df_len ASCII bytes from #df_src onto the screen at

```

```

98 ' BACKTAB offset df_pos, in color #df_color. MODE 0 gives full GROM (cards
99 ' 0-255), so unlike 5cardstud's MODE 1 client, lowercase is available --
100 ' server prompts/names arrive lowercased and are shown as-is. Stops at
101 ' NUL and pads the rest of the field with spaces; clamps anything outside
102 ' the printable range to a space rather than drawing garbage.
103 -----
104 draw_field: PROCEDURE
105   df_stop = 0
106   FOR df_i = 0 TO df_len - 1
107     df_c = PEEK(#df_src + df_i) AND 255
108     IF df_c = 0 THEN df_stop = 1
109     IF df_stop THEN df_c = 32
110     IF df_c < 32 OR df_c > 127 THEN df_c = 32
111     #BACKTAB(df_pos + df_i) = (df_c - 32) * 8 + #df_color
112   NEXT df_i
113 END
114
115 '
116 ' fn_putnum: append the decimal (no leading zeros) representation of pn_val
117 ' (0-999) into FN_TX at the current #fn_txlen. IntyBASIC has no built-in
118 ' itoa; placement values run 0-199 and attack positions 0-99, so three
119 ' digits is generous headroom.
120 -----
121 fn_putnum: PROCEDURE
122   pn_h = pn_val / 100
123   pn_t = (pn_val / 10) % 10
124   pn_o = pn_val % 10
125   pn_started = 0
126   IF pn_h > 0 THEN
127     POKE (FN_TX + #fn_txlen), pn_h + 48 : #fn_txlen = #fn_txlen + 1
128     pn_started = 1
129   END IF
130   IF pn_t > 0 OR pn_started THEN
131     POKE (FN_TX + #fn_txlen), pn_t + 48 : #fn_txlen = #fn_txlen + 1
132   END IF
133   POKE (FN_TX + #fn_txlen), pn_o + 48 : #fn_txlen = #fn_txlen + 1
134 END
135
136 '
137 ' compose_url: build "N:<endpoint><path...><suffix>" directly into FN_TX.
138 ' For url_path=3 (place), SC_MYSHIPS's 5 bytes are comma-joined. For
139 ' url_path=4 (attack), atk_pos is appended. Always requests &bin=1&w=2 --
140 ' v=1 omits the winner's ships and forces activePlayer=-1 at game over,
141 ' both of which render_gameover needs.
142 -----
143 compose_url: PROCEDURE
144   #fn_txlen = 0
145   #fn_src = VARPTR lit_n_colon(0) : fn_len = 2 : GOSUB fn_putstr
146   #fn_src = SC_ENDPT : ls_max = 65 : GOSUB fn_strlen : GOSUB fn_putstr
147
148   IF url_path = 0 THEN
149     #fn_src = VARPTR lit_tables(0) : fn_len = 6 : GOSUB fn_putstr
150     #fn_src = VARPTR lit_qbin(0) : fn_len = 6 : GOSUB fn_putstr
151
152   ELSE
153     IF url_path = 1 THEN
154       #fn_src = VARPTR lit_state(0) : fn_len = 5 : GOSUB fn_putstr
155     ELSEIF url_path = 2 THEN
156       #fn_src = VARPTR lit_ready(0) : fn_len = 5 : GOSUB fn_putstr
157     ELSEIF url_path = 3 THEN
158       #fn_src = VARPTR lit_place(0) : fn_len = 6 : GOSUB fn_putstr
159       FOR cu_i = 0 TO 4
160         pn_val = PEEK(SC_MYSHIPS + cu_i) AND 255
161         GOSUB fn_putnum
162         IF cu_i < 4 THEN
163           #fn_src = VARPTR lit_comma(0) : fn_len = 1 : GOSUB fn_putstr
164         END IF
165       NEXT cu_i
166     ELSEIF url_path = 4 THEN
167       #fn_src = VARPTR lit_attack(0) : fn_len = 7 : GOSUB fn_putstr
168       pn_val = atk_pos
169       GOSUB fn_putnum
170     ELSE
171       #fn_src = VARPTR lit_leave(0) : fn_len = 5 : GOSUB fn_putstr
172     END IF
173
174     #fn_src = VARPTR lit_qtable(0) : fn_len = 7 : GOSUB fn_putstr
175     #fn_src = SC_TABLE : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr
176     #fn_src = VARPTR lit_ahplayer(0) : fn_len = 8 : GOSUB fn_putstr
177     #fn_src = SC_NAME : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr
178     #fn_src = VARPTR lit_abinv2(0) : fn_len = 10 : GOSUB fn_putstr
179   END IF
180
181 ' -----
182 ' split_room url: given a lobby-supplied "https://host/?table=xyz" value
183 ' already sitting in SC_ENDPT with its length in fn_len (as left by
184 ' apkey_read), split it into the bare endpoint (SC_ENDPT, truncated at
185 ' the '?') and the table id (SC_TABLE). Mirrors the C clients'
186 ' welcomeActionVerifyServerDetails() '?' scan. Leaves SC_TABLE empty (a
187 ' leading NUL) if there's no '?', the query isn't "table=...", or the id
188 ' contains anything other than A-Z/a-z/0-9 -- it goes straight into a
189 ' rebuilt query string unescaped, same reasoning as the username check
190 ' above.
191 -----
192 split_room_url: PROCEDURE
193   POKE SC_TABLE, 0
194
195   sp_found = 255 ' sentinel: not found
196   sp_i = 0
197   WHILE sp_i < fn_len
198     IF (PEEK(SC_ENDPT + sp_i) AND 255) = 63 THEN ' '?'
199       sp_found = sp_i
200       EXIT WHILE
201     END IF
202     sp_i = sp_i + 1
203   WEND

```

```

204 IF sp_found = 255 THEN RETURN
205
206 ' Truncate the endpoint at the '?' regardless of what follows.
207 POKE (SC_ENDPT + sp_found), 0
208
209 ' Require "table=" (6 bytes) right after the '?'.
210 sp_ok = 0
211 IF sp_found + 7 <= fn_len THEN
212     sp_ok = 1
213     IF (PEEK(SC_ENDPT + sp_found + 1) AND 255) <> 116 THEN sp_ok = 0 ' t
214     IF (PEEK(SC_ENDPT + sp_found + 2) AND 255) <> 97 THEN sp_ok = 0 ' a
215     IF (PEEK(SC_ENDPT + sp_found + 3) AND 255) <> 98 THEN sp_ok = 0 ' b
216     IF (PEEK(SC_ENDPT + sp_found + 4) AND 255) <> 108 THEN sp_ok = 0 ' l
217     IF (PEEK(SC_ENDPT + sp_found + 5) AND 255) <> 101 THEN sp_ok = 0 ' e
218     IF (PEEK(SC_ENDPT + sp_found + 6) AND 255) <> 61 THEN sp_ok = 0 ' =
219 END IF
220 IF sp_ok = 0 THEN RETURN
221
222 ' Copy the id: up to 8 bytes, stopping at NUL or '&'.
223 sp_j = sp_found + 7
224 sp_k = 0
225 WHILE sp_k < 8 AND sp_j < fn_len
226     sp_c = PEEK(SC_ENDPT + sp_j) AND 255
227     IF sp_c = 0 OR sp_c = 38 THEN EXIT WHILE ' NUL or '&'
228     POKE (SC_TABLE + sp_k), sp_c
229     sp_k = sp_k + 1
230     sp_j = sp_j + 1
231 WEND
232 POKE (SC_TABLE + sp_k), 0
233
234 ' Validate: A-Z/a-z/0-9 only.
235 sp_valid = 1
236 FOR sp_m = 0 TO sp_k - 1
237     sp_c = PEEK(SC_TABLE + sp_m) AND 255
238     IF (sp_c < 65 OR sp_c > 90) AND (sp_c < 97 OR sp_c > 122) AND (sp_c < 48
OR sp_c > 57) THEN sp_valid = 0
239 NEXT sp_m
240 IF sp_valid = 0 THEN POKE SC_TABLE, 0
241 END
242
243 '-----
244 ' write_room_appkey: persist SC_ENDPT + "?table=" + SC_TABLE back to the
245 ' lobby server appkey slot, so a reboot without going back through the
246 ' lobby rejoins the same room (mirrors the C clients' "Update server app
247 ' key in case of reboot"). Builds the payload directly in FN_TX and writes
248 ' it from there -- appkey_write's byte-by-byte copy from #fn_src into
249 ' FN_TX is a safe no-op when #fn_src is FN_TX itself.
250 '-----
251 write_room_appkey: PROCEDURE
252     #fn_txlen = 0
253     #fn_src = SC_ENDPT : ls_max = 65 : GOSUB fn_strlen : GOSUB fn_putstr
254     #fn_src = VARPTR lit_qtable(0) : fn_len = 7 : GOSUB fn_putstr
255     #fn_src = SC_TABLE : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr

```

```

256
257     ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
258     ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 1
259     GOSUB appkey_open
260     IF fn_ok THEN
261         fn_len = #fn_txlen : #fn_src = FN_TX
262         GOSUB appkey_write
263         GOSUB appkey_close
264     END IF
265 END
266
267 '-----
268 ' clear_room_appkey: blank the lobby server appkey slot on a deliberate
269 ' leave, so a later reboot lands on table select instead of silently
270 ' rejoining a table the player just quit.
271 '-----
272 clear_room_appkey: PROCEDURE
273     ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
274     ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 1
275     GOSUB appkey_open
276     IF fn_ok THEN
277         fn_len = 0 : #fn_src = FN_TX
278         GOSUB appkey_write
279         GOSUB appkey_close
280     END IF
281     POKE SC_TABLE, 0
282 END
283
284 ' =====
285 ' Boot
286 ' =====
287 boot_start:
288     MODE 0, CS_BLACK, CS_BLACK, CS_BLACK, CS_BLACK : WAIT
289     CLS
290     PRINT AT 0 COLOR COL_TEXT, "BATTLESHIP"
291     PRINT AT 40, "CONNECTING TO FUJINET"
292     GOSUB fn_wait_mailbox
293     IF fn_ok = 0 THEN
294         PRINT AT 40, "NO CARTRIDGE MAILBOX"
295         GOTO halt
296     END IF
297
298 '-----
299 ' Name: try the shared lobby appkey (creator 1 / app 1 / key 0) first, with
300 ' one retry (a cold RP2040/ESP32 link right after the mailbox comes up can
301 ' time out the very first transaction); fall back to the disc letter picker
302 ' if the read fails, comes back empty, or holds anything outside A-Z0-9 --
303 ' this slot is shared by every FujiNet client, and a stray character here
304 ' goes straight into the query string unescaped.
305 '-----
306     gs_i = 0
307     FOR ak_try = 0 TO 1
308         ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1 : ak_key = 0 :

```

```

ak_mode = 0
309 GOSUB appkey_open
310 IF fn_ok THEN EXIT FOR
311 NEXT ak_try
312 IF fn_ok THEN
313   #fn_src = SC_NAME : ls_max = 9 : GOSUB appkey_read
314   GOSUB appkey_close
315   IF fn_len > 0 THEN
316     gs_i = 1
317     FOR gs_j = 0 TO fn_len - 1
318       gs_char = PEEK(SC_NAME + gs_j) AND 255
319       IF gs_char >= 97 AND gs_char <= 122 THEN gs_char = gs_char - 32
320       IF (gs_char < 65 OR gs_char > 90) AND (gs_char < 48 OR gs_char >
57) THEN gs_i = 0
321     NEXT gs_j
322   END IF
323 END IF
324 IF gs_i = 0 THEN GOSUB name_entry_screen
325
326 '-----
327 ' Room: check the lobby server appkey (creator 1 / app 1 / key
328 ' AK_LOBBY_KEY_SERVER). If the lobby wrote a room there, split it into
329 ' SC_ENDPT/SC_TABLE and skip straight past table select. Seed the
330 ' compiled-in default endpoint first so SC_ENDPT is always valid even if
331 ' the appkey is empty or the read fails -- compose_url relies on it from
332 ' here on for every request, not just /tables.
333 '-----
334 FOR gs_i = 0 TO LEN_HTTPS - 1
335   POKE (SC_ENDPT + gs_i), PEEK(VARPTR lit_https(0) + gs_i) AND 255
336 NEXT gs_i
337 POKE (SC_ENDPT + LEN_HTTPS), 0
338 POKE SC_TABLE, 0
339
340 FOR ak_try = 0 TO 1
341   ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
342   ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 0
343   GOSUB appkey_open
344   IF fn_ok THEN EXIT FOR
345 NEXT ak_try
346 IF fn_ok THEN
347   #fn_src = SC_ENDPT : ls_max = 65 : GOSUB appkey_read
348   GOSUB appkey_close
349   IF fn_len > 0 THEN
350     GOSUB split_room_url
351   ELSE
352     ' appkey_read always NUL-terminates at fn_len, even when
353     ' empty, which would otherwise wipe out the default just
354     ' seeded above -- restore it.
355     FOR gs_i = 0 TO LEN_HTTPS - 1
356       POKE (SC_ENDPT + gs_i), PEEK(VARPTR lit_https(0) + gs_i) AND 255
357     NEXT gs_i
358     POKE (SC_ENDPT + LEN_HTTPS), 0
359   END IF

```

```

360 END IF
361
362 IF (PEEK(SC_TABLE) AND 255) <> 0 THEN GOTO table_joined
363
364 '=====
365 ' Table select (re-entered after leaving a table)
366 '=====
367 table_select:
368 CLS
369 PRINT AT 0 COLOR COL_TEXT, "CHOOSE A TABLE"
370 PRINT AT 40, "REFRESHING..."
371
372 url_path = 0 : GOSUB compose_url
373 #net_readlen = TABLES_MAXLEN
374 GOSUB api_call
375
376 IF fn_ok = 1 THEN
377   #tmp_field = 1 + (PEEK(FN_RX) AND 255) * TABLE_STRIDE
378   IF #net_gotlen < #tmp_field THEN fn_ok = 0
379 END IF
380
381 IF fn_ok = 0 THEN
382   PRINT AT 40, "SERVER UNREACHABLE "
383   inp_lock = 90
384   WHILE inp_lock > 0
385     inp_lock = inp_lock - 1
386     WAIT
387   WEND
388   GOTO table_select
389 END IF
390
391 tbl_count = PEEK(FN_RX) AND 255
392 IF tbl_count > 8 THEN tbl_count = 8
393 IF tbl_count = 0 THEN
394   PRINT AT 40, "NO TABLES AVAILABLE "
395   inp_lock = 90
396   WHILE inp_lock > 0
397     inp_lock = inp_lock - 1
398     WAIT
399   WEND
400   GOTO table_select
401 END IF
402
403 CLS
404 PRINT AT 0 COLOR COL_TEXT, "CHOOSE A TABLE"
405 FOR gs_i = 0 TO tbl_count - 1
406   #tmp_addr = table_addr(gs_i)
407   #df_src = #tmp_addr + TBL_NAME : df_pos = 20 + gs_i * 20 + 1 : df_len =
14 : #df_color = COL_TEXT
408   GOSUB draw_field
409   #df_src = #tmp_addr + TBL_PLAYERS : df_pos = 20 + gs_i * 20 + 15 :
df_len = 5 : #df_color = COL_TEXT
410   GOSUB draw_field

```

```

411 NEXT gs_i
412
413 tbl_sel = 0
414 inp_lock = 0
415 ts_input:
416 WAIT
417 FOR gs_i = 0 TO tbl_count - 1
418 #gs_c = COL_TEXT
419 IF gs_i = tbl_sel THEN #gs_c = COL_HILITE
420 #BACKTAB(20 + gs_i * 20) = (62 - 32) * 8 + #gs_c ' '>' cursor glyph
421 NEXT gs_i
422
423 IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO ts_input
424
425 IF CONT1.DOWN THEN
426 tbl_sel = tbl_sel + 1
427 IF tbl_sel >= tbl_count THEN tbl_sel = 0
428 inp_lock = 8
429 GOSUB sound_cursor
430 GOTO ts_input
431 END IF
432 IF CONT1.UP THEN
433 IF tbl_sel = 0 THEN tbl_sel = tbl_count
434 tbl_sel = tbl_sel - 1
435 inp_lock = 8
436 GOSUB sound_cursor
437 GOTO ts_input
438 END IF
439 IF CONT1.BUTTON = 0 THEN GOTO ts_input
440 GOSUB sound_select
441
442 #tmp_addr = table_addr(tbl_sel)
443 FOR gs_i = 0 TO 8
444 POKE (SC_TABLE + gs_i), PEEK(#tmp_addr + TBL_ID + gs_i) AND 255
445 NEXT gs_i
446
447 ' Update the lobby server appkey so a reboot without going back
448 ' through the lobby rejoins this table.
449 GOSUB write_room_appkey
450
451 table_joined:
452 ' =====
453 ' In a table: poll /state (or submit a pending ready/place/attack action in
454 ' its place) and dispatch on status. A full CLS + board_init happens only
455 ' on the transition out of the lobby (or on first entry) -- every other
456 ' poll updates in place, matching the differential-render approach 5 Card
457 ' Stud settled on for the same reason (IntyBASIC/the STIC have no true
458 ' page-flip; a per-poll CLS would visibly blank the screen every ~1s).
459 ' =====
460 prev_status = 255 ' sentinel: forces the initial CLS/board_init
461 prev_seen_pc = 255 ' sentinel: skip the join/leave cue on the first poll
462 prev_result_status = 255
463 prev_result_pos = 255

```

```

464 poll_wait = 0
465 want_leave = 0
466 has_action = 0
467 GOSUB sound_join
468
469 tl_loop:
470 WAIT
471
472 IF poll_wait > 0 THEN
473 poll_wait = poll_wait - 1
474 GOTO tl_input
475 END IF
476
477 IF has_action = 1 THEN
478 url_path = 2
479 ELSEIF has_action = 2 THEN
480 url_path = 3
481 ELSEIF has_action = 3 THEN
482 url_path = 4
483 ELSE
484 url_path = 1
485 END IF
486 has_action = 0
487 GOSUB compose_url
488 #net_readlen = GAME_MAXLEN
489 GOSUB api_call
490 GOSUB validate_state
491
492 IF fn_ok = 0 THEN
493 poll_wait = 30
494 GOTO tl_input
495 END IF
496
497 cur_status = state_status
498 cur_pc = state_playercount
499 cur_pstatus = state_playerstatus
500
501 IF prev_seen_pc <> 255 THEN
502 IF cur_pc > prev_seen_pc THEN GOSUB sound_player_join
503 IF cur_pc < prev_seen_pc THEN GOSUB sound_player_left
504 END IF
505 prev_seen_pc = cur_pc
506
507 IF cur_status = STATUS_LOBBY THEN
508 IF prev_status <> STATUS_LOBBY THEN CLS
509 GOSUB render_lobby
510 poll_wait = 45
511 ELSEIF cur_status = STATUS_GAMEOVER THEN
512 IF prev_status = STATUS_LOBBY OR prev_status = 255 THEN
513 CLS
514 GOSUB board_init
515 END IF
516 IF prev_status <> STATUS_GAMEOVER THEN GOSUB render_gameover

```

```

517     poll_wait = 60
518 ELSE
519     ' STATUS_PLACE_SHIPS, GAMESTART, MISS, HIT, SUNK.
520     IF prev_status = STATUS_LOBBY OR prev_status = 255 THEN
521         CLS
522         GOSUB board_init
523         ships_drawn = 0
524     END IF
525
526     IF cur_status = STATUS_PLACE_SHIPS THEN
527         IF cur_pstatus = PSTATUS_PLACE_SHIPS THEN
528             GOSUB handle_placement
529             has_action = 2
530             poll_wait = 0
531         ELSE
532             #df_src = FN_RX + GAME_PROMPT : df_pos = STATUS_ROW : df_len =
533 ROWCELLS : #df_color = COL_TEXT
534             GOSUB draw_field
535             poll_wait = 30
536         END IF
537     ELSE
538         GOSUB render_game_board
539         GOSUB render_panel
540         #df_src = FN_RX + GAME_PROMPT : df_pos = STATUS_ROW : df_len =
541 ROWCELLS : #df_color = COL_TEXT
542         GOSUB draw_field
543
544         ' status (MISS/HIT/SUNK) reflects the *last* attack's result and
545         ' can legitimately repeat across several polls if no one has
546         ' fired since -- pair it with lastAttackPos so the sound plays
547         ' exactly once per real event, not once per poll it's visible.
548         IF cur_status = STATUS_MISS OR cur_status = STATUS_HIT OR cur_status
= STATUS_SUNK THEN 6
549             rb_lastattack = PEEK(FN_RX + GAME_LASTATTACK) AND 255
550             IF cur_status <> prev_result_status OR rb_lastattack <>
prev_result_pos THEN
551                 IF cur_status = STATUS_MISS THEN
552                     GOSUB sound_miss
553                 ELSEIF cur_status = STATUS_HIT THEN
554                     GOSUB sound_hit
555                 ELSE
556                     GOSUB sound_sink
557                 END IF
558                 prev_result_status = cur_status
559                 prev_result_pos = rb_lastattack
560             END IF
561         END IF
562
563         IF active_player = 0 AND cur_pstatus = PSTATUS_PLAYING THEN
564             GOSUB targeting
565             IF has_targeted THEN
566                 GOSUB sound_myturn

```

```

565             has_action = 3
566             poll_wait = 0
567         ELSE
568             poll_wait = 20
569         END IF
570     ELSE
571         poll_wait = 20
572     END IF
573 END IF
574
575     prev_status = cur_status
576
577     tl_input:
578     IF CONT1.KEY = 10 THEN GOSUB ingame_menu
579     IF want_leave THEN
580         want_leave = 0
581         GOTO table_select
582     END IF
583     GOTO tl_loop
584
585     ' =====
586     ' render_lobby: server name, ready list, prompt. Trigger toggles /ready.
587     ' =====
588     render_lobby: PROCEDURE
589     #df_src = FN_RX + LOBBY_SERVERNAME : df_pos = 20 : df_len = 20 : #df_color =
COL_TEXT
590     GOSUB draw_field
591     FOR gs_i = 0 TO cur_pc - 1
592         #tmp_addr = lobby_player_addr(gs_i)
593         #df_src = #tmp_addr + LP_NAME : df_pos = 60 + gs_i * 20 : df_len = 9 :
#df_color = COL_TEXT
594         GOSUB draw_field
595         #gs_c = 46 ' '.'
596         IF (PEEK(#tmp_addr + LP_READY) AND 255) <> 0 THEN #gs_c = 42 ' '*'
597         #BACKTAB(60 + gs_i * 20 + 10) = (#gs_c - 32) * 8 + COL_TEXT
598     NEXT gs_i
599     #df_src = FN_RX + GAME_PROMPT : df_pos = STATUS_ROW : df_len = ROWCELLS :
#df_color = COL_TEXT
600     GOSUB draw_field
601
602     IF CONT1.BUTTON THEN
603         GOSUB sound_select
604         url_path = 2 : GOSUB compose_url
605         #net_readlen = GAME_MAXLEN
606         GOSUB api_call
607         poll_wait = 15
608     END IF
609 END
610
611     ' =====
612     ' render_gameover: reveal the winner's ships (myShips[5..9]) on their
613     ' quadrant, show the prompt. Held on screen (poll_wait=60) until the server

```



```

615 ' resets the table back to the lobby.
616 ' =====
617 render_gameover: PROCEDURE
618     GOSUB sound_gamedone
619     IF active_player >= 0 AND active_player <= 3 THEN
620         ds_bq = active_player : ds_base = GAME_MYSHIPS + 5 : ds_color =
BCOL_SUNKSHIP
621         GOSUB draw_ship_set
622     END IF
623     #df_src = FN_RX + GAME_PROMPT : df_pos = STATUS_ROW : df_len = ROWCELLS :
#df_color = COL_HILITE
624     GOSUB draw_field
625 END
626
627 ' -----
628 ' draw_ship_set: paint the 5 ships encoded at FN_RX+ds_base (each byte
629 ' cell+100*dir, state.bas's myShips layout) onto quadrant ds_bq in colour
630 ' ds_color. Shared by render_game_board (own ships, ds_base=GAME_MYSHIPS)
631 ' and render_gameover (winner's ships, ds_base=GAME_MYSHIPS+5).
632 ' -----
633 draw_ship_set: PROCEDURE
634     FOR rb_i = 0 TO 4
635         rb_ship = PEEK(FN_RX + ds_base + rb_i) AND 255
636         IF rb_ship > 0 THEN
637             rb_sx = (rb_ship % 100) % 10
638             rb_sy = (rb_ship % 100) / 10
639             rb_sdir = rb_ship / 100
640             rb_ssize = ship_size(rb_i)
641             FOR rb_j = 0 TO rb_ssize - 1
642                 IF rb_sdir = 0 THEN
643                     bc_x = rb_sx + rb_j : bc_y = rb_sy
644                 ELSE
645                     bc_x = rb_sx : bc_y = rb_sy + rb_j
646                 END IF
647                 bc_bq = ds_bq : bc_color = ds_color
648                 GOSUB board_cell
649             NEXT rb_j
650         END IF
651     NEXT rb_i
652 END
653
654 ' =====
655 ' render_panel: right-hand chrome (columns 11-19). Three rows per player
656 ' (name, ships-left, blank separator) x 4 players fits rows 0-10 exactly.
657 ' Redrawn in full every poll -- cheap (36 characters max) and simpler than
658 ' diffing four short text fields.
659 ' =====
660 render_panel: PROCEDURE
661     FOR rb_i = 0 TO 3
662         #gs_c = COL_TEXT
663         IF rb_i < cur_pc THEN
664             IF rb_i = active_player THEN #gs_c = COL_HILITE
665             #df_src = player_addr(rb_i) + PL_NAME : df_pos =

```

```

screenpos(PANEL_COL, rb_i * 3) : df_len = 9 : #df_color = #gs_c
666     GOSUB draw_field
667     FOR rb_j = 0 TO 4
668         rb_state = PEEK(player_addr(rb_i) + PL_SHIPSLEFT + rb_j) AND 255
669         rb_ch = 46 ' '.' -- sunk
670         IF rb_state < 0 THEN rb_ch = 35 ' '#' -- afloat
671         #BACKTAB(screenpos(PANEL_COL + rb_j, rb_i * 3 + 1)) = (rb_ch -
32) * 8 + #gs_c
672     NEXT rb_j
673 ELSE
674     FOR rb_j = 0 TO 8
675         #BACKTAB(screenpos(PANEL_COL + rb_j, rb_i * 3)) = COL_TEXT
676         #BACKTAB(screenpos(PANEL_COL + rb_j, rb_i * 3 + 1)) = COL_TEXT
677     NEXT rb_j
678 END IF
679 NEXT rb_i
680 END
681
682 ' =====
683 ' render_game_board: diff each seated player's 100-byte gamefield against
684 ' its SC_PREVFIELD shadow and repaint only the cells that changed (board
685 ' cells are always painted water by board_init, and SC_PREVFIELD is seeded
686 ' to a sentinel below whenever a fresh board_init just ran, so the first
687 ' call after entering GAMESTART paints every real cell exactly once). Own
688 ' ship overlay is drawn once from the wire's myShips bytes, authoritative
689 ' over whatever handle_placement guessed client-side (covers reconnects).
690 ' =====
691 render_game_board: PROCEDURE
692     IF ships_drawn = 0 THEN
693         FOR rb_i = 0 TO 99
694             POKE (SC_PREVFIELD + 0 * 100 + rb_i), 255
695             POKE (SC_PREVFIELD + 1 * 100 + rb_i), 255
696             POKE (SC_PREVFIELD + 2 * 100 + rb_i), 255
697             POKE (SC_PREVFIELD + 3 * 100 + rb_i), 255
698         NEXT rb_i
699     END IF
700
701     FOR rb_i = 0 TO cur_pc - 1
702         rb_pstatus = PEEK(player_addr(rb_i) + PL_STATUS) AND 255
703         IF rb_pstatus <> PSTATUS_PLACE_SHIPS THEN
704             #rb_field = player_addr(rb_i) + PL_GAMEFIELD
705             FOR rb_c = 0 TO 99
706                 rb_state = PEEK(#rb_field + rb_c) AND 255
707                 IF rb_state <> (PEEK(SC_PREVFIELD + rb_i * 100 + rb_c) AND 255)
THEN
708                     bc_bq = rb_i
709                     bc_x = rb_c % 10
710                     bc_y = rb_c / 10
711                     IF rb_state = 1 THEN
712                         bc_color = BCOL_HIT
713                     ELSEIF rb_state = 2 THEN
714                         bc_color = BCOL_MISS

```

```

715         ELSE
716             bc_color = BCOL_WATER
717         END IF
718         GOSUB board_cell
719         POKE (SC_PREVFIELD + rb_i * 100 + rb_c), rb_state
720     END IF
721     NEXT rb_c
722 END IF
723 NEXT rb_i
724
725 IF ships_drawn = 0 THEN
726     FOR rb_i = 0 TO 4
727         rb_ship = PEEK(FN_RX + GAME_MYSHIPS + rb_i) AND 255
728         rb_sx = (rb_ship % 100) % 10
729         rb_sy = (rb_ship % 100) / 10
730         rb_sdir = rb_ship / 100
731         rb_ssize = ship_size(rb_i)
732         FOR rb_j = 0 TO rb_ssize - 1
733             IF rb_sdir = 0 THEN
734                 bc_x = rb_sx + rb_j : bc_y = rb_sy
735             ELSE
736                 bc_x = rb_sx : bc_y = rb_sy + rb_j
737             END IF
738             bc_bq = 0 : bc_color = BCOL_SHIP
739             GOSUB board_cell
740         NEXT rb_j
741     NEXT rb_i
742     ships_drawn = 1
743 END IF
744
745 ' =====
746 ' targeting: your turn. Cursor moves in lockstep over every live enemy
747 ' quadrant at once -- the server's core multiplayer rule is that a single
748 ' shot fires at that coordinate on every other playing board simultaneously,
749 ' so there's only ever one cursor position to choose, not one per opponent.
750 ' Sets has_targeted + atk_pos on a confirmed attack.
751 ' =====
752 targeting: PROCEDURE 5
753
754     tg_x = 0 : tg_y = 0
755     has_targeted = 0
756     inp_lock = 0
757     GOSUB targeting_draw_cursor
758
759     ' moveTime (server-computed, already net of a network round-trip grace
760     ' period -- see gameLogic.go) is the countdown's starting point; count
761     ' it down locally frame-by-frame rather than re-polling every second.
762     ' Shown in the prompt row's last 2 columns, same corner Scardstud uses
763     ' for its stopwatch.
764     tg_timeleft = PEEK(FN_RX + GAME_MOVETIME) AND 255
765     tg_framecount = 0
766     ' moveTime can run up to 250 (state.bas), so the field needs 3 digits --

```

```

767 ' a 2-digit field silently produces garbage glyphs rather than truncating
768 ' (PRNUM16.z's digit loop assumes higher place-values were already
769 ' stripped by wider field positions it never visits). Space-padded
770 ' (<3>, PRNUM16.b) rather than zero-padded (<3>, PRNUM16.z): .z's entry
771 ' point never initializes the register it uses to pick blank-vs-zero
772 ' padding, so on a value needing fewer than 3 digits it can print
773 ' whatever garbage that register held as the leading "digit" instead of
774 ' an actual 0. .b's entry point explicitly clears it first.
775 PRINT AT STATUS_ROW + 17 COLOR COL_TEXT, <3>tg_timeleft
776
777 tg_loop:
778     WAIT
779
780     tg_framecount = tg_framecount + 1
781     IF tg_framecount >= 60 THEN
782         tg_framecount = 0
783         IF tg_timeleft > 0 THEN tg_timeleft = tg_timeleft - 1
784         PRINT AT STATUS_ROW + 17 COLOR COL_TEXT, <3>tg_timeleft
785         GOSUB sound_tick
786         ' The server enforces its own 45s move limit and skips a turn that
787         ' runs out (gameLogic.go's PLAYER_TIME_LIMIT) -- without this bail,
788         ' a player who never acts leaves this loop blocked on WAIT forever,
789         ' so the client would never poll again to discover its turn had
790         ' already moved on.
791         IF tg_timeleft = 0 THEN
792             GOSUB targeting_erase_cursor
793             RETURN
794         END IF
795     END IF
796
797     IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO tg_loop
798
799     ' Compute the candidate new position into tg_newx/tg_newy without
800     ' touching tg_x/tg_y yet -- targeting_erase_cursor reads tg_x/tg_y to
801     ' know which cell to restore, so it must still see the *old* position
802     ' when it runs, not the one we're about to move to.
803     tg_moved = 0
804     tg_newx = tg_x : tg_newy = tg_y
805     IF CONT1.RIGHT AND tg_newx < 9 THEN tg_newx = tg_newx + 1 : tg_moved = 1
806     IF CONT1.LEFT AND tg_newx > 0 THEN tg_newx = tg_newx - 1 : tg_moved = 1
807     IF CONT1.DOWN AND tg_newy < 9 THEN tg_newy = tg_newy + 1 : tg_moved = 1
808     IF CONT1.UP AND tg_newy > 0 THEN tg_newy = tg_newy - 1 : tg_moved = 1
809
810     IF tg_moved THEN
811         GOSUB targeting_erase_cursor
812         tg_x = tg_newx : tg_y = tg_newy
813         GOSUB targeting_draw_cursor
814         inp_lock = 6
815         GOSUB sound_cursor
816         GOTO tg_loop
817     END IF
818
819     IF CONT1.KEY = 10 THEN

```

```

820     GOSUB targeting_erase_cursor
821     RETURN
822 END IF
823 IF CONT1.BUTTON = 0 THEN GOTO tg_loop
824
825 GOSUB sound_attack
826 GOSUB targeting_erase_cursor
827 atk_pos = tg_y * 10 + tg_x
828 has_targeted = 1
829
830 END
831 targeting_draw_cursor: PROCEDURE
832   FOR tc_q = 1 TO 3
833     IF tc_q < cur_pc THEN
834       tc_state = PEEK(player_addr(tc_q) + PL_STATUS) AND 255
835       IF tc_state = PSTATUS_PLAYING THEN
836         bc_bq = tc_q : bc_x = tg_x : bc_y = tg_y : bc_color =
BCOL_CURSOR
837         GOSUB board_cell
838       END IF
839     END IF
840   NEXT tc_q
841 END
842
843 targeting_erase_cursor: PROCEDURE
844   FOR tc_q = 1 TO 3
845     IF tc_q < cur_pc THEN
846       tc_state = PEEK(player_addr(tc_q) + PL_STATUS) AND 255
847       IF tc_state = PSTATUS_PLAYING THEN
848         tc_field = PEEK(SC_PREVFIELD + tc_q * 100 + tg_y * 10 + tg_x)
AND 255
849         tc_color = BCOL_WATER
850         IF tc_field = 1 THEN tc_color = BCOL_HIT
851         IF tc_field = 2 THEN tc_color = BCOL_MISS
852         bc_bq = tc_q : bc_x = tg_x : bc_y = tg_y : bc_color = tc_color
853         GOSUB board_cell
854       END IF
855     END IF
856   NEXT tc_q
857 END
858
859 ' The compiled program overflows IntyBASIC's default $5000-$6FFF (8192-word)
860 ' window -- the compiler happily keeps allocating past $6FFF, but $7000 is
861 ' not in the manual's list of ranges "usable without additional programming"
862 ' on modern flash cart/homebrew PCBs (that list jumps from $6FFF to $A000).
863 ' Relocating everything from here to halt: into $C100-$FFFF (also on that
864 ' list) via an explicit ASM ORG keeps the whole ROM inside documented-safe
865 ' territory. This is a code segment, not just data (contrary to the
866 ' manual's "easier... if you put only data in these areas" suggestion),
867 ' but cross-segment GOSUB/GOTO compiles to ordinary absolute CP-1610
868 ' jumps/calls, so that's only a bookkeeping caveat, not a correctness one.
869   ASM ORG $D000
870

```

```

871 ' =====
872 ' handle_placement: place all 5 ships (sizes 5,4,3,3,2, fixed order). Each
873 ' ship starts at a random legal position, then the disc moves it, B1
874 ' rotates it, and the action button confirms (rejected locally if it
875 ' overlaps an already-confirmed ship -- no need to round-trip to the server
876 ' to find that out). Fills SC_MYSHIPS for compose_url's url_path=3.
877 ' =====
878 handle_placement: PROCEDURE ④
879
880 ' q0 is already water -- board_init just painted all four boards before
881 ' this was reached (see tl_loop's STATUS_LOBBY-or-255 transition). Only
882 ' the occupancy map needs clearing here.
883 FOR rb_i = 0 TO 99
884   POKE (SC_OCCUPY + rb_i), 0
885 NEXT rb_i
886
887 FOR ship_idx = 0 TO 4
888   pc_size = ship_size(ship_idx)
889
890   DO
891     pc_dir = RANDOM(2)
892     IF pc_dir = 0 THEN
893       pc_x = RANDOM(11 - pc_size)
894       pc_y = RANDOM(10)
895     ELSE
896       pc_x = RANDOM(10)
897       pc_y = RANDOM(11 - pc_size)
898     END IF
899     GOSUB place_fits
900     LOOP WHILE pc_ok = 0
901
902     pc_prevx = pc_x : pc_prevy = pc_y : pc_prevdir = pc_dir
903     GOSUB place_draw
904
905     ph_shipnum = ship_idx + 1
906     ' "/5 MOVE B1=ROT" is exactly 14 chars, filling columns 6-19 of the
907     ' 20-column STATUS_ROW with no overflow -- the original "/5 DISC=
908     ' MOVE B1=ROT" (20 chars) ran 6 columns past the end of the row and
909     ' off the end of #BACKTAB, corrupting the rest of the screen.
910     PRINT AT STATUS_ROW COLOR COL_TEXT, "SHIP "
911     PRINT AT STATUS_ROW + 5, <ph_shipnum
912     PRINT AT STATUS_ROW + 6, "/5 MOVE B1=ROT"
913
914     inp_lock = 0
915   ph_loop:
916     WAIT
917     IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO ph_loop
918
919     pc_mx = 9 : pc_my = 9
920     IF pc_dir = 0 THEN pc_mx = 10 - pc_size ELSE pc_my = 10 - pc_size
921
922     pc_moved = 0
923     IF CONT1.RIGHT AND pc_x < pc_mx THEN pc_x = pc_x + 1 : pc_moved = 1
924

```

```

923 IF CONT1.LEFT AND pc_x > 0 THEN pc_x = pc_x - 1 : pc_moved = 1
924 IF CONT1.DOWN AND pc_y < pc_my THEN pc_y = pc_y + 1 : pc_moved = 1
925 IF CONT1.UP AND pc_y > 0 THEN pc_y = pc_y - 1 : pc_moved = 1
926
927 IF CONT1.B1 THEN
928   pc_dir = 1 - pc_dir
929   IF pc_dir = 0 AND pc_x > 10 - pc_size THEN pc_x = 10 - pc_size
930   IF pc_dir = 1 AND pc_y > 10 - pc_size THEN pc_y = 10 - pc_size
931   pc_moved = 1
932 END IF
933
934 IF pc_moved THEN
935   GOSUB place_erase
936   pc_prevx = pc_x : pc_prevy = pc_y : pc_prevdir = pc_dir
937   GOSUB place_draw
938   inp_lock = 6
939   GOSUB sound_cursor
940   GOTO ph_loop
941 END IF
942
943 IF CONT1.BUTTON = 0 THEN GOTO ph_loop
944
945 GOSUB place_fits
946 IF pc_ok = 0 THEN
947   GOSUB sound_invalid
948   GOTO ph_loop
949 END IF
950
951 GOSUB place_mark
952 GOSUB sound_place_ship
953 FOR pc_i = 0 TO pc_size - 1
954   IF pc_dir = 0 THEN
955     bc_x = pc_x + pc_i : bc_y = pc_y
956   ELSE
957     bc_x = pc_x : bc_y = pc_y + pc_i
958   END IF
959   bc_bq = 0 : bc_color = BCOL_SHIP
960   GOSUB board_cell
961 NEXT pc_i
962
963 POKE (SC_MYSHIPS + ship_idx), pc_x + pc_y * 10 + 100 * pc_dir
964 NEXT ship_idx
965 END
966
967 place_fits: PROCEDURE
968   pc_ok = 1
969   IF pc_dir = 0 THEN
970     IF pc_x + pc_size > 10 THEN pc_ok = 0
971   ELSE
972     IF pc_y + pc_size > 10 THEN pc_ok = 0
973   END IF
974   IF pc_ok THEN
975     FOR pc_i = 0 TO pc_size - 1

```

```

976   IF pc_dir = 0 THEN
977     pc_cx = pc_x + pc_i : pc_cy = pc_y
978   ELSE
979     pc_cx = pc_x : pc_cy = pc_y + pc_i
980   END IF
981   IF (PEEK(SC_OCCUPY + pc_cy * 10 + pc_cx) AND 255) <> 0 THEN pc_ok =
0
982   NEXT pc_i
983 END IF
984 END
985
986 place_mark: PROCEDURE
987   FOR pc_i = 0 TO pc_size - 1
988     IF pc_dir = 0 THEN
989       pc_cx = pc_x + pc_i : pc_cy = pc_y
990     ELSE
991       pc_cx = pc_x : pc_cy = pc_y + pc_i
992     END IF
993     POKE (SC_OCCUPY + pc_cy * 10 + pc_cx), 1
994   NEXT pc_i
995 END
996
997 place_erase: PROCEDURE
998   FOR pc_i = 0 TO pc_size - 1
999     IF pc_prevdir = 0 THEN
1000       pc_cx = pc_prevx + pc_i : pc_cy = pc_prevy
1001     ELSE
1002       pc_cx = pc_prevx : pc_cy = pc_prevy + pc_i
1003     END IF
1004     bc_bq = 0 : bc_x = pc_cx : bc_y = pc_cy : bc_color = BCOL_WATER
1005     GOSUB board_cell
1006   NEXT pc_i
1007 END
1008
1009 place_draw: PROCEDURE
1010   FOR pc_i = 0 TO pc_size - 1
1011     IF pc_dir = 0 THEN
1012       pc_cx = pc_x + pc_i : pc_cy = pc_y
1013     ELSE
1014       pc_cx = pc_x : pc_cy = pc_y + pc_i
1015     END IF
1016     bc_bq = 0 : bc_x = pc_cx : bc_y = pc_cy : bc_color = BCOL_CURSOR
1017     GOSUB board_cell
1018   NEXT pc_i
1019 END
1020
1021 ' =====
1022 ' ingame_menu: keypad CLEAR overlay, drawn over the prompt row and the two
1023 ' rows above it. Disc up/down to choose, action button to confirm, Clear
1024 ' again cancels straight back to RESUME.
1025 ' =====
1026 ingame_menu: PROCEDURE
1027   im_sel = 0

```

```

1028   inp_lock = 0
1029 im_loop:
1030   PRINT AT STATUS_ROW - 40 COLOR COL_TEXT, "          "
1031   PRINT AT STATUS_ROW - 20 COLOR COL_TEXT, "          "
1032   PRINT AT STATUS_ROW COLOR COL_TEXT, "          "
1033   PRINT AT STATUS_ROW - 40 COLOR COL_TEXT, "TABLE MENU"
1034   #gs_c = COL_TEXT
1035   IF im_sel = 0 THEN #gs_c = COL_HILITE
1036   PRINT AT STATUS_ROW - 20 COLOR #gs_c, "RESUME"
1037   #gs_c = COL_TEXT
1038   IF im_sel = 1 THEN #gs_c = COL_HILITE
1039   PRINT AT STATUS_ROW COLOR #gs_c, "QUIT TABLE"
1040
1041   WAIT
1042   IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO im_loop
1043
1044   IF CONT1.UP OR CONT1.DOWN THEN
1045     im_sel = 1 - im_sel
1046     inp_lock = 10
1047     GOSUB sound_cursor
1048     GOTO im_loop
1049   END IF
1050   IF CONT1.KEY = 10 THEN GOTO im_done
1051   IF CONT1.BUTTON = 0 THEN GOTO im_loop
1052   GOSUB sound_select
1053
1054   IF im_sel = 1 THEN
1055     url_path = 5 : GOSUB compose_url
1056     #net_readlen = 8
1057     GOSUB api_call
1058     GOSUB clear_room_appkey
1059     want_leave = 1
1060   END IF
1061 im_done:
1062   prev_status = 255 ' force a full CLS/board_init redraw on return
1063 END
1064
1065 ' =====
1066 ' name_entry_screen: disc letter picker, max 8 chars. Disc up/down cycles
1067 ' the character under the cursor through A-Z, 0-9, space; left/right moves
1068 ' the cursor; the action button accepts (at least 1 non-space char).
1069 ' Adapted from fujinet-Scardstud/intv/Scard.bas, verbatim apart from colors.
1070 ' =====
1071 name_entry_screen: PROCEDURE
1072   CLS
1073   PRINT AT 0 COLOR COL_TEXT, "ENTER YOUR NAME"
1074   FOR ne_i = 0 TO 7
1075     ne_buf(ne_i) = 36 ' space
1076   NEXT ne_i
1077   ne_cur = 0
1078   inp_lock = 0
1079
1080 ne_loop:

```

```

1081   FOR ne_i = 0 TO 7
1082     #gs_c = COL_TEXT
1083     IF ne_i = ne_cur THEN #gs_c = COL_HILITE
1084     ne_j = ne_buf(ne_i)
1085     IF ne_j < 26 THEN
1086       gs_j = 65 + ne_j
1087     ELSEIF ne_j < 36 THEN
1088       gs_j = 48 + ne_j - 26
1089     ELSE
1090       gs_j = 95 ' underscore stands in for a visible blank
1091     END IF
1092     #BACKTAB(60 + 6 + ne_i) = (gs_j - 32) * 8 + #gs_c
1093   NEXT ne_i
1094
1095   WAIT
1096   IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO ne_loop
1097
1098   IF CONT1.RIGHT THEN
1099     ne_cur = ne_cur + 1
1100     IF ne_cur > 7 THEN ne_cur = 0
1101     inp_lock = 8
1102     GOSUB sound_cursor
1103     GOTO ne_loop
1104   END IF
1105   IF CONT1.LEFT THEN
1106     IF ne_cur = 0 THEN ne_cur = 8
1107     ne_cur = ne_cur - 1
1108     inp_lock = 8
1109     GOSUB sound_cursor
1110     GOTO ne_loop
1111   END IF
1112   IF CONT1.UP THEN
1113     ne_buf(ne_cur) = ne_buf(ne_cur) + 1
1114     IF ne_buf(ne_cur) > 36 THEN ne_buf(ne_cur) = 0
1115     inp_lock = 6
1116     GOSUB sound_cursor
1117     GOTO ne_loop
1118   END IF
1119   IF CONT1.DOWN THEN
1120     IF ne_buf(ne_cur) = 0 THEN ne_buf(ne_cur) = 37
1121     ne_buf(ne_cur) = ne_buf(ne_cur) - 1
1122     inp_lock = 6
1123     GOSUB sound_cursor
1124     GOTO ne_loop
1125   END IF
1126   IF CONT1.BUTTON = 0 THEN GOTO ne_loop
1127   GOSUB sound_select
1128
1129   ne_len = 8
1130   WHILE ne_len > 0 AND ne_buf(ne_len - 1) = 36
1131     ne_len = ne_len - 1
1132   WEND
1133   IF ne_len = 0 THEN GOTO ne_loop

```

```

1134 FOR ne_i = 0 TO ne_len - 1
1135     ne_j = ne_buf(ne_i)
1136     IF ne_j < 26 THEN
1137         gs_j = 65 + ne_j
1138     ELSE
1139         gs_j = 48 + ne_j - 26
1140     END IF
1141     POKE (SC_NAME + ne_i), gs_j
1142 NEXT ne_i
1143 POKE (SC_NAME + ne_len), 0
1144
1145 ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1 : ak_key = 0 : ak_mode =
1
1147 GOSUB appkey_open
1148 IF fn_ok THEN
1149     #fn_src = SC_NAME : fn_len = ne_len
1150     GOSUB appkey_write
1151     IF fn_ok = 0 THEN
1152         PRINT AT 100 COLOR COL_TEXT, "NAME NOT SAVED FOR "
1153         PRINT AT 120 COLOR COL_TEXT, "NEXT TIME "
1154         inp_lock = 90
1155         WHILE inp_lock > 0
1156             inp_lock = inp_lock - 1
1157             WAIT
1158         WEND
1159     END IF
1160     GOSUB appkey_close
1161 END IF
1162 END
1163
1164 halt:
1165     WAIT
1166     GOTO halt
1167

```

LISTING 7 state.bas — Battleship wire format

```

1 ' state.bas -- binary GameState/Tables wire format, read in place out of FN_RX.
2 '
3 ' Per the pattern established by fujinet-5cardstud/intv/state.bas: never copy
4 ' the reply into IntyBASIC variables -- these are just fixed offsets (matching
5 ' server/util.go's serializeResults binary layout, requested with
6 ' &bin=1&v=2) plus small DEF FN accessors that PEEK straight out of FN_RX.
7 ' v=2 is not optional here: v=1 omits the winner's ships and forces
8 ' activePlayer=-1 at game over, both of which the gameover screen needs.
9 '
10 ' Header (always present), 38 bytes:
11 '   0  playerCount (1)
12 '   1  prompt[33]      lowercased, NUL-padded
13 '   34 status (1)      0 lobby, 1 place-ships, 10 gamestart,

```

```

14 '
15 '   35  playerStatus (1)  your status: 0 playing, 1 defeated, 2 viewing,
16 '                        3 ready, 10 place-ships
17 '   36  activePlayer (signed, $FF = -1; 0 = your turn)
18 '   37  moveTime (1, seconds, 0-250)
19 '
20 ' If status = 0 (STATUS_LOBBY), the header is followed by:
21 '   38  serverName[21]
22 '   59  players[N] -- N = playerCount, 10 bytes each: name[9] ready(1)
23 '
24 ' Otherwise (in a table, placing/playing/over):
25 '   38  lastAttackPos (1)  0-99
26 '   39  myShips[10]       [0..4] your ships, [5..9] winner's ships

```

```

27 '          (v=2; only nonzero at game over), each byte is
28 '          cell + 100*dir (dir 0=horizontal, 1=vertical)
29 49  players[N] -- N = playerCount, 115 bytes each:
30 '          name[9] status(1) gamefield[100] shipsLeft[5]
31 '          (gamefield/shipsLeft are present only once ships are placed and
32 '          the game has started -- see GAME_MINLEN/expected-length notes
33 '          below; entries for players who haven't placed are shorter, but
34 '          in practice the server only starts sending player records once
35 '          GAMESTART, by which point every playing player has placed.)
36
37 /tables -> Tables struct:
38 0  count (1)
39 ' then N x 36 bytes: table[9] name[21] players[6] (literal "cur / max")
40
41 CONST GAME_PLAYERCOUNT = 0
42 CONST GAME_PROMPT = 1 ' 33 bytes
43 CONST GAME_STATUS = 34
44 CONST GAME_PLAYERSTATUS = 35
45 CONST GAME_ACTIVEPLAYER = 36
46 CONST GAME_MOVETIME = 37
47
48 CONST LOBBY_SERVERNAME = 38 ' 21 bytes
49 CONST LOBBY_PLAYERS = 59
50 CONST LOBBY_PLAYER_STRIDE = 10
51 CONST LP_NAME = 0 ' 9 bytes
52 CONST LP_READY = 9 ' 1 byte
53
54 CONST GAME_LASTATTACK = 38
55 CONST GAME_MYSHIPS = 39 ' 10 bytes
56 CONST GAME_PLAYERS = 49
57
58 CONST PLAYER_STRIDE = 115
59 CONST PL_NAME = 0 ' 9 bytes
60 CONST PL_STATUS = 9 ' 1 byte
61 CONST PL_GAMEFIELD = 10 ' 100 bytes
62 CONST PL_SHIPSLEFT = 110 ' 5 bytes
63
64 CONST TABLE_STRIDE = 36
65 CONST TBL_ID = 0 ' 9 bytes
66 CONST TBL_NAME = 9 ' 21 bytes
67 CONST TBL_PLAYERS = 30 ' 6 bytes, literal "cur / max"
68
69 CONST GAME_MAXLEN = 509
70 CONST GAME_MINLEN = 38 ' fixed header, before branching on status
71 CONST TABLES_MAXLEN = 361
72
73 ' Status values.
74 CONST STATUS_LOBBY = 0
75 CONST STATUS_PLACE_SHIPS = 1
76 CONST STATUS_GAMESTART = 10
77 CONST STATUS_MISS = 11
78 CONST STATUS_HIT = 12
79 CONST STATUS_SUNK = 13

```

```

80 CONST STATUS_GAMEOVER = 99
81
82 ' Player status values.
83 CONST PSTATUS_PLAYING = 0
84 CONST PSTATUS_DEFEATED = 1
85 CONST PSTATUS_VIEWING = 2
86 CONST PSTATUS_READY = 3
87 CONST PSTATUS_PLACE_SHIPS = 10
88
89 ' Ship sizes, fixed order (index also selects the shipsLeft/myShips slot).
90 ship_size: DATA 5, 4, 3, 3, 2
91
92 ' player_addr(i): address of player i's 115-byte in-game record in FN_RX.
93 DEF FN player_addr(i) = FN_RX + GAME_PLAYERS + i * PLAYER_STRIDE
94
95 ' lobby_player_addr(i): address of player i's 10-byte lobby record in FN_RX.
96 DEF FN lobby_player_addr(i) = FN_RX + LOBBY_PLAYERS + i *
97 LOBBY_PLAYER_STRIDE
98
99 ' table_addr(i): address of table i's 36-byte record in FN_RX (i is 0-based,
100 ' the record right after the leading count byte).
101 DEF FN table_addr(i) = FN_RX + 1 + i * TABLE_STRIDE
102
103 ' active_player: activePlayer as a signed value (-1..3), converting the
104 ' wire's $FF sentinel. Called as plain 'active_player' (no parens -- DEF FN
105 ' with no arguments is defined and invoked without them).
106 DEF FN active_player = ((PEEK(FN_RX + GAME_ACTIVEPLAYER) AND 255) *
107 -256 + (PEEK(FN_RX + GAME_ACTIVEPLAYER) AND 255))
108
109 ' state_status / state_playercount / state_playerstatus: unsigned byte reads,
110 ' broken out as DEF FNs purely so call sites read as intent, not offsets.
111 DEF FN state_status = PEEK(FN_RX + GAME_STATUS) AND 255
112 DEF FN state_playercount = PEEK(FN_RX + GAME_PLAYERCOUNT) AND 255
113 DEF FN state_playerstatus = PEEK(FN_RX + GAME_PLAYERSTATUS) AND 255
114
115 -----
116 ' validate_state: layered sanity check on a /state, /ready, /place, /attack,
117 ' or /view reply already sitting in FN_RX (#net_gotten bytes). FN_RX is never
118 ' cleared between calls, so a short read leaves stale bytes from a previous,
119 ' larger response -- fixed-offset reads would happily render that as
120 ' garbage. Sets fn_ok = 0 (in addition to whatever net/transact left it) if
121 ' any gate fails; callers must check fn_ok after calling this, not just after
122 ' api_call.
123 -----
124 DIM vs_status, vs_pc, #vs_expect
125 validate_state: PROCEDURE 3
126 IF fn_ok = 0 OR #net_gotten < GAME_MINLEN THEN
127   fn_ok = 0
128   RETURN
129 END IF
130
131 vs_status = state_status

```

```

130 vs_pc = state_playercount
131
132 IF vs_pc > 4 THEN
133     fn_ok = 0
134     RETURN
135 END IF
136
137 IF vs_status <> STATUS_LOBBY AND vs_status <> STATUS_PLACE_SHIPS AND
vs_status <> STATUS_GAMESTART AND vs_status <> STATUS_MISS AND vs_status <>
STATUS_HIT AND vs_status <> STATUS_SUNK AND vs_status <> STATUS_GAMEOVER THEN
138     fn_ok = 0
139     RETURN
140 END IF
141
142 IF vs_status = STATUS_LOBBY THEN
143     #vs_expect = LOBBY_PLAYERS + vs_pc * LOBBY_PLAYER_STRIDE
144 ELSEIF vs_status = STATUS_PLACE_SHIPS THEN
145     ' No players[] array yet -- see the GAME_MINLEN/expected-length note
146     ' above header comment: the server only starts sending player
147     ' records once GAMESTART. Header + lastAttackPos + myShips only.
148     #vs_expect = GAME_PLAYERS
149 ELSE
150     #vs_expect = GAME_PLAYERS + vs_pc * PLAYER_STRIDE
151 END IF
152
153 IF #net_gotlen < #vs_expect THEN fn_ok = 0
154 END
155
```

LISTING 8 board.bas — the colored-squares kernel

```

1 ' board.bas -- colored-squares (40x24 4x4-block) render kernel for the four
2 ' 10x10 Battleship boards. Requires MODE 0 (Color Stack) -- colored-squares
3 ' cards do not exist in MODE 1 (constants.bas / IntyBASIC manual).
4 '
5 ' A BACKTAB word with bit 12 = 1 and bit 11 = 0 is not a GROM/GRAM card at
6 ' all: it is four independently-coloured 4x4 pixel quadrants (constants.bas
7 ' calls them "pixels" 0-3, laid out TL/TR/BL/BR). Colours 0-6 are the STIC
8 ' primaries; 7 = transparent (shows the current colour-stack colour). This
9 ' file never uses 7 -- every cell always gets an explicit colour, so unlike
10 ' Carlsson's original battleships-display-example.bas there is no need to
11 ' cycle the colour stack for decoration; MODE 0's four stack slots are left
12 ' at their compiled-in defaults and ignored.
13 '
14 ' Board layout (screen is 20 cards x 12 rows; each board is 5x5 cards):
15 '
16 '   col  0-4    5    6-10    11-19
17 ' row0 q1 | div | q2 |   right-hand chrome panel
18 '   -4    |   |   |   (player names, ships-left, clock --
19 ' row5 ---- divider row ---- owned by the caller, not this file)
20 ' row6 q0 | div | q3 |

```

```

21 '   -10    |   |
22 ' row11 ..... 20-char prompt / status line .....
23 '
24 ' Quadrant numbering matches the C clients' graphics.h convention: 0-3
25 ' starting bottom-left, going clockwise; the local player is always
26 ' quadrant 0.
27 '
28 '   board_col: DATA 0, 0, 6, 6    ' q0..q3 origin column (card coords)
29 '   board_row: DATA 6, 0, 0, 6    ' q0..q3 origin row
30 '
31 ' cs_mask(sub): AND-mask that clears sub-square `sub`'s bits (0=TL,1=TR,
32 ' 2=BL,3=BR) plus bit 11 (must always be 0 on a colored-squares card) and
33 ' bit 12 (the enable bit) -- cs_plot ADDs CS_COLOUR_SQUARES_ENABLE back in
34 ' unconditionally afterward, so the mask must clear it here rather than
35 ' preserve it. An earlier version of this table preserved bit 12: on a card
36 ' that was already a colored square (true for every cell after its first
37 ' write), that left the ADD doubling an already-set bit 12, which carries
38 ' into bit 13 and clears bit 12 -- turning the card back into a plain GROM/
39 ' GRAM text card and rendering as a stray letter instead of a color.
40 ' $D1FF (BR) additionally clears bit 13 (the BR quadrant's relocated high

```



```

41 ' colour bit -- see cs_val below).
42   cs_mask: DATA $E7F8, $E7C7, $E63F, $C1FF 1
43
44 ' cs_val(sub*8 + color): pre-shifted bit pattern for sub-square 'sub' (0-3)
45 ' showing STIC colour 'color' (0-6 primary, 7 = transparent/background).
46 ' Pulled directly from constants.bas's CS_PIX0 *..CS_PIX3 * -- CS_PIX3 *
47 ' already has the bit11->bit13 relocation baked in for colours 4-6 (a
48 ' straight *512 shift for colour>=4 would land in bit 11, illegal on a
49 ' colored-squares card; the STIC expects the BR quadrant's high colour bit
50 ' at bit 13 instead).
51   cs_val: DATA CS_PIX0_BLACK, CS_PIX0_BLUE, CS_PIX0_RED, CS_PIX0_TAN,
52   CS_PIX0_DARKGREEN, CS_PIX0_GREEN, CS_PIX0_YELLOW, CS_PIX0_BACKGROUND
53   DATA CS_PIX1_BLACK, CS_PIX1_BLUE, CS_PIX1_RED, CS_PIX1_TAN,
54   CS_PIX1_DARKGREEN, CS_PIX1_GREEN, CS_PIX1_YELLOW, CS_PIX1_BACKGROUND
55   DATA CS_PIX2_BLACK, CS_PIX2_BLUE, CS_PIX2_RED, CS_PIX2_TAN,
56   CS_PIX2_DARKGREEN, CS_PIX2_GREEN, CS_PIX2_YELLOW, CS_PIX2_BACKGROUND
57   DATA CS_PIX3_BLACK, CS_PIX3_BLUE, CS_PIX3_RED, CS_PIX3_TAN,
58   CS_PIX3_DARKGREEN, CS_PIX3_GREEN, CS_PIX3_YELLOW, CS_PIX3_BACKGROUND
59
60 ' Cell-state colours. Chosen so "empty/unknown" reads calmly (blue water)
61 ' and hit/miss/ship read at a glance; see board_draw below for the mapping
62 ' from the server's gamefield byte values (state.bas: 0 unknown, 1 hit,
63 ' 2 miss) to these.
64   CONST BCOL_MISS    = 0 ' black
65   CONST BCOL_WATER   = 1 ' blue -- unknown/untouched cell
66   CONST BCOL_HIT     = 2 ' red
67   CONST BCOL_SUNKSHIP = 3 ' tan -- winner's revealed ships at game over
68   CONST BCOL_SHIP    = 5 ' green -- own ship, own board only
69   CONST BCOL_CURSOR  = 6 ' yellow
70   CONST BCOL_DIVIDER = 0 ' black
71
72 ' -----
73 ' cs_fill: paint an entire card (all four sub-squares) with one colour.
74 ' Inputs: cs_col, cs_row (card coords), cs_color.
75 ' -----
76 DIM cs_col, cs_row, cs_q, cs_color
77 DIM #cs_word, cs_addr
78 cs_fill: PROCEDURE
79   #cs_word = CS_COLOUR_SQUARES_ENABLE + cs_val(cs_color) + cs_val(8 +
80   #cs_color) + cs_val(16 + cs_color) + cs_val(24 + cs_color)
81   cs_addr = screenpos(cs_col, cs_row)
82   #BACKTAB(cs_addr) = #BACKTAB(cs_addr) AND cs_mask(cs_q) +
83   CS_COLOUR_SQUARES_ENABLE + cs_val(cs_q * 8 + cs_color)
84 END
85
86 ' -----
87 ' cs_plot: read-modify-write a single sub-square, preserving the other
88 ' three. Inputs: cs_col, cs_row (card coords), cs_q (0-3), cs_color.
89 ' -----
90 cs_plot: PROCEDURE
91   cs_addr = screenpos(cs_col, cs_row)
92   #BACKTAB(cs_addr) = (#BACKTAB(cs_addr) AND cs_mask(cs_q)) +
93   CS_COLOUR_SQUARES_ENABLE + cs_val(cs_q * 8 + cs_color)
94 END
95
96 ' -----
97 ' board_init: paint all four boards to water and the dividers to black.
98 ' Call Once after MODE 0 / CLS, before the first board_draw.
99 ' -----
100 DIM bi_bq, bi_cc, bi_cr
101 board_init: PROCEDURE
102   FOR bi_bq = 0 TO 3
103     FOR bi_cr = 0 TO 4
104       FOR bi_cc = 0 TO 4
105         cs_col = board_col(bi_bq) + bi_cc
106         cs_row = board_row(bi_bq) + bi_cr
107         cs_color = BCOL_WATER
108         GOSUB cs_fill
109       NEXT bi_cc
110     NEXT bi_cr
111   NEXT bi_bq
112
113 ' Divider: column 5 for all 11 board rows, row 5 for all 11 board
114 ' columns except column 5 itself (already painted by the first loop).
115 FOR bi_cr = 0 TO 10
116   cs_col = 5 : cs_row = bi_cr : cs_color = BCOL_DIVIDER : GOSUB cs_fill
117 NEXT bi_cr
118 FOR bi_cc = 0 TO 10
119   IF bi_cc <> 5 THEN
120     cs_col = bi_cc : cs_row = 5 : cs_color = BCOL_DIVIDER : GOSUB
121 cs_fill
122   END IF
123 NEXT bi_cc
124 WAIT
125 END
126
127 ' -----
128 ' board_draw: full 25-card repaint of quadrant bd_bq's 10x10 grid from a
129 ' 100-byte gamefield buffer (state.bas values: 0 unknown, 1 hit, 2 miss) at
130 ' RAM address #bd_field. One WAIT per board -- 25 single-write cards per
131 ' frame stays well inside vblank; the lesson from Scardstud's differential
132 ' renderer (5card.bas) is that an unbroken multi-region burst does not.
133 ' Ship overlay (own board only) is layered on afterward by the caller via
134 ' board_cell, since the gamefield alone doesn't carry ship positions.
135 ' -----
136 DIM bd_bq, #bd_field
137 DIM bd_cc, bd_cr, bd_sub, bd_cell, bd_state, bd_color
138 DIM #bd_word, bd_addr
139 board_draw: PROCEDURE
140   FOR bd_cr = 0 TO 4
141     FOR bd_cc = 0 TO 4
142       #bd_word = CS_COLOUR_SQUARES_ENABLE
143       FOR bd_sub = 0 TO 3
144         sub 0=TL(x even,y even) 1=TR(x odd,y even)
145         sub 2=BL(x even,y odd) 3=BR(x odd,y odd)
146       NEXT bd_sub
147     NEXT bd_cc
148   NEXT bd_cr
149 END

```

```

139         bd_cell = (bd_cr * 2 + bd_sub / 2) * 10 + (bd_cc * 2 + (bd_sub
AND 1))
140         bd_state = PEEK(#bd_field + bd_cell) AND 255
141         IF bd_state = 1 THEN
142             bd_color = BCOL_HIT
143         ELSEIF bd_state = 2 THEN
144             bd_color = BCOL_MISS
145         ELSE
146             bd_color = BCOL_WATER
147         END IF
148         #bd_word = #bd_word + cs_val(bd_sub * 8 + bd_color)
149     NEXT bd_sub
150     bd_addr = screenpos(board_col(bd_bq) + bd_cc, board_row(bd_bq) +
bd_cr)
151     #BACKTAB(bd_addr) = #bd_word
152     NEXT bd_cc
153     WAIT
154     NEXT bd_cr
155 END
156
157 '-----
158 ' board_cell: update a single cell (x,y both 0-9) on quadrant bc_bq to
159 ' bc_color. Used for incremental attack-result updates, own-ship overlay
160 ' during/after placement, and the targeting cursor (caller is responsible
161 ' for restoring the underlying colour when the cursor moves off a cell --
162 ' this file is stateless and does not remember what was drawn).
163 '-----
164 DIM bc_bq, bc_x, bc_y, bc_color
165 board_cell: PROCEDURE
166     cs_col = board_col(bc_bq) + bc_x / 2
167     cs_row = board_row(bc_bq) + bc_y / 2
168     cs_q = (bc_y AND 1) * 2 + (bc_x AND 1)
169     cs_color = bc_color
170     GOSUB cs_plot
171 END
172

```

LISTING 9 sound.bas — Battleship effects

```

1 ' sound.bas -- sound effects, adapted from fujinet-5cardstud/intv/sound.bas.
2 ' The transport primitive (play tone) and several UI-generic effects are
3 ' kept verbatim; the game-specific effects are replaced with Battleship's
4 ' own set, matching the C clients' platform-specific/sound.h event names
5 ' (soundCursor/Select/MyTurn/JoinGame/... plus Attack/Hit/Sink/Miss/Invalid
6 ' which 5 Card Stud has no equivalent of).
7
8 ' All effects play on PSG channel 0 -- turn-based game, non-overlapping UI
9 ' events, no need for polyphony (and SOUND's channel argument must be a
10 ' compile-time constant anyway, which rules out a generic multi-channel
11 ' helper).
12

```

```

13 ' SOUND's frequency argument is a 12-bit PSG divisor, computed as
14 ' round(3579545/32/hz) for NTSC. These are precomputed rather than divided
15 ' at runtime: 3579545/32 alone is ~111861, which overflows IntyBASIC's
16 ' 16-bit variables (max 65535), so it only works as a compile-time-folded
17 ' literal division -- not as CONST-divided-by-a-runtime-variable. Every
18 ' effect here uses a fixed, known-in-advance frequency, so precomputing
19 ' sidesteps the overflow entirely.
20 ' 50Hz->2237 60Hz->1864 70Hz->1598 80Hz->1398 100Hz->1119
21 ' 150Hz->746 200Hz->559 300Hz->373 311Hz->360 330Hz->339
22 ' 340Hz->329 350Hz->320 392Hz->285 415Hz->270 430Hz->260
23 ' 500Hz->224 800Hz->140
24 ' CONST SND_VOL = 12

```

```

25     DIM #snd_val, snd_gate, snd_post, snd_i
26
27
28 '-----
29 ' play_tone: one square-wave note on channel 0. #snd_val = precomputed PSG
30 ' divisor (see table above), snd_gate = frames the tone sounds, snd_post =
31 ' frames of silence after.
32 '-----
33 play_tone: PROCEDURE
34     SOUND 0, #snd_val, SND_VOL
35     FOR snd_i = 1 TO snd_gate
36         WAIT
37     NEXT snd_i
38     SOUND 0, 0, 0
39     FOR snd_i = 1 TO snd_post
40         WAIT
41     NEXT snd_i
42 END
43
44 ' sound_join: sit down at a table (3-tone rising figure: 430,340,500 Hz).
45 sound_join: PROCEDURE
46     #snd_val = 260 : snd_gate = 5 : snd_post = 8 : GOSUB play_tone
47     #snd_val = 329 : snd_gate = 5 : snd_post = 0 : GOSUB play_tone
48     #snd_val = 224 : snd_gate = 5 : snd_post = 0 : GOSUB play_tone
49 END
50
51 ' sound_myturn: it's your turn to fire (double beep, 430,430 Hz).
52 sound_myturn: PROCEDURE
53     #snd_val = 260 : snd_gate = 4 : snd_post = 2 : GOSUB play_tone
54     #snd_val = 260 : snd_gate = 4 : snd_post = 2 : GOSUB play_tone
55 END
56
57 ' sound_gamedone: game over, win fanfare (4-tone rising: 311,330,392,415 Hz).
58 sound_gamedone: PROCEDURE
59     #snd_val = 360 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
60     #snd_val = 339 : snd_gate = 20 : snd_post = 0 : GOSUB play_tone
61     #snd_val = 285 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
62     #snd_val = 270 : snd_gate = 20 : snd_post = 0 : GOSUB play_tone
63 END
64
65 ' sound_player_join: another player sits down mid-lobby (5-tone rising
66 ' sweep: 50,60,70,80 Hz).
67 sound_player_join: PROCEDURE
68     #snd_val = 2237 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
69     #snd_val = 1864 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
70     #snd_val = 1598 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
71     #snd_val = 1398 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
72 END
73
74 ' sound_player_left: a player leaves mid-game (falling sweep, mirror of join).
75 sound_player_left: PROCEDURE
76     #snd_val = 1398 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
77     #snd_val = 1598 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone

```

```

78     #snd_val = 1864 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
79     #snd_val = 2237 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
80 END
81
82 ' sound_select: a menu/table/ready choice was confirmed (2-tone rising
83 ' chirp: 300,350 Hz).
84 sound_select: PROCEDURE
85     #snd_val = 373 : snd_gate = 3 : snd_post = 1 : GOSUB play_tone
86     #snd_val = 320 : snd_gate = 3 : snd_post = 0 : GOSUB play_tone
87 END
88
89 ' sound_cursor: cursor/ship moved one position (300 Hz).
90 sound_cursor: PROCEDURE
91     #snd_val = 373 : snd_gate = 2 : snd_post = 0 : GOSUB play_tone
92 END
93
94 ' sound_tick: countdown clock tick during targeting (short 300 Hz click).
95 sound_tick: PROCEDURE
96     #snd_val = 373 : snd_gate = 1 : snd_post = 0 : GOSUB play_tone
97 END
98
99 ' sound_place_ship: a ship was confirmed placed (short rising chirp,
100 ' 300,500 Hz).
101 sound_place_ship: PROCEDURE
102     #snd_val = 373 : snd_gate = 2 : snd_post = 0 : GOSUB play_tone
103     #snd_val = 224 : snd_gate = 3 : snd_post = 2 : GOSUB play_tone
104 END
105
106 ' sound_attack: a shot was fired (falling swoosh, 500,300 Hz).
107 sound_attack: PROCEDURE
108     #snd_val = 224 : snd_gate = 3 : snd_post = 0 : GOSUB play_tone
109     #snd_val = 373 : snd_gate = 3 : snd_post = 0 : GOSUB play_tone
110 END
111
112 ' sound_miss: shot landed in water (single splash-like click, 800 Hz).
113 sound_miss: PROCEDURE
114     #snd_val = 140 : snd_gate = 4 : snd_post = 4 : GOSUB play_tone
115 END
116
117 ' sound_hit: shot struck a ship (low descending buzz, 200,100 Hz).
118 sound_hit: PROCEDURE
119     #snd_val = 559 : snd_gate = 4 : snd_post = 0 : GOSUB play_tone
120     #snd_val = 1119 : snd_gate = 6 : snd_post = 4 : GOSUB play_tone
121 END
122
123 ' sound_sink: a ship was sunk (4-tone falling fanfare, mirror of
124 ' sound_gamedone: 415,392,330,311 Hz).
125 sound_sink: PROCEDURE
126     #snd_val = 270 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
127     #snd_val = 285 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
128     #snd_val = 339 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
129     #snd_val = 360 : snd_gate = 15 : snd_post = 0 : GOSUB play_tone
130 END

```

```

131
132 ' sound_invalid: rejected input -- ship placement collision, spent attack
133 ' cell, out-of-turn action (single low buzz, 100 Hz, held).
134 sound_invalid: PROCEDURE
135 #snd_val = 1119 : snd_gate = 10 : snd_post = 5 : GOSUB play_tone
136 END
137

```

LISTING 10 fujitzee.bas — Fujitzee client (Chapter 10)

```

1 ' fujitzee.bas -- FujiNet Fujitzee (Yahtzee-style) for Intellivision, entry
2 ' point + state machine. Follows the pattern established by
3 ' fujinet-5cardstud/intv/5card.bas and fujinet-battleship/intv/battleship.bas
4 ' (name entry -> table select -> game loop -> in-game menu), adapted for a
5 ' scorecard game: the 20x12 screen can't show all 6 players' 15-category
6 ' cards at once, so one player's card is shown at a time (card.bas), paged
7 ' with the disc when it isn't your turn, alongside a permanent 6-seat strip.
8
9 ' GOTO boot_start jumps over every INCLUDE below before any of it runs --
10 ' see fujinet.bas for why: falling into a PROCEDURE or DATA block by
11 ' straight-line execution corrupts the return stack.
12 ' GOTO boot_start
13
14 INCLUDE "constants.bas"
15 INCLUDE "fujinet.bas"
16 INCLUDE "state.bas"
17 INCLUDE "dice.bas"
18 INCLUDE "card.bas"
19 INCLUDE "sound.bas"
20
21 CONST ROWCELLS = 20
22 CONST STATUS_ROW = screenpos(0, 11)
23
24 '-----
25 ' URL literals (ASCII DATA). url_path selects the endpoint:
26 ' 0 tables 1 state 2 ready 3 roll<mask> 4 score<idx> 5 leave
27 '-----
28 lit_n_colon: DATA 78,58
29 lit_https: DATA 104,116,116,112,115,58,47,102,117,106,105,116,122,101,101,46,
99,97,114,114,45,100,101,115,105,103,110,115,46,99,111,109,47
30 lit_tables: DATA 116,97,98,108,101,115
31 lit_qbin: DATA 63,98,105,110,61,49
32 lit_state: DATA 115,116,97,116,101
33 lit_ready: DATA 114,101,97,100,121
34 lit_roll: DATA 114,111,108,108,47
35 lit_score: DATA 115,99,111,114,101,47
36 lit_leave: DATA 108,101,97,118,101
37 lit_qtable: DATA 63,116,97,98,108,101,61
38 lit_ain: DATA 38,112,108,97,121,101,114,61
39 lit_abin: DATA 38,98,105,110,61,49
40
41 CONST LEN_HTTPS = 34
42
43 ' Lobby appkey: creator 1 / app 1, key = this game's registered slot.
44 ' See fujinet-fujitzee/src/misc.h AK_LOBBY_KEY_SERVER.
45 CONST AK_LOBBY_KEY_SERVER = 3
46
47 '-----
48 ' Globals
49 '-----
50 DIM gs_i, gs_j, #gs_char, #gs_c
51 DIM ne_i, ne_j, ne_cur, ne_len
52 DIM ne_buf(8)
53 DIM ak_try
54 DIM #tbl_count, tbl_sel
55 DIM inp_lock
56 DIM want_leave, im_sel
57 DIM url_path
58 DIM #tmp_addr
59
60 ' split_room_url locals (see procedure below)
61 DIM sp_i, sp_found, sp_ok, sp_j, sp_k, sp_c, sp_valid, sp_m
62
63 DIM #cur_round, #cur_pc, prev_round, prev_active, prev_seen_pc, my_turn
64 DIM poll_wait, has_action
65 DIM turn_changed, roll_changed, #prev_dice_rollsleft, shadow_seeded
66 DIM view_lock, ui_mode, dice_cur, crs_idx, score_idx
67
68 DIM pn_val, pn_h, pn_t, pn_o, pn_started
69 DIM #ti_timeleft, ti_framecount, ti_allkept, pt_i, score_is_fujitzee,
at_bottom_row
70
71 '-----
72 ' fn_putnum: append the decimal (no leading zeros) representation of pn_val
73 ' (0-999) into FN_TX at the current #fn_txlen. IntyBASIC has no built-in
74 ' itoa; score indices only run 0-14, but a 3-digit routine costs nothing
75 ' extra and matches fujinet-battleship/intv/battleship.bas's version.
76 '-----
77 fn_putnum: PROCEDURE
78 pn_h = pn_val / 100
79 pn_t = (pn_val / 10) % 10
80 pn_o = pn_val % 10

```

```

81  pn_started = 0
82  IF pn_h > 0 THEN
83    POKE (FN_TX + #fn_txlen), pn_h + 48 : #fn_txlen = #fn_txlen + 1
84    pn_started = 1
85  END IF
86  IF pn_t > 0 OR pn_started THEN
87    POKE (FN_TX + #fn_txlen), pn_t + 48 : #fn_txlen = #fn_txlen + 1
88  END IF
89  POKE (FN_TX + #fn_txlen), pn_o + 48 : #fn_txlen = #fn_txlen + 1
90 END
91
92 ' -----
93 ' draw_field: render df_len ASCII bytes from #df_src onto the screen at
94 ' BACKTAB offset df_pos, in color #df_color. MODE 1 only exposes GROM cards
95 ' 0-63 (ASCII 32-95, no lowercase), so unlike Battleship's MODE 0 client
96 ' this uppercases -- server strings (names, prompts) arrive lowercased.
97 ' Stops at a NUL and pads the rest of the field with spaces; clamps
98 ' anything outside the printable range to a space rather than drawing
99 ' garbage. Ported from fujinet-5cardstud/intv/5card.bas's version.
100 ' -----
101 DIM df_i, #df_c, df_stop
102 draw_field: PROCEDURE
103   df_stop = 0
104   FOR df_i = 0 TO df_len - 1
105     #df_c = (PEEK(#df_src + df_i) AND 255)
106     IF #df_c = 0 THEN df_stop = 1
107     IF df_stop THEN
108       #df_c = 32
109     ELSEIF #df_c >= 97 AND #df_c <= 122 THEN
110       #df_c = #df_c - 32
111     END IF
112     IF #df_c < 32 OR #df_c > 95 THEN #df_c = 32
113     #BACKTAB(df_pos + df_i) = (#df_c - 32) * 8 + #df_color
114   NEXT df_i
115 END
116
117 ' -----
118 ' cls_blue: CLS, then fill the whole 240-cell BACKTAB with a blank blue
119 ' card. CLS alone leaves every cell at card 0 / color 0 (black), so
120 ' without this every screen would still show a black field behind
121 ' whatever text gets drawn on top of it. Every CLS in this file goes
122 ' through here instead, matching fujinet-5cardstud/intv/5card.bas's
123 ' fill_bg pattern.
124 ' -----
125 DIM cb_i
126 cls_blue: PROCEDURE
127   CLS
128   FOR cb_i = 0 TO 239
129     #BACKTAB(cb_i) = COL_TEXT
130   NEXT cb_i
131 END
132
133 ' -----

```

```

134 ' compose_url: build "N:<endpoint><path...><suffix>" directly into FN_TX.
135 ' url_path=3 (roll) appends SC_KEEP's 5 raw ascii mask bytes directly (it's
136 ' already in wire format, no NUL). url_path=4 (score) appends score_idx via
137 ' fn_putnum.
138 ' -----
139 compose_url: PROCEDURE
140   #fn_txlen = 0
141   #fn_src = VARPTR lit_n_colon(0) : fn_len = 2 : GOSUB fn_putstr
142   #fn_src = SC_ENDPT : ls_max = 65 : GOSUB fn_strlen : GOSUB fn_putstr
143
144   IF url_path = 0 THEN
145     #fn_src = VARPTR lit_tables(0) : fn_len = 6 : GOSUB fn_putstr
146     #fn_src = VARPTR lit_qbin(0) : fn_len = 6 : GOSUB fn_putstr
147   ELSE
148     IF url_path = 1 THEN
149       #fn_src = VARPTR lit_state(0) : fn_len = 5 : GOSUB fn_putstr
150     ELSEIF url_path = 2 THEN
151       #fn_src = VARPTR lit_ready(0) : fn_len = 5 : GOSUB fn_putstr
152     ELSEIF url_path = 3 THEN
153       #fn_src = VARPTR lit_roll(0) : fn_len = 5 : GOSUB fn_putstr
154       #fn_src = SC_KEEP : fn_len = 5 : GOSUB fn_putstr
155     ELSEIF url_path = 4 THEN
156       #fn_src = VARPTR lit_score(0) : fn_len = 6 : GOSUB fn_putstr
157       pn_val = score_idx
158       GOSUB fn_putnum
159     ELSE
160       #fn_src = VARPTR lit_leave(0) : fn_len = 5 : GOSUB fn_putstr
161     END IF
162
163     #fn_src = VARPTR lit_qtable(0) : fn_len = 7 : GOSUB fn_putstr
164     #fn_src = SC_TABLE : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr
165     #fn_src = VARPTR lit_ain(0) : fn_len = 8 : GOSUB fn_putstr
166     #fn_src = SC_NAME : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr
167     #fn_src = VARPTR lit_abin(0) : fn_len = 6 : GOSUB fn_putstr
168   END IF
169 END
170
171 ' =====
172 ' Boot
173 ' =====
174 boot_start:
175   MODE 1
176   GOSUB cls_blue
177   GOSUB dice_init
178   PRINT AT 0 COLOR COL_TEXT, "FUJITZEE"
179   PRINT AT 40, "CONNECTING TO FUJINET"
180   GOSUB fn_wait_mailbox
181   IF fn_ok = 0 THEN
182     PRINT AT 40, "NO CARTRIDGE MAILBOX"
183     GOTO halt
184   END IF
185
186 ' -----

```

```

187 ' Name: try the shared lobby appkey (creator 1 / app 1 / key 0) first, with
188 ' one retry (a cold RP2040/ESP32 link right after the mailbox comes up can
189 ' time out the very first transaction); fall back to the disc letter picker
190 ' if the read fails, comes back empty, or holds anything outside A-Z0-9 --
191 ' this slot is shared by every FujiNet client, and a stray character here
192 ' goes straight into the query string unescaped.
193 '-----
194   gs_i = 0
195   FOR ak_try = 0 TO 1
196     ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1 : ak_key = 0 :
197   ak_mode = 0
198   GOSUB appkey_open
199   IF fn_ok THEN EXIT FOR
200   NEXT ak_try
201   IF fn_ok THEN
202     #fn_src = SC_NAME : ls_max = 9 : GOSUB appkey_read
203     GOSUB appkey_close
204     IF fn_len > 0 THEN
205       gs_i = 1
206       FOR gs_j = 0 TO fn_len - 1
207         #gs_char = (PEEK(SC_NAME + gs_j) AND 255)
208         IF #gs_char >= 97 AND #gs_char <= 122 THEN #gs_char = #gs_char -
32
208         IF (#gs_char < 65 OR #gs_char > 90) AND (#gs_char < 48 OR
#gs_char > 57) THEN gs_i = 0
209       NEXT gs_j
210     END IF
211   END IF
212   IF gs_i = 0 THEN GOSUB name_entry_screen
213
214 '-----
215 ' Room: check the lobby server appkey (creator 1 / app 1 / key
216 ' AK_LOBBY_KEY_SERVER). If the lobby wrote a room there, split it into
217 ' SC_ENDPT/SC_TABLE and skip straight past table select. Seed the
218 ' compiled-in default endpoint first so SC_ENDPT is always valid even if
219 ' the appkey is empty or the read fails -- compose_url relies on it from
220 ' here on for every request, not just /tables.
221 '-----
222   FOR gs_i = 0 TO LEN_HTTPS - 1
223     POKE (SC_ENDPT + gs_i), PEEK(VARPTR lit_https(0) + gs_i) AND 255
224   NEXT gs_i
225   POKE (SC_ENDPT + LEN_HTTPS), 0
226   POKE SC_TABLE, 0
227
228   FOR ak_try = 0 TO 1
229     ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
230     ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 0
231     GOSUB appkey_open
232     IF fn_ok THEN EXIT FOR
233   NEXT ak_try
234   IF fn_ok THEN
235     #fn_src = SC_ENDPT : ls_max = 65 : GOSUB appkey_read
236     GOSUB appkey_close

```

```

237   IF fn_len > 0 THEN
238     GOSUB split_room_url
239   ELSE
240     ' appkey_read always NUL-terminates at fn_len, even when
241     ' empty, which would otherwise wipe out the default just
242     ' seeded above -- restore it.
243     FOR gs_i = 0 TO LEN_HTTPS - 1
244       POKE (SC_ENDPT + gs_i), PEEK(VARPTR lit_https(0) + gs_i) AND 255
245     NEXT gs_i
246     POKE (SC_ENDPT + LEN_HTTPS), 0
247   END IF
248 END IF
249
250   IF (PEEK(SC_TABLE) AND 255) <> 0 THEN GOTO table_joined
251
252 ' =====
253 ' Table select (re-entered after leaving a table)
254 ' =====
255 table_select:
256   GOSUB cls_blue
257   PRINT AT 0 COLOR COL_TEXT, "CHOOSE A TABLE"
258   PRINT AT 40, "REFRESHING..."
259
260   url_path = 0 : GOSUB compose_url
261   #net_readlen = TABLES_MAXLEN
262   GOSUB api_call
263
264   IF fn_ok = 1 THEN
265     #tmp_addr = 1 + (PEEK(FN_RX) AND 255) * TABLE_STRIDE
266     IF #net_gotlen < #tmp_addr THEN fn_ok = 0
267   END IF
268
269   IF fn_ok = 0 THEN
270     PRINT AT 40, "SERVER UNREACHABLE "
271     inp_lock = 90
272     WHILE inp_lock > 0
273       inp_lock = inp_lock - 1
274     WAIT
275   WEND
276   GOTO table_select
277 END IF
278
279   #tbl_count = (PEEK(FN_RX) AND 255)
280   IF #tbl_count > 8 THEN #tbl_count = 8
281   IF #tbl_count = 0 THEN
282     PRINT AT 40, "NO TABLES AVAILABLE "
283     inp_lock = 90
284     WHILE inp_lock > 0
285       inp_lock = inp_lock - 1
286     WAIT
287   WEND
288   GOTO table_select
289 END IF

```

```

290
291 GOSUB cls_blue
292 PRINT AT 0 COLOR COL_TEXT, "CHOOSE A TABLE"
293 FOR gs_i = 0 TO #tbl_count - 1
294   #tmp_addr = table_addr(gs_i)
295   #df_src = #tmp_addr + TBL_NAME : df_pos = 20 + gs_i * 20 + 1 : df_len =
14 : #df_color = COL_TEXT
296   GOSUB draw_field
297   #df_src = #tmp_addr + TBL_PLAYERS : df_pos = 20 + gs_i * 20 + 15 :
df_len = 5 : #df_color = COL_TEXT
298   GOSUB draw_field
299   NEXT gs_i
300
301   tbl_sel = 0
302   inp_lock = 0
303   ts_input:
304   WAIT
305   FOR gs_i = 0 TO #tbl_count - 1
306     #gs_c = COL_TEXT
307     IF gs_i = tbl_sel THEN #gs_c = COL_HILITE
308     #BACKTAB(20 + gs_i * 20) = (62 - 32) * 8 + #gs_c ' '>' cursor glyph
309   NEXT gs_i
310
311   IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO ts_input
312
313   IF CONT1.DOWN THEN
314     tbl_sel = tbl_sel + 1
315     IF tbl_sel >= #tbl_count THEN tbl_sel = 0
316     inp_lock = 8
317     GOSUB sound_cursor
318     GOTO ts_input
319   END IF
320   IF CONT1.UP THEN
321     IF tbl_sel = 0 THEN tbl_sel = #tbl_count
322     tbl_sel = tbl_sel - 1
323     inp_lock = 8
324     GOSUB sound_cursor
325     GOTO ts_input
326   END IF
327   IF CONT1.BUTTON = 0 THEN GOTO ts_input
328   GOSUB sound_select
329
330   #tmp_addr = table_addr(tbl_sel)
331   FOR gs_i = 0 TO 8
332     POKE (SC_TABLE + gs_i), PEEK(#tmp_addr + TBL_ID + gs_i) AND 255
333   NEXT gs_i
334
335   ' Update the lobby server appkey so a reboot without going back
336   ' through the lobby rejoins this table.
337   GOSUB write_room_appkey
338
339   table_joined:
340   ' =====

```

```

341 ' In a table: poll /state (or submit a pending ready/roll/score action in
342 ' its place) and dispatch on round. A full CLS happens only on the
343 ' transition into/out of play (lobby<->play<->gameover); every other poll
344 ' updates cells in place -- the STIC has no page-flip, so a per-poll CLS
345 ' would visibly blank the screen every cycle.
346 ' =====
347   prev_round = 255 ' sentinel: forces the initial CLS/card_init
348   prev_active = 255 ' sentinel: forces the initial turn-start reset+sound
349   prev_seen_pc = 255 ' sentinel: skip the join/leave cue on the first poll
350   poll_wait = 0
351   want_leave = 0
352   has_action = 0
353   view_idx = 0 : view_lock = 0
354   ui_mode = 0 : crs_idx = 0 : dice_cur = 0
355   GOSUB sound_join
356
357   game_loop:
358   WAIT
359
360   IF poll_wait > 0 THEN
361     poll_wait = poll_wait - 1
362     GOTO gl_input
363   END IF
364
365   IF has_action = 1 THEN
366     url_path = 2
367   ELSEIF has_action = 2 THEN
368     url_path = 3
369   ELSEIF has_action = 3 THEN
370     url_path = 4
371   ELSE
372     url_path = 1
373   END IF
374   has_action = 0
375   GOSUB compose_url
376   #net_readlen = GAME_MAXLEN
377   GOSUB api_call
378   GOSUB validate_state
379
380   IF fn_ok = 0 THEN
381     poll_wait = 30
382     GOTO gl_input
383   END IF
384
385   #cur_round = state_round
386   #cur_pc = state_playercount
387   IF view_idx >= #cur_pc THEN view_idx = 0
388
389   IF prev_seen_pc <> 255 THEN
390     IF #cur_pc > prev_seen_pc THEN GOSUB sound_player_join
391     IF #cur_pc < prev_seen_pc THEN GOSUB sound_player_left
392   END IF
393   prev_seen_pc = #cur_pc

```

```

394
395 IF #cur_round = ROUND_LOBBY THEN
396     IF prev_round <> ROUND_LOBBY THEN GOSUB cls_blue
397     GOSUB render_lobby
398     poll_wait = 45
399 ELSEIF #cur_round = ROUND_GAMEOVER THEN
400     IF prev_round <> ROUND_GAMEOVER THEN
401         GOSUB cls_blue
402         GOSUB render_gameover
403     END IF
404     poll_wait = 60
405 ELSE
406     IF prev_round = ROUND_LOBBY OR prev_round = 255 OR prev_round =
ROUND_GAMEOVER THEN
407         GOSUB cls_blue
408         GOSUB card_init
409         view_idx = 0 : ui_mode = 0 : crs_idx = 0 : dice_cur = 0
410         #prev_dice_rollsleft = 255 ' sentinel: don't animate the first paint
411         shadow_seeded = 0 ' sentinel: don't reveal-check against garbage
412         FOR pt_i = 0 TO 4
413             POKE (SC_KEEP + pt_i), 49
414         NEXT pt_i
415     END IF
416
417     ' Whenever the active player changes (a new turn begins, yours or
418     ' someone else's), snap the viewed card to follow them, like the
419     ' other clients -- no need to manually page to see who's rolling.
420     ' Manual disc-paging in gl_input still works to peek elsewhere
421     ' while waiting; it's just overridden at the next turn change.
422     turn_changed = 0
423     IF active_player <> prev_active THEN turn_changed = 1
424
425     ' Someone other than you just finished their turn (scored, or
426     ' timed out and got auto-scored) -- flash whatever they picked
427     ' before the view below flips over to whoever's active now. Skips
428     ' your own turn ending: you already saw that happen live, driven
429     ' by turn_input.
430     IF shadow_seeded THEN
431         IF turn_changed THEN
432             IF prev_active <> 0 THEN
433                 IF prev_active >= 0 THEN
434                     IF prev_active < #cur_pc THEN GOSUB check_score_reveal
435                 END IF
436             END IF
437         END IF
438     END IF
439
440     ' Deliberately nested single-condition IFs, not
441     ' "IF active_player = 0 AND state_viewing = 0 THEN": IntyBASIC
442     ' v1.4.2 miscompiles a chain of two "X = const" comparisons into
443     ' by AND when one side's value comes from an expression that
444     ' itself contains "AND 255" (state_viewing's DEF FN body, or
445     ' active_player's sign-extension formula) -- the fused codegen

```

```

' ANDs in a stray always-zero register instead of the second
' comparison, so the whole condition is always false. Confirmed
' by reading the generated .lst: it never called turn_input.
my_turn = 0
IF active_player = 0 THEN
    IF state_viewing = 0 THEN my_turn = 1
END IF

IF turn_changed THEN
    IF active_player >= 0 THEN
        IF active_player < #cur_pc THEN view_idx = active_player
    END IF
    IF my_turn THEN
        GOSUB sound_myturn
        ui_mode = 0 : crs_idx = 0 : dice_cur = 0
        FOR pt_i = 0 TO 4
            POKE (SC_KEEP + pt_i), 49
        NEXT pt_i
    END IF
END IF

' Snapshot whoever's active now, so whenever *their* turn ends we
' have a valid "before" state for check_score_reveal to diff
' against -- done every poll (cheap: 15 words), not just on
' turn_changed, since it must reflect their latest scoring right
' up until the moment they stop being active.
IF active_player >= 0 THEN
    IF active_player < #cur_pc THEN
        GOSUB seed_score_shadow
        shadow_seeded = 1
    END IF
END IF

' A roll landed for whoever's turn it is -- rollsLeft changed
' since the last poll, or the turn itself just changed (covers
' the server's automatic opening roll). Flicker the dice the
' wire's keepRoll says were rerolled before drawing the settled
' result; see dice.bas's animate_roll for why this same check
' animates both your own rolls and every other player's.
roll_changed = 0

IF #prev_dice_rollsleft <> 255 THEN
    IF state_rollsleft <> #prev_dice_rollsleft THEN roll_changed = 1
    IF turn_changed THEN roll_changed = 1
    IF roll_changed THEN GOSUB animate_roll
END IF
#prev_dice_rollsleft = state_rollsleft

' Out of rolls -- jump to SCORE mode automatically, like the
' other clients (gamelogic.c defaults cursorPos onto the
' highest-scoring open category the moment rollsLeft hits 0).
' Gated on roll_changed so this fires once, not every poll while
' sitting at 0 (which would otherwise fight any manual navigation

```



```

498 ' the player had already done).
499 IF roll_changed THEN
500     IF my_turn THEN
501         IF state_rollsleft = 0 THEN
502             ui_mode = 1
503             GOSUB pick_best_score
504         END IF
505     END IF
506 END IF
507
508 GOSUB render_playscreen
509
510 IF my_turn THEN
511     GOSUB turn_input
512     IF has_action THEN poll_wait = 0 ELSE poll_wait = 20
513 ELSE
514     poll_wait = 20
515 END IF
516 END IF
517 prev_active = active_player
518 prev_round = #cur_round
519
520 gl_input:
521 ' Page the viewed player's card with the disc when it isn't your turn
522 ' (during your own turn, turn_input owns the disc for die/category
523 ' selection instead). No network round-trip needed -- every seated
524 ' player's record already arrived in the same poll. Reuses my_turn
525 ' (computed once per successful poll, above) rather than re-testing
526 ' active_player/state_viewing here -- see the comment at that
527 ' computation for why chaining those DEF FNs into a multi-condition
528 ' AND on one line miscompiles on this compiler.
529 IF #cur_round >= 1 THEN
530     IF #cur_round <= ROUND_FINAL THEN
531         IF my_turn = 0 THEN
532             IF view_lock > 0 THEN view_lock = view_lock - 1
533             IF CONT1.RIGHT AND view_lock = 0 THEN
534                 view_idx = view_idx + 1
535                 IF view_idx >= #cur_pc THEN view_idx = 0
536                 view_lock = 8
537                 GOSUB sound_cursor
538                 GOSUB render_header
539                 GOSUB render_card_for_view
540             ELSEIF CONT1.LEFT AND view_lock = 0 THEN
541                 IF view_idx = 0 THEN view_idx = #cur_pc
542                 view_idx = view_idx - 1
543                 view_lock = 8
544                 GOSUB sound_cursor
545                 GOSUB render_header
546                 GOSUB render_card_for_view
547             END IF
548         END IF
549     END IF
550 END IF

```

```

551
552 IF CONT1.KEY = 10 THEN GOSUB ingame_menu
553 IF want_leave THEN
554     want_leave = 0
555     GOTO table_select
556 END IF
557 GOTO game_loop
558
559 ' =====
560 ' render lobby: server name, ready list, prompt. Trigger toggles /ready.
561 ' Uses the same 42-byte player records as play -- scores[0] doubles as the
562 ' ready flag while round=0 (SCORE_READY/SCORE_UNREADY/SCORE_VIEWING).
563 ' =====
564 DIM #lb_pc, lb_ready
565 render_lobby: PROCEDURE
566     #df_src = FN_RX + GAME_SERVERNAME : df_pos = screenpos(0, 0) : df_len = 20 :
567     #df_color = COL_TEXT
568     GOSUB draw_field
569
570     #lb_pc = state_playercount
571     IF #lb_pc > 8 THEN #lb_pc = 8
572     FOR gs_i = 0 TO #lb_pc - 1
573         #tmp_addr = player_addr(gs_i)
574         pc_idx = gs_i
575         GOSUB player_color
576         #df_src = #tmp_addr + PL_NAME : df_pos = screenpos(0, gs_i + 2) : df_len
577         = 9 : #df_color = #pc_color
578         GOSUB draw_field
579         #gs_c = 46 ' '.'
580         lb_ready = score_at(#tmp_addr, 0)
581         IF lb_ready = SCORE_READY THEN #gs_c = 42 ' '*'
582         #BACKTAB(screenpos(10, gs_i + 2)) = (#gs_c - 32) * 8 + COL_TEXT
583     NEXT gs_i
584
585     #df_src = FN_RX + GAME_PROMPT : df_pos = STATUS_ROW : df_len = ROWCELLS :
586     #df_color = COL_TEXT
587     GOSUB draw_field
588
589     IF CONT1.BUTTON THEN
590         GOSUB sound_select
591         url_path = 2 : GOSUB compose_url
592         #net_readlen = GAME_MAXLEN
593         GOSUB api_call
594         poll_wait = 15
595     END IF
596 END
597
598 ' =====
599 ' render_gameover: standings (name + final total, same running-total math
600 ' that's live during play, so it doesn't depend on the server's own
601 ' scores[15] convention) plus the prompt naming the winner. Held on screen
602 ' (poll_wait=60) until the server resets the table back to the lobby.
603 ' =====

```

```

601 render_gameover: PROCEDURE
602   GOSUB sound_gamedone
603   prl_top = 1
604   GOSUB render_player_list
605   #df_src = FN_RX + GAME_PROMPT : df_pos = STATUS_ROW : df_len = ROWCELLS :
#df_color = COL_HILITE
606   GOSUB draw_field
607   END
608
609 ' =====
610 ' render_header: viewed player's name and the round counter.
611 ' =====
612 render_header: PROCEDURE
613   #tmp_addr = player_addr(view_idx)
614   pc_idx = view_idx
615   GOSUB player_color
616   #df_src = #tmp_addr + PL_NAME : df_pos = screenpos(0, 0) : df_len = 9 :
#df_color = #pc_color
617   GOSUB draw_field
618   PRINT AT screenpos(10, 0) COLOR COL_ROUND, "R"
619   PRINT AT screenpos(12, 0) COLOR COL_ROUND, "<.2>#cur_round
620   PRINT AT screenpos(14, 0) COLOR COL_ROUND, "/13"
621   PRINT AT screenpos(18, 0) COLOR COL_TEXT, "<>"
622   END
623
624 ' =====
625 ' render_card_for_view: point card.bas's render_card at the currently
626 ' viewed player, enabling the green "potential score" hint only on your own
627 ' card during your own turn.
628 ' =====
629 render_card_for_view: PROCEDURE
630   #rc_addr = player_addr(view_idx)
631   rc_own = 0
632   ' Nested single-condition IFs, not a chained "A = 0 AND B = 0 AND
633   ' C = 0" -- see the comment at game_loop's turn check for why that
634   ' shape miscompiles to always-false on this compiler.
635   IF view_idx = 0 THEN
636     IF active_player = 0 THEN
637       IF state_viewing = 0 THEN rc_own = 1
638     END IF
639   END IF
640   rc_mode = ui_mode
641   IF crs_idx < 6 THEN rc_selcat = crs_idx ELSE rc_selcat = crs_idx + 2
642   rc_reveal = 255
643   GOSUB render_card
644   END
645
646 ' =====
647 ' seed_score_shadow: snapshot the current active player's scores[0..14]
648 ' into SC_PREVSCORES. Called every poll for whoever's active right now, so
649 ' that whenever *their* turn eventually ends, check_score_reveal has a
650 ' valid "before" state to diff against.
651 ' =====

```

```

652 DIM #rv_addr, rv_i, #rv_new, #rv_old, rv_found, rv_cat, rv_frame
653 seed_score_shadow: PROCEDURE 4
654   #rv_addr = player_addr(active_player)
655   FOR rv_i = 0 TO 14
656     #rv_new = score_at(#rv_addr, rv_i)
657     POKE (SC_PREVSCORES + rv_i * 2), #rv_new AND 255
658     POKE (SC_PREVSCORES + rv_i * 2 + 1), (#rv_new / 256) AND 255
659   NEXT rv_i
660   END
661
662 ' =====
663 ' check_score_reveal: called right when turn_changed detects someone other
664 ' than you just finished their turn. Diffs their scores[] against the
665 ' shadow captured the last time seed_score_shadow saw them become active,
666 ' to find which single category they just filled in (skipping indices 6/7,
667 ' the computed upper-total/bonus rows -- a player never directly picks
668 ' those, they only change as a side effect of an upper-section pick, which
669 ' the 0-5 scan already catches first). Shows their card with that cell
670 ' highlighted for about 3/4 of a second before the normal turn-changed
671 ' flow (which reassigns view_idx to whoever's active now) takes over. A
672 ' no-op if nothing actually changed -- e.g. the departing "player" is a
673 ' sentinel (-1, nobody), or the shadow was never seeded for them.
674 ' =====
675 check_score_reveal: PROCEDURE
676   #rv_addr = player_addr(prev_active)
677   rv_found = 0
678   FOR rv_i = 0 TO 14
679     IF rv_i <> 6 THEN
680       IF rv_i <> 7 THEN
681         #rv_new = score_at(#rv_addr, rv_i)
682         #rv_old = (PEEK(SC_PREVSCORES + rv_i * 2 + 1) AND 255) * 256 +
(PEEK(SC_PREVSCORES + rv_i * 2) AND 255)
683         IF rv_found = 0 THEN
684           IF #rv_new <> #rv_old THEN
685             IF #rv_new <> SCORE_UNSET THEN
686               IF #rv_new <> SCORE_VIEWING THEN
687                 rv_found = 1 : rv_cat = rv_i
688               END IF
689             END IF
690           END IF
691         END IF
692       END IF
693     END IF
694   NEXT rv_i
695
696   IF rv_found THEN
697     pc_idx = prev_active
698     GOSUB player_color
699     #df_src = #rv_addr + PL_NAME : df_pos = screenpos(0, 0) : df_len = 9 :
#df_color = #pc_color
700     GOSUB draw_field
701     #rc_addr = #rv_addr

```

```

702     rc_own = 0 : rc_mode = 0 : rc_selcat = 0 : rc_reveal = rv_cat
703     GOSUB render_card
704     GOSUB sound_score
705     FOR rv_frame = 1 TO 45
706         WAIT
707     NEXT rv_frame
708 END IF
709 END
710
711 ' =====
712 ' render_playscreen: full redraw of the round 1-13 screen from whatever is
713 ' currently sitting in FN_RX -- header, card, seat strip, dice, prompt.
714 ' Split with WAITs between regions (render_card already breaks its own two
715 ' columns) so one poll's redraw can't tear a frame; matches board.bas's
716 ' per-quadrant WAIT and 5card.bas's per-seat WAIT, for the same reason
717 ' (STIC has no page-flip; a big unbroken BACKTAB write can spill past
718 ' vblank into active display).
719 ' =====
720 render_playscreen: PROCEDURE
721     GOSUB render_header
722     GOSUB render_card_for_view
723     WAIT
724     GOSUB render_seatstrip
725     dr_mode = ui_mode : dr_cursor = dice_cur
726     GOSUB draw_dice_row
727     #df_src = FN_RX + GAME_PROMPT : df_pos = STATUS_ROW : df_len = ROWCELLS :
728     #df_color = COL_TEXT
729     GOSUB draw_field
730 END
731
732 ' =====
733 ' pick_best_score: point crs_idx at whichever open, selectable category
734 ' scores the most with the current (final) roll. Used when rollsLeft hits
735 ' 0 and SCORE mode is entered automatically -- matches the reference C
736 ' clients defaulting cursorPos onto the highest score once no rolls
737 ' remain (gameLogic.c). Categories 0-5 map to crs_idx 0-5 directly;
738 ' 8-14 map to crs_idx 6-12 (see render_card's/turn_input's own comments
739 ' for that mapping). Falls back to crs_idx=0 if nothing is open at all
740 ' (shouldn't happen mid-round, but leaves a sane cursor position either
741 ' way).
742 ' =====
743 DIM pbs_found, #pbs_best, pbs_i, #pbs_v
744 pick_best_score: PROCEDURE 2
745     pbs_found = 0
746     #pbs_best = 0
747     crs_idx = 0
748     FOR pbs_i = 0 TO 5
749         #pbs_v = valid_at(pbs_i)
750         IF #pbs_v <> 255 THEN
751             IF pbs_found = 0 THEN
752                 pbs_found = 1 : #pbs_best = #pbs_v : crs_idx = pbs_i
753             ELSEIF #pbs_v > #pbs_best THEN
754                 #pbs_best = #pbs_v : crs_idx = pbs_i
755             END IF
756         END IF
757     NEXT pbs_i
758 END

```

```

753     #pbs_best = #pbs_v : crs_idx = pbs_i
754 END IF
755 END IF
756 NEXT pbs_i
757 FOR pbs_i = 8 TO 14
758     #pbs_v = valid_at(pbs_i)
759     IF #pbs_v <> 255 THEN
760         IF pbs_found = 0 THEN
761             pbs_found = 1 : #pbs_best = #pbs_v : crs_idx = pbs_i - 2
762         ELSEIF #pbs_v > #pbs_best THEN
763             #pbs_best = #pbs_v : crs_idx = pbs_i - 2
764         END IF
765     END IF
766 NEXT pbs_i
767 END
768
769 ' =====
770 ' turn_input: your turn. Blocks in its own WAIT loop (like Battleship's
771 ' targeting) handling DICE mode (pick a die, toggle its reroll flag, submit
772 ' a roll) and SCORE mode (pick a category, score it), switched with the
773 ' bottom-left button. Sets has_action + score_idx/SC_KEEP and returns to
774 ' let the outer poll loop perform the actual network call, exactly like
775 ' Battleship's has_targeted/atk_pos handoff.
776 ' =====
777 turn_input: PROCEDURE 1
778     inp_lock = 0
779     has_action = 0
780     #ti_timeleft = (PEEK(FN_RX + GAME_MOVETIME) AND 255)
781     ti_framecount = 0
782     ' moveTime can run up to 250 (state.bas comment), so the field needs 3
783     ' digits -- see battleship.bas's targeting for why <.3> (space-padded,
784     ' PRNUM16.b) is used instead of <3> (zero-padded, PRNUM16.2): .z never
785     ' initializes the register that picks blank-vs-zero padding, so a
786     ' shorter value can print garbage in the unused leading digit(s).
787     PRINT AT STATUS_ROW + 17 COLOR COL_TEXT, <.3>#ti_timeleft
788
789     ti_loop:
790         WAIT
791
792         ti_framecount = ti_framecount + 1
793         IF ti_framecount >= 60 THEN
794             ti_framecount = 0
795             IF #ti_timeleft > 0 THEN #ti_timeleft = #ti_timeleft - 1
796             PRINT AT STATUS_ROW + 17 COLOR COL_TEXT, <.3>#ti_timeleft
797             GOSUB sound_tick
798             ' The server enforces its own move time limit and force-scores a
799             ' player who runs out (gameLogic.go's forceHumanMove) -- without
800             ' this bail, a player who never acts would stay stuck in this loop
801             ' forever and never poll again to discover the turn moved on.
802             IF #ti_timeleft = 0 THEN RETURN
803         END IF
804

```

```

805 IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO ti_loop
806
807 IF CONT1.KEY = 10 THEN
808   GOSUB ingame_menu
809   IF want_leave THEN RETURN
810   GOTO ti_loop
811 END IF
812 IF CONT1.KEY = 11 THEN
813   GOSUB show_standings
814   GOSUB render_card_for_view
815   GOTO ti_loop
816 END IF
817
818 IF ui_mode = 0 THEN
819   ' DICE mode. B1/B2/UP are checked *before* BUTTON deliberately:
820   ' the Intellivision controller's three action-button signals share
821   ' overlapping bits (constants.bas's BUTTON_MASK = BUTTON_1 OR
822   ' BUTTON_2 OR BUTTON_3), so CONT1.BUTTON reads non-zero for *any*
823   ' of the three side buttons, not just the top one. Checking it
824   ' first meant every button press -- including B1 and B2 -- fell
825   ' into the "toggle hold" branch and B1/B2 were never reached.
826   ' Battleship's handle_placement uses this same before-BUTTON
827   ' ordering for CONT1.B1 for the identical reason.
828   ' dice_cur ranges 0-5: 0 is the on-screen ROLL tile (drawn at
829   ' col 0 by draw_dice_row), 1-5 are dice 0-4 -- matching the
830   ' reference clients' own cursorPos convention (0=roll, 1-5=dice),
831   ' rather than a dedicated controller button for rolling.
832   IF CONT1.RIGHT THEN
833     dice_cur = (dice_cur + 1) % 6
834     inp_lock = 8 : GOSUB sound_cursor
835     dr_mode = ui_mode : dr_cursor = dice_cur : GOSUB draw_dice_row
836     GOTO ti_loop
837   END IF
838   IF CONT1.LEFT THEN
839     dice_cur = (dice_cur + 5) % 6
840     inp_lock = 8 : GOSUB sound_cursor
841     dr_mode = ui_mode : dr_cursor = dice_cur : GOSUB draw_dice_row
842     GOTO ti_loop
843   END IF
844   IF CONT1.UP OR CONT1.B1 THEN
845     ' Move from the dice row up into the scorecard -- your current
846     ' roll's potential scores are already shown in green there
847     ' (render_card_for_view/draw_cat_cell), so this is how you
848     ' "bank" a roll instead of rolling again.
849     ui_mode = 1
850     inp_lock = 10 : GOSUB sound_select
851     dr_mode = ui_mode : dr_cursor = dice_cur : GOSUB draw_dice_row
852     GOSUB render_card_for_view
853     GOTO ti_loop
854   END IF
855   IF CONT1.BUTTON THEN
856     IF dice_cur = 0 THEN
857       ' On the ROLL tile -- submit. Rejected (sound_invalid,
858
859   ' stay put) if there are no rolls left, or if every die is
860   ' marked KEEP (mask='0') so nothing would actually
861   ' reroll. The untouched default (every die still
862   ' '1'=reroll) is itself a valid "reroll all 5" submission.
863   IF state_rollsleft = 0 THEN
864     GOSUB sound_invalid
865     GOTO ti_loop
866   END IF
867   ti_allkept = 1
868   FOR pt_i = 0 TO 4
869     IF (PEEK(SC_KEEP + pt_i) AND 255) = 49 THEN ti_allkept = 0
870   NEXT pt_i
871   IF ti_allkept THEN
872     GOSUB sound_invalid
873     GOTO ti_loop
874   END IF
875   GOSUB sound_roll
876   has_action = 2
877   RETURN
878 ELSE
879   ' On a die -- toggle it between its default (reroll, '1')
880   ' and kept ('0').
881   pt_i = dice_cur - 1
882   IF (PEEK(SC_KEEP + pt_i) AND 255) = 49 THEN
883     POKE (SC_KEEP + pt_i), 48
884   ELSE
885     POKE (SC_KEEP + pt_i), 49
886   END IF
887   GOSUB sound_hold
888   dr_mode = ui_mode : dr_cursor = dice_cur : GOSUB draw_dice_row
889   inp_lock = 8
890   GOTO ti_loop
891 END IF
892 END IF
893 ELSE
894   ' SCORE mode.
895   IF CONT1.DOWN THEN
896     ' Pressing down from the bottom row of either column drops
897     ' back to DICE mode instead of wrapping to the top -- unless
898     ' there are no rolls left, in which case DICE mode has
899     ' nothing to do (matches the B1 guard just above/below).
900     at_bottom_row = 0
901     IF crs_idx = 5 THEN at_bottom_row = 1
902     IF crs_idx = 12 THEN at_bottom_row = 1
903     IF at_bottom_row THEN
904       IF state_rollsleft <= 0 THEN
905         ui_mode = 0
906         inp_lock = 10 : GOSUB sound_select
907         dr_mode = ui_mode : dr_cursor = dice_cur : GOSUB
908         GOSUB render_card_for_view
909         GOTO ti_loop
910       END IF
911     END IF
912   END IF

```

```

910     END IF
911     IF crs_idx < 6 THEN crs_idx = (crs_idx + 1) % 6 ELSE crs_idx = 6 +
((crs_idx - 6 + 1) % 7)
912     inp_lock = 8 : GOSUB sound_cursor : GOSUB render_card_for_view
913     GOTO ti_loop
914 END IF
915 IF CONT1.UP THEN
916     IF crs_idx < 6 THEN crs_idx = (crs_idx + 5) % 6 ELSE crs_idx = 6 +
((crs_idx - 6 + 5) % 7)
917     inp_lock = 8 : GOSUB sound_cursor : GOSUB render_card_for_view
918     GOTO ti_loop
919 END IF
920 IF CONT1.RIGHT AND crs_idx < 6 THEN
921     pt_i = crs_idx : IF pt_i > 6 THEN pt_i = 6
922     crs_idx = 6 + pt_i
923     inp_lock = 8 : GOSUB sound_cursor : GOSUB render_card_for_view
924     GOTO ti_loop
925 END IF
926 IF CONT1.LEFT AND crs_idx >= 6 THEN
927     pt_i = crs_idx - 6 : IF pt_i > 5 THEN pt_i = 5
928     crs_idx = pt_i
929     inp_lock = 8 : GOSUB sound_cursor : GOSUB render_card_for_view
930     GOTO ti_loop
931 END IF
932 IF CONT1.B1 THEN
933     ' No rolls left means DICE mode has nothing to do (B2 there
934     ' would just reject every attempt) -- stay in SCORE mode
935     ' instead of bouncing back to a dead end.
936     IF state_rollsleft = 0 THEN
937         GOSUB sound_invalid
938         GOTO ti_loop
939     END IF
940     ui_mode = 0
941     inp_lock = 10 : GOSUB sound_select
942     dr_mode = ui_mode : dr_cursor = dice_cur : GOSUB draw_dice_row
943     GOSUB render_card_for_view
944     GOTO ti_loop
945 END IF
946 IF CONT1.BUTTON THEN
947     IF crs_idx < 6 THEN score_idx = crs_idx ELSE score_idx = crs_idx + 2
948
949     IF valid_at(score_idx) = 255 THEN
950         GOSUB sound_invalid
951         GOTO ti_loop
952     END IF
953     score_is_fujitzee = 0
954     IF score_idx = SCORE_FUJITZEE THEN
955         IF valid_at(score_idx) > 0 THEN score_is_fujitzee = 1
956     END IF
957     IF score_is_fujitzee THEN
958         GOSUB sound_fujitzee
959     ELSE

```

```

959         GOSUB sound_score
960     END IF
961     has_action = 3
962     RETURN
963 END IF
964 END IF
965 GOTO ti_loop
966 END
967
968 ' The compiled program overflows IntyBASIC's default $5000-$6FFF
969 ' (8192-word) window -- the compiler happily keeps allocating past $6FFF,
970 ' but $7000 is not in the manual's list of ranges "usable without
971 ' additional programming" on modern flash cart/homebrew PCBs (that list
972 ' jumps from $6FFF to $A000). This ORG was originally placed just before
973 ' halt:, at the very end of the file -- which put it "after"
974 ' name_entry_screen, so name_entry_screen's own tail end (its "NAME NOT
975 ' SAVED" error path) still spilled about 20 words into that unsafe $7000
976 ' gap (confirmed in the .lst: code was landing at $7000-$7013). Moving the
977 ' ORG here, ahead of both ingame_menu and name_entry_screen, pushes that
978 ' whole "cold" tail (menu, name entry) into $D000+ instead, closing the
979 ' gap entirely. This matters beyond spec-purity: FujiNet GO's boot loader
980 ' hung on mount with the previous placement, consistent with $7000 not
981 ' being safely mapped as ROM on at least one real target this ships to,
982 ' even though jzIntv's emulation is permissive enough not to visibly
983 ' choke on it. Relocating everything from here to halt: into $D000-$FFFF
984 ' keeps the whole ROM inside the address ranges the manual documents as
985 ' safe. Cross-segment GOSUB/GOTO compiles to ordinary absolute CP-1610
986 ' jumps/calls, so this is a bookkeeping split, not a correctness one.
987     ASM ORG $D000
988
989     -----
990     split_room_url / write_room_apkey / clear_room_apkey: lobby room-
991     apkey handling for the intv<->lobby handoff. Placed here (past the
992     ORG $D000 split above) rather than back up near compose_url where they
993     were first written -- they're only ever called from boot, table-select,
994     and ingame_menu, none of which are the hot per-frame game_loop path, so
995     like ingame_menu/name_entry_screen below they belong in the "cold" tail.
996     Adding them before the split pushed the $5000-$6FFF segment over its
997     8192-word budget and spilled ~200 words into the unsafe $7000-$7FFF
998     gap, which is exactly the "hung on mount" failure mode described above
999     -- moving them here is the fix. Cross-segment GOSUB/GOTO from boot_start
1000     and table_select back up before the split compiles to ordinary absolute
1001     jumps, so this is bookkeeping only, not a correctness change.
1002     -----
1003     split_room_url: given a lobby-supplied "https://host/?table=xyz" value
1004     already sitting in SC_ENDPT with its length in fn_len (as left by
1005     apkey_read), split it into the bare endpoint (SC_ENDPT, truncated at
1006     the '?') and the table id (SC_TABLE). Mirrors the C clients'
1007     welcomeActionVerifyServerDetails() '?' scan. Leaves SC_TABLE empty (a
1008     leading NUL) if there's no '?', the query isn't "table=...", or the id
1009     contains anything other than A-Z/a-z/0-9 -- it goes straight into a
1010     rebuilt query string unescaped, same reasoning as the username check
1011     above.

```

```

1012 split_room_url: PROCEDURE
1013     POKE SC_TABLE, 0
1014
1015     sp_found = 255 ' sentinel: not found
1016     sp_i = 0
1017     WHILE sp_i < fn_len
1018         IF (PEEK(SC_ENDPT + sp_i) AND 255) = 63 THEN ' '?'
1019             sp_found = sp_i
1020             EXIT WHILE
1021         END IF
1022         sp_i = sp_i + 1
1023     WEND
1024     IF sp_found = 255 THEN RETURN
1025
1026     ' Truncate the endpoint at the '?' regardless of what follows.
1027     POKE (SC_ENDPT + sp_found), 0
1028
1029     ' Require "table=" (6 bytes) right after the '?'.
1030     sp_ok = 0
1031     IF sp_found + 7 <= fn_len THEN
1032         sp_ok = 1
1033         IF (PEEK(SC_ENDPT + sp_found + 1) AND 255) <> 116 THEN sp_ok = 0 ' t
1034         IF (PEEK(SC_ENDPT + sp_found + 2) AND 255) <> 97 THEN sp_ok = 0 ' a
1035         IF (PEEK(SC_ENDPT + sp_found + 3) AND 255) <> 98 THEN sp_ok = 0 ' b
1036         IF (PEEK(SC_ENDPT + sp_found + 4) AND 255) <> 108 THEN sp_ok = 0 ' l
1037         IF (PEEK(SC_ENDPT + sp_found + 5) AND 255) <> 101 THEN sp_ok = 0 ' e
1038         IF (PEEK(SC_ENDPT + sp_found + 6) AND 255) <> 61 THEN sp_ok = 0 ' =
1039     END IF
1040     IF sp_ok = 0 THEN RETURN
1041
1042     ' Copy the id: up to 8 bytes, stopping at NUL or '&'.
1043     sp_j = sp_found + 7
1044     sp_k = 0
1045     WHILE sp_k < 8 AND sp_j < fn_len
1046         sp_c = PEEK(SC_ENDPT + sp_j) AND 255
1047         IF sp_c = 0 OR sp_c = 38 THEN EXIT WHILE ' NUL or '&'
1048         POKE (SC_TABLE + sp_k), sp_c
1049         sp_k = sp_k + 1
1050         sp_j = sp_j + 1
1051     WEND
1052     POKE (SC_TABLE + sp_k), 0
1053
1054     ' Validate: A-Z/a-z/0-9 only.
1055     sp_valid = 1
1056     FOR sp_m = 0 TO sp_k - 1
1057         sp_c = PEEK(SC_TABLE + sp_m) AND 255
1058         IF (sp_c < 65 OR sp_c > 90) AND (sp_c < 97 OR sp_c > 122) AND (sp_c < 48
OR sp_c > 57) THEN sp_valid = 0
1059     NEXT sp_m
1060     IF sp_valid = 0 THEN POKE SC_TABLE, 0
1061 END
1062
1063 ' write_room_appkey: persist SC_ENDPT + "?table=" + SC_TABLE back to the

```

```

1064 ' lobby server appkey slot, so a reboot without going back through the
1065 ' lobby rejoins the same room (mirrors the C clients' "Update server app
1066 ' key in case of reboot"). Builds the payload directly in FN_TX and writes
1067 ' it from there -- appkey write's byte-by-byte copy from #fn_src into
1068 ' FN_TX is a safe no-op when #fn_src is FN_TX itself.
1069 write_room_appkey: PROCEDURE
1070     #fn_txlen = 0
1071     #fn_src = SC_ENDPT : ls_max = 65 : GOSUB fn_strlen : GOSUB fn_putstr
1072     #fn_src = VARPTR lit_qtable(0) : fn_len = 7 : GOSUB fn_putstr
1073     #fn_src = SC_TABLE : ls_max = 9 : GOSUB fn_strlen : GOSUB fn_putstr
1074
1075     ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
1076     ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 1
1077     GOSUB appkey_open
1078     IF fn_ok THEN
1079         fn_len = #fn_txlen : #fn_src = FN_TX
1080         GOSUB appkey_write
1081         GOSUB appkey_close
1082     END IF
1083 END
1084
1085 ' clear_room_appkey: blank the lobby server appkey slot on a deliberate
1086 ' leave, so a later reboot lands on table select instead of silently
1087 ' rejoining a table the player just quit.
1088 clear_room_appkey: PROCEDURE
1089     ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1
1090     ak_key = AK_LOBBY_KEY_SERVER : ak_mode = 1
1091     GOSUB appkey_open
1092     IF fn_ok THEN
1093         fn_len = 0 : #fn_src = FN_TX
1094         GOSUB appkey_write
1095         GOSUB appkey_close
1096     END IF
1097     POKE SC_TABLE, 0
1098 END
1099
1100 ' =====
1101 ' ingame_menu: keypad CLEAR overlay, drawn over the prompt row and the two
1102 ' rows above it. Disc up/down to choose, action button to confirm, Clear
1103 ' again cancels straight back to RESUME.
1104 ' =====
1105 ingame_menu: PROCEDURE
1106     im_sel = 0
1107     inp_lock = 0
1108     ' Wait for the CLEAR press that opened the menu to release before
1109     ' accepting any input. Without this, the same still-held key
1110     ' immediately satisfies "IF CONT1.KEY = 10 THEN GOTO im_done" below on
1111     ' the very first frame, closing the menu the instant it opens -- the
1112     ' manual's own advice ("it's suggested to wait for CONT1.KEY to
1113     ' contain 12 before waiting for a key") for exactly this reason.
1114     im_wait_release:
1115     WAIT
1116     IF CONT1.KEY <> 12 THEN GOTO im_wait_release

```

```

1117
1118 im_loop:
1119     PRINT AT STATUS_ROW - 40 COLOR COL_TEXT, "
1120     PRINT AT STATUS_ROW - 20 COLOR COL_TEXT, "
1121     PRINT AT STATUS_ROW COLOR COL_TEXT, "
1122     PRINT AT STATUS_ROW - 40 COLOR COL_TEXT, "TABLE MENU"
1123     #gs_c = COL_TEXT
1124     IF im_sel = 0 THEN #gs_c = COL_HILITE
1125     PRINT AT STATUS_ROW - 20 COLOR #gs_c, "RESUME"
1126     #gs_c = COL_TEXT
1127     IF im_sel = 1 THEN #gs_c = COL_HILITE
1128     PRINT AT STATUS_ROW COLOR #gs_c, "QUIT TABLE"
1129
1130     WAIT
1131     IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO im_loop
1132
1133     IF CONT1.UP OR CONT1.DOWN THEN
1134         im_sel = 1 - im_sel
1135         inp_lock = 10
1136         GOSUB sound_cursor
1137         GOTO im_loop
1138     END IF
1139     IF CONT1.KEY = 10 THEN GOTO im_done
1140     IF CONT1.BUTTON = 0 THEN GOTO im_loop
1141     GOSUB sound_select
1142
1143     IF im_sel = 1 THEN
1144         url_path = 5 : GOSUB compose_url
1145         #net_readlen = 8
1146         GOSUB api_call
1147         GOSUB clear_room_appkey
1148         want_leave = 1
1149     END IF
1150 im_done:
1151     prev_round = 255 ' safety net, in case some other path still relies on it
1152     ' Redraw the whole screen immediately from whatever's already cached
1153     ' in FN_RX -- not just next poll. ingame_menu is called both from
1154     ' gl_input (outer loop) and from turn_input's own blocking ti_loop;
1155     ' in the latter case, control doesn't return to game_loop's own
1156     ' CLS/redraw dispatch until the current turn ends, which could be a
1157     ' long time away, so the menu's overlay (rows 9-11) would otherwise
1158     ' just sit there behind whatever turn_input draws around it next.
1159     IF want_leave = 0 THEN
1160         GOSUB cls_blue
1161         IF #cur_round = ROUND_LOBBY THEN
1162             GOSUB render_lobby
1163         ELSEIF #cur_round = ROUND_GAMEOVER THEN
1164             GOSUB render_gameover
1165         ELSE
1166             GOSUB card_init
1167             GOSUB render_playscreen
1168         END IF
1169     END IF

```

```

1170 END
1171
1172 ' =====
1173 ' name_entry_screen: disc letter picker, max 8 chars. Disc up/down cycles
1174 ' the character under the cursor through A-Z, 0-9, space; left/right moves
1175 ' the cursor; the action button accepts (at least 1 non-space char).
1176 ' Adapted from fujinet-battleship/intv/battleship.bas, verbatim apart from
1177 ' colors.
1178 ' =====
1179 name_entry_screen: PROCEDURE
1180     GOSUB cls_blue
1181     PRINT AT 0 COLOR COL_TEXT, "ENTER YOUR NAME"
1182     FOR ne_i = 0 TO 7
1183         ne_buf(ne_i) = 36 ' space
1184     NEXT ne_i
1185     ne_cur = 0
1186     inp_lock = 0
1187
1188 ne_loop:
1189     FOR ne_i = 0 TO 7
1190         #gs_c = COL_TEXT
1191         IF ne_i = ne_cur THEN #gs_c = COL_HILITE
1192         ne_j = ne_buf(ne_i)
1193         IF ne_j < 26 THEN
1194             gs_j = 65 + ne_j
1195         ELSEIF ne_j < 36 THEN
1196             gs_j = 48 + ne_j - 26
1197         ELSE
1198             gs_j = 95 ' underscore stands in for a visible blank
1199         END IF
1200         #BACKTAB(60 + 6 + ne_i) = (gs_j - 32) * 8 + #gs_c
1201     NEXT ne_i
1202
1203     WAIT
1204     IF inp_lock > 0 THEN inp_lock = inp_lock - 1 : GOTO ne_loop
1205
1206     IF CONT1.RIGHT THEN
1207         ne_cur = ne_cur + 1
1208         IF ne_cur > 7 THEN ne_cur = 0
1209         inp_lock = 8
1210         GOSUB sound_cursor
1211         GOTO ne_loop
1212     END IF
1213     IF CONT1.LEFT THEN
1214         IF ne_cur = 0 THEN ne_cur = 8
1215         ne_cur = ne_cur - 1
1216         inp_lock = 8
1217         GOSUB sound_cursor
1218         GOTO ne_loop
1219     END IF
1220     IF CONT1.UP THEN
1221         ne_buf(ne_cur) = ne_buf(ne_cur) + 1
1222         IF ne_buf(ne_cur) > 36 THEN ne_buf(ne_cur) = 0

```

```

1223     inp_lock = 6
1224     GOSUB sound_cursor
1225     GOTO ne_loop
1226 END IF
1227 IF CONT1.DOWN THEN
1228     IF ne_buf(ne_cur) = 0 THEN ne_buf(ne_cur) = 37
1229     ne_buf(ne_cur) = ne_buf(ne_cur) - 1
1230     inp_lock = 6
1231     GOSUB sound_cursor
1232     GOTO ne_loop
1233 END IF
1234 IF CONT1.BUTTON = 0 THEN GOTO ne_loop
1235 GOSUB sound_select
1236
1237 ne_len = 8
1238 WHILE ne_len > 0 AND ne_buf(ne_len - 1) = 36
1239     ne_len = ne_len - 1
1240 WEND
1241 IF ne_len = 0 THEN GOTO ne_loop
1242
1243 FOR ne_i = 0 TO ne_len - 1
1244     ne_j = ne_buf(ne_i)
1245     IF ne_j < 26 THEN
1246         gs_j = 65 + ne_j
1247     ELSE
1248         gs_j = 48 + ne_j - 26
1249     END IF
1250     POKE (SC_NAME + ne_i), gs_j
1251 NEXT ne_i
1252 POKE (SC_NAME + ne_len), 0
1253
1254 ak_creator_lo = 1 : ak_creator_hi = 0 : ak_app = 1 : ak_key = 0 : ak_mode =
1
1255 GOSUB appkey_open
1256 IF fn_ok THEN
1257     #fn_src = SC_NAME : fn_len = ne_len
1258     GOSUB appkey_write
1259     IF fn_ok = 0 THEN
1260         PRINT AT 100 COLOR COL_TEXT, "NAME NOT SAVED FOR "
1261         PRINT AT 120 COLOR COL_TEXT, "NEXT TIME "
1262         inp_lock = 90
1263         WHILE inp_lock > 0
1264             inp_lock = inp_lock - 1
1265             WAIT
1266         WEND
1267     END IF
1268     GOSUB appkey_close
1269 END IF
1270 END
1271 halt:
1272 WAIT
1273
1274     GOTO halt
1275

```


LISTING 11 state.bas — Fujitzee wire format

```

1 ' state.bas -- binary GameState/Tables wire format, read in place out of FN_RX.
2
3 ' Per the pattern established by fujinet-5cardstud/intv/state.bas and
4 ' fujinet-battleship/intv/state.bas: never copy the reply into IntyBASIC
5 ' variables -- these are just fixed offsets (matching server/util.go's
6 ' serializeResults binary layout, requested with &bin=1) plus small DEF FN
7 ' accessors that PEEK straight out of FN_RX.
8
9 ' GameState layout, always this shape regardless of round (util.go:76-108):
10 ' 0 playerCount (1)
11 ' 1 serverName[21]          lowercased, NUL-padded
12 ' 22 prompt[41]            lowercased, NUL-padded
13 ' 63 round (1)             0=lobby, 1..13=play, 99=gameover
14 ' 64 rollsLeft (1)
15 ' 65 activePlayer (signed, $FF = -1; 0 = your turn)
16 ' 66 moveTime (1, seconds, <=255 by design)
17 ' 67 viewing (1)           1 = you are a spectator
18 ' 68 dice[6]              5 ascii digits '1'..'6' + NUL; empty at turn start
19 ' 74 keepRoll[6]          5 ascii '0'/'1' + NUL (1 = will re-roll)
20 ' 80 validScores[15]      signed bytes; $FF = -1 = not selectable.
21 '                               Only meaningful when round is 1..13; the
22 '                               server sends all zero bytes here in the
23 '                               lobby (nil slice, not -1 -- see util.go's
24 '                               ValidScores loop), so never read this
25 '                               outside GAME_MINLEN's play branch.
26 ' 95 players[N] -- N = playerCount (can exceed 6; spectators are
27 '   appended beyond the 6 seated players), 42 bytes each:
28 '   name[9] alias(1) scores[16] (u16 LE each; $FFFF = -1 = unset)
29 '   scores[] is meaningful only at index 0 (1=ready, 0=not, -2=spectator)
30 '   while round=0; all 16 are real once round>=1.
31
32 /tables -> Tables struct:
33 ' 0 count (1)
34 ' then N x 36 bytes: table[9] name[21] players[6] (literal "cur / max")
35
36 CONST GAME_PLAYERCOUNT = 0
37 CONST GAME_SERVERNAME = 1 ' 21 bytes
38 CONST GAME_PROMPT = 22 ' 41 bytes
39 CONST GAME_ROUND = 63
40 CONST GAME_ROLLSLEFT = 64
41 CONST GAME_ACTIVEPLAYER = 65
42 CONST GAME_MOVETIME = 66
43 CONST GAME_VIEWING = 67
44 CONST GAME_DICE = 68 ' 6 bytes
45 CONST GAME_KEEPROLL = 74 ' 6 bytes
46 CONST GAME_VALIDSCORES = 80 ' 15 bytes
47 CONST GAME_PLAYERS = 95
48
49 ' Scratch RAM, ours, below fujinet.bas's own SC_* buffers (which stop at
50 ' SC_QUERY $9150 + 48 = $9180) and well below the $9C00 mailbox. Declared
51 ' here (rather than in fujitzee.bas) so dice.bas/card.bas -- included right
52 ' after this file, before fujitzee.bas's own CONST section runs -- can
53 ' reference SC_KEEPP without a forward reference.
54 CONST SC_TABLE = $9180 ' selected table id, 9 bytes (8 + NUL)
55 CONST SC_KEEPP = $9190 ' 5-char reroll mask, ascii '0'/'1', no NUL
56 CONST SC_PREVSCORES = $91A0 ' shadow of the active player's scores[16] (u16
57 ' LE), 32 bytes
58
59 ' Cross-file client display globals. IntyBASIC auto-registers a bare
60 ' identifier the first time it's mentioned at all (read or write), not
61 ' just on an explicit DIM -- card.bas (included right after this file)
62 ' reads/writes view_idx and draw_field's df_* parameters before
63 ' fujitzee.bas's own DIM section would otherwise declare them, which
64 ' turns that later DIM into a "variable already defined" error. Declaring
65 ' them here, ahead of every include that touches them, avoids that.
66 DIM view_idx
67 DIM df_pos, df_len, #df_color, #df_src
68
69 CONST PLAYER_STRIDE = 42
70 CONST PL_NAME = 0 ' 9 bytes
71 CONST PL_ALIAS = 9 ' 1 byte
72 CONST PL_SCORES = 10 ' 16 x uint16 LE
73
74 CONST TABLE_STRIDE = 36
75 CONST TBL_ID = 0 ' 9 bytes
76 CONST TBL_NAME = 9 ' 21 bytes
77 CONST TBL_PLAYERS = 30 ' 6 bytes, literal "cur / max"
78
79 ' PLAYER_MAX matches the reference C client's misc.h (spectators can
80 ' push playerCount past the 6 seated-player cap); GAME_MAXLEN sizes the
81 ' one poll buffer big enough for the worst case, 95 + 12*42 = 599 bytes,
82 ' well inside FN_RX's 704-byte window ($9D40-$9FFF).
83 CONST PLAYER_MAX = 12
84 CONST GAME_MAXLEN = GAME_PLAYERS + PLAYER_MAX * PLAYER_STRIDE
85 CONST GAME_MINLEN = GAME_PLAYERS
86 CONST TABLES_MAXLEN = 361
87
88 ' Round values.
89 CONST ROUND_LOBBY = 0
90 CONST ROUND_FINAL = 13
91 CONST ROUND_GAMEOVER = 99
92
93 ' Scorecard indices (gameLogic.go SCORE_* constants).
94 CONST SCORE_ONES = 0
95 CONST SCORE_UPPER_TOTAL = 6
96 CONST SCORE_UPPER_BONUS = 7
97 CONST SCORE_SET3 = 8
98 CONST SCORE_SET4 = 9
99 CONST SCORE_FULLHOUSE = 10

```

```

99  CONST SCORE_SRUN      = 11
100  CONST SCORE_LRUN     = 12
101  CONST SCORE_CHANCE   = 13
102  CONST SCORE_FUJITZEE = 14
103  CONST SCORE_TOTAL    = 15
104
105  ' Lobby-only meaning of scores[0].
106  CONST SCORE_VIEWING = 65534 ' wire $FFFF -> u16 65534 (== -2 signed)
107  CONST SCORE_UNSET   = 65535 ' wire $FFFF -> -1 signed
108  CONST SCORE_READY   = 1
109  CONST SCORE_UNREADY = 0
110
111  ' player_addr(i): address of player i's 42-byte record in FN_RX.
112  DEF FN player_addr(i) = FN_RX + GAME_PLAYERS + i * PLAYER_STRIDE
113
114  ' table_addr(i): address of table i's 36-byte record in FN_RX (i is 0-based,
115  ' the record right after the leading count byte).
116  DEF FN table_addr(i) = FN_RX + 1 + i * TABLE_STRIDE
117
118  ' score_at(addr, i): player record at 'addr's scores[i] as an unsigned
119  ' 16-bit word (65535 = unset/-1, 65534 = viewing/-2 in the lobby slot).
120  DEF FN score_at(addr, i) = (PEEK(addr + PL_SCORES + i * 2 + 1) AND 255) *
256 + (PEEK(addr + PL_SCORES + i * 2) AND 255)
121
122  ' valid_at(i): validScores[i] as an unsigned byte (255 = -1 = not selectable).
123  ' Parenthesized "(... AND 255)" deliberately -- see state_round's comment
124  ' below for why an unparenthesized trailing "AND 255" is dangerous.
125  DEF FN valid_at(i) = (PEEK(FN_RX + GAME_VALIDSCORES + i) AND 255)
126
127  ' active_player: activePlayer as a signed value (-1..11), converting the
128  ' wire's $FF sentinel. Called as plain 'active_player' (no parens -- DEF FN
129  ' with no arguments is defined and invoked without them).
130  DEF FN active_player = ((PEEK(FN_RX + GAME_ACTIVEPLAYER) AND 255) = 255) *
-256 + (PEEK(FN_RX + GAME_ACTIVEPLAYER) AND 255)
131
132  ' state_round / state_playercount / state_rollsleft / state_viewing:
133  ' unsigned byte reads, broken out as DEF FNs purely so call sites read as
134  ' intent, not offsets.
135  '
136  ' Parenthesized "(... AND 255)" deliberately, not left bare. IntyBASIC's
137  ' "=" binds *tighter* than "AND" (same trap as C's "a & b == c"), so an
138  ' unparenthesized "PEEK(x) AND 255" substituted into "state_viewing = 0"
139  ' compiles as "PEEK(x) AND (255 = 0)" -- the 255 gets grouped with the
140  ' comparison instead of the AND, folding to "PEEK(x) AND 0", which is
141  ' always zero regardless of what was actually peeked. Confirmed by reading
142  ' the generated .lst for "IF active_player = 0 AND state_viewing = 0
143  ' THEN": active_player's own formula (already parenthesized below) came
144  ' out correct, but state_viewing's unparenthesized body made the whole
145  ' condition permanently false, so turn_input was never reached at all.
146  DEF FN state_round      = (PEEK(FN_RX + GAME_ROUND) AND 255)
147  DEF FN state_playercount = (PEEK(FN_RX + GAME_PLAYERCOUNT) AND 255)
148  DEF FN state_rollsleft  = (PEEK(FN_RX + GAME_ROLLSLEFT) AND 255)
149  DEF FN state_viewing    = (PEEK(FN_RX + GAME_VIEWING) AND 255)

```

```

150
151  '-----
152  ' validate_state: layered sanity check on a /state, /ready, /roll, or /score
153  ' reply already sitting in FN_RX (#net_gotlen bytes). FN_RX is never cleared
154  ' between calls, so a short read leaves state bytes from a previous, larger
155  ' response -- fixed-offset reads would happily render that as garbage. Sets
156  ' fn_ok = 0 (in addition to whatever net/transact left it) if any gate
157  ' fails; callers must check fn_ok after calling this, not just after
158  ' api_call.
159  '-----
160  DIM vs_round, vs_pc, #vs_expect
161  validate_state: PROCEDURE
162  IF fn_ok = 0 OR #net_gotlen < GAME_MINLEN THEN
163      fn_ok = 0
164      RETURN
165  END IF
166
167  vs_pc = state_playercount
168  IF vs_pc > PLAYER_MAX THEN
169      fn_ok = 0
170      RETURN
171  END IF
172
173  vs_round = state_round
174  IF vs_round <> ROUND_GAMEOVER AND vs_round > ROUND_FINAL THEN
175      fn_ok = 0
176      RETURN
177  END IF
178
179  #vs_expect = GAME_PLAYERS + vs_pc * PLAYER_STRIDE
180  IF #net_gotlen < #vs_expect THEN fn_ok = 0
181  END
182

```

LISTING 12 dice.bas — Fujitzee dice

```

1 ' dice.bas -- die-face GRAM bitmaps and draw routine, MODE 1 (FG/BG). Same
2 ' technique as Fujinet-5cardstud/intv/gfx.bas's cards: each face is one 8x8
3 ' GRAM card, loaded once at boot, drawn by poking a BACKTAB cell to
4 ' (index + 256) * 8 + color.
5 '
6 ' Six faces (pip counts 1-6, GRAM index 0-5), a standard 3x3 pip grid on an
7 ' outlined 8x8 box. A die marked to re-roll isn't a separate GRAM tile --
8 ' it's the same face drawn with the cursor/held background color, exactly
9 ' like card.bas's scorecard cursor and board.bas's cell coloring. That
10 ' keeps the GRAM budget at 6 of 64 cards, leaving the rest free for a
11 ' future sprite-based roll animation (neither Scardstud nor Battleship use
12 ' any MOBs at all).
13 '
14 '-----
15 ' dice_init: load the 6 die faces into GRAM slots 0-5 (screen codes 256-261).
16 ' Must run once at startup. GRAM loads take effect on the next video frame,
17 ' so DEFINE is followed by WAIT -- a second DEFINE in the same frame would
18 ' silently overwrite the first.
19 '-----
20 dice_init: PROCEDURE
21     DEFINE 0, 6, diceart : WAIT
22 END
23 '
24 '-----
25 ' print die: draw one die face at BACKTAB cell dp_pos. Inputs: dp_pos (cell
26 ' offset), dp_val (1-6; anything outside that range draws a blank cell so an
27 ' empty dice string at turn start doesn't render garbage), #dp_color.
28 '-----
29 DIM dp_pos, dp_val, #dp_color
30 print die: PROCEDURE
31     IF dp_val >= 1 AND dp_val <= 6 THEN
32         #BACKTAB(dp_pos) = #dp_color + (dp_val - 1 + 256) * 8
33     ELSE
34         #BACKTAB(dp_pos) = #dp_color + 32 * 8 ' blank (space glyph)
35     END IF
36 END
37 '
38 '-----
39 ' draw_dice_row: a ROLL tile at BACKTAB row 10 col 0, then the 5 dice at
40 ' cols 2,4,6,8,10, plus a small rolls-left readout. Reads the live wire
41 ' dice string (FN_RX+GAME_DICE) and the local keep mask (SC_KEEP) directly
42 ' rather than taking them as parameters -- both are always current by the
43 ' time this is called (either just-pollled, or unrolled state.bas globals
44 ' set up before a redraw).
45 '
46 ' SC_KEEP holds the wire's roll-mask convention ('1'=49 will reroll,
47 ' '0'=48 will be kept -- see fujitzee.bas's compose_url, which sends it
48 ' straight through to /roll/<mask>), but the *default* per die, before the
49 ' player touches anything, is '1' (reroll) -- pressing the button marks a
50 ' die to be KEPT, matching how every other Fujitzee client's dice
51 ' selection reads (you pick what you're holding onto, not what you're
52 ' throwing away). So the highlight here is for the KEEP state (mask='0'),
53 ' not the reroll state.
54 '
55 ' Inputs: dr_mode (0=DICE,1=SCORE), dr_cursor (0-5: 0=the ROLL tile, 1-5=
56 ' dice 0-4, matching the reference clients' own cursorPos convention --
57 ' only drawn highlighted when dr_mode=0).
58 '-----
59     CONST DICE_ROW = 10
60     CONST ROLL_COL = 0
61     CONST DICE_COL0 = 2
62     CONST COL_DICE_NORMAL = FG_BLACK + BG_WHITE
63     CONST COL_DICE_KEPT = FG_BLACK + BG_YELLOW
64     CONST COL_DICE_CURSOR = FG_BLACK + BG_GREEN
65
66 DIM dr_mode, dr_cursor, dr_i, #dr_ch, dr_kept, #dr_rolls
67 draw_dice_row: PROCEDURE
68     #dp_color = COL_DICE_NORMAL
69     IF dr_mode = 0 AND dr_cursor = 0 THEN #dp_color = COL_DICE_CURSOR
70     PRINT AT screenpos(ROLL_COL, DICE_ROW) COLOR #dp_color, "R"
71
72     FOR dr_i = 0 TO 4
73         #dr_ch = (PEEK(FN_RX + GAME_DICE + dr_i) AND 255)
74         IF #dr_ch >= 49 AND #dr_ch <= 54 THEN dp_val = #dr_ch - 48 ELSE dp_val =
75
76         dr_kept = (PEEK(SC_KEEP + dr_i) AND 255) = 48
77         #dp_color = COL_DICE_NORMAL
78         IF dr_kept THEN #dp_color = COL_DICE_KEPT
79         IF dr_mode = 0 AND dr_cursor = dr_i + 1 THEN #dp_color = COL_DICE_CURSOR
80
81         dp_pos = screenpos(DICE_COL0 + dr_i * 2, DICE_ROW)
82         GOSUB print_die
83     NEXT dr_i
84
85 ' Blank cols 11-19 first, not just the 2 cells the count actually
86 ' needs -- "x" used to sit at col 17 here as a separator, and being
87 ' lowercase it was outside MODE 1's GROM range (ASCII 32-95 only,
88 ' uppercase/digits/symbols), rendering as a garbage glyph. Blanking
89 ' the whole gap defensively means any future stray write here (or
90 ' leftover from a still-undiagnosed one) can't leave visible debris
91 ' either, rather than only ever touching the exact 2 cells this
92 ' PRINT needs.
93 FOR dr_i = 11 TO 19
94     #BACKTAB(screenpos(dr_i, DICE_ROW)) = COL_DICE_NORMAL
95 NEXT dr_i
96 #dr_rolls = state_rollsleft
97 PRINT AT screenpos(18, DICE_ROW) COLOR COL_DICE_NORMAL, <.2>#dr_rolls
98 END
99

```

```

100 '-----
101 animate_roll: dice-rolling flourish shown right after any player's roll
102 lands (yours or another seat's -- see game loop's roll_changed check,
103 which fires this for anyone). FN_RX already holds the settled result at
104 this point (the server computes the whole roll in one shot), so this
105 doesn't determine the outcome -- it flickers whichever dice the *wire's*
106 keepRoll field says were rerolled (GAME_KEEPROLL; the server echoes
107 back whatever mask was actually submitted, by whoever just rolled, not
108 our own local SC_KEEP -- that's what makes this work for other players'
109 turns too) through random faces a few times, with a tick each flicker,
110 before the normal render draws the real values. Kept dice are drawn at
111 their real (unchanging) value throughout, echoing the reference C
112 clients' rollFrames/ROLL_SOUND_MOD animation (gamelogic.c). Wire
113 convention is unaffected by the local keep/reroll UI relabeling above:
114 keepRoll's '1' still means "this die was rerolled."
115 '-----
116 CONST ROLL_ANIM_FRAMES = 24
117 CONST ROLL_ANIM_STEP = 3
118
119 DIM ra_frame, ra_i, #ra_ch
120 animate_roll: PROCEDURE
121   FOR ra_frame = 1 TO ROLL_ANIM_FRAMES
122     IF ra_frame % ROLL_ANIM_STEP = 1 THEN
123       FOR ra_i = 0 TO 4
124         IF (PEEK(FN_RX + GAME_KEEPROLL + ra_i) AND 255) = 49 THEN
125           dp_val = RAND(6) + 1
126         ELSE
127           #ra_ch = (PEEK(FN_RX + GAME_DICE + ra_i) AND 255)
128           IF #ra_ch >= 49 AND #ra_ch <= 54 THEN dp_val = #ra_ch - 48
129         END IF
130         dp_pos = screenpos(DICE_COL0 + ra_i * 2, DICE_ROW) : #dp_color =
131           COL_DICE_NORMAL
132         GOSUB print_die
133       NEXT ra_i
134       GOSUB sound_tick
135     END IF
136     WAIT
137   NEXT ra_frame
138 END
139
140 ' Pip grid: border at rows/cols 0 and 7; pips at rows/cols {2,4,6} of an
141 ' 8x8 cell. Row4 col4 is the single center pip (face 1); rows/cols 2 and 6
142 ' are the four corners (faces 4-6); row4 cols 2/6 are the middle side pips
143 ' (face 6 only).
144 diceart:
145   ' Face 1 (index 0): center pip.
146   BITMAP "00000000"
147   BITMAP "0.....0"
148   BITMAP "0.....0"
149   BITMAP "0.....0"
150   BITMAP "0.....0"
151
152   BITMAP "0.....0"
153   BITMAP "00000000"
154
155   ' Face 2 (index 1): top-left, bottom-right.
156   BITMAP "00000000"
157   BITMAP "0.....0"
158   BITMAP "0.....0"
159   BITMAP "0.....0"
160   BITMAP "0.....0"
161   BITMAP "0.....0"
162   BITMAP "0.....00"
163   BITMAP "00000000"
164
165   ' Face 3 (index 2): top-left, center, bottom-right.
166   BITMAP "00000000"
167   BITMAP "0.....0"
168   BITMAP "0.....0"
169   BITMAP "0.....0"
170   BITMAP "0.....0"
171   BITMAP "0.....00"
172   BITMAP "00000000"
173
174   ' Face 4 (index 3): four corners.
175   BITMAP "00000000"
176   BITMAP "0.....0"
177   BITMAP "0.....00"
178   BITMAP "0.....0"
179   BITMAP "0.....0"
180   BITMAP "0.....0"
181   BITMAP "0.....00"
182   BITMAP "00000000"
183
184   ' Face 5 (index 4): four corners + center.
185   BITMAP "00000000"
186   BITMAP "0.....0"
187   BITMAP "0.....00"
188   BITMAP "0.....0"
189   BITMAP "0.....0"
190   BITMAP "0.....0"
191   BITMAP "0.....00"
192   BITMAP "00000000"
193
194   ' Face 6 (index 5): four corners + two middle side pips.
195   BITMAP "00000000"
196   BITMAP "0.....0"
197   BITMAP "0.....00"
198   BITMAP "0.....0"
199   BITMAP "0.....00"
200   BITMAP "0.....0"
201   BITMAP "0.....00"
202   BITMAP "00000000"
203

```

LISTING 13 card.bas — Fujitzee scorecard

```

1 ' card.bas -- scorecard grid rendering, cursor, seat strip and standings
2 ' overlay for the round 1..13 game screen. One player's full 15-category
3 ' card is shown at a time (BACKTAB rows 1-8); a permanent 6-seat strip
4 ' (row 9) shows who's who, and holding keypad ENTER swaps rows 1-8 for a
5 ' name+total standings list.
6
7 ' Layout (20x12 BACKTAB):
8 ' row0 viewed player's name, round
9 ' row1-6 ONE..SIX | SET3,SET4,HOUS,SRUN,LRUN,CNT,FUJI (rows1-7)
10 ' row7 UP (computed) | FUJI (row7 of the right column)
11 ' row8 BON (computed) | TOT (computed)
12 ' row9 6-seat strip (alias letters, color-coded)
13 ' row10 5 dice + rolls-left
14 ' row11 server prompt + move timer
15
16 ' Category placement:
17 ' left column, rows 1-6: score indices 0-5 (ONE..SIX)
18 ' left column, row 7: UP (computed upper total, index 6 on the wire)
19 ' left column, row 8: BON (computed bonus, index 7 on the wire)
20 ' right column, rows 1-7: score indices 8-14 (SET3,SET4,HOUS,SRUN,LRUN,CNT,
FUJI)
21 ' right column, row 8: TOT (computed running grand total)
22
23 CONST CARD_ROW0 = screenpos(0, 0)
24 CONST CARD_LCOL_LABEL = 0
25 CONST CARD_LCOL_VALUE = 5
26 CONST CARD_RCOL_LABEL = 10
27 CONST CARD_RCOL_VALUE = 15
28 CONST SEAT_ROW = 9
29
30 ' Page background is blue (cls blue in fujitzee.bas fills every CLS'd
31 ' screen with this), so every color below pairs with BG.BLUE instead
32 ' of BG.BLACK -- anything still paired with black would sit on an
33 ' invisible matching square rather than actually showing a black box.
34 CONST COL_TEXT = FG_WHITE + BG_BLUE
35 CONST COL_LABEL = FG_TAN + BG_BLUE
36 CONST COL_HILITE = FG_YELLOW + BG_BLUE
37 CONST COL_POTENTIAL = FG_GREEN + BG_BLUE
38 CONST COL_DASH = FG_BLACK + BG_BLUE
39 CONST COL_CURSOR = FG_BLACK + BG_YELLOW
40 CONST COL_ACTIVE = FG_BLACK + BG_YELLOW
41 CONST COL_VIEWED = FG_BLACK + BG_GREEN
42 CONST COL_SEATEMPTY = FG_BLACK + BG_BLUE
43 ' MODE 1's foreground palette is fixed at 8 colors (no orange) --
44 ' orange only exists as one of the 16 background shades. So "orange
45 ' text" for the round indicator is rendered as black-on-orange
46 ' instead of a foreground color.
47 CONST COL_ROUND = FG_BLACK + BG_ORANGE
48
49 -----
50 ' player_color_tbl / player_color: a distinct FG color per seat (0-5),
51 ' used everywhere a player's name is drawn (header, seat strip, standings,
52 ' lobby, score reveal) so a given player reads consistently across every
53 ' screen. FG_BLUE is deliberately excluded -- it would be invisible on
54 ' the page's own blue background. Only 6 colors are available without
55 ' reusing COL_TEXT/COL_LABEL/the status colors above, matching
56 ' MAX_PLAYERS; seats beyond that (spectators, in the standings/lobby
57 ' lists which show up to 8) fall back to plain COL_TEXT.
58 ' -----
59 player_color_tbl: DATA FG_RED, FG_TAN, FG_DARKGREEN, FG_GREEN, FG_YELLOW,
FG_WHITE
60
61 DIM pc_idx, #pc_color
62 player_color: PROCEDURE
63 IF pc_idx < 6 THEN
64 #pc_color = player_color_tbl(pc_idx) + BG_BLUE
65 ELSE
66 #pc_color = COL_TEXT
67 END IF
68 END
69
70 -----
71 ' card_init: draw the static label frame once (labels never change; only
72 ' the values in the adjoining 3-char fields do). Callers CLS and call this
73 ' on the transition into round 1..13; the value cells drawn each poll after
74 ' this leave the labels alone.
75 ' -----
76 card_init: PROCEDURE
77 PRINT AT CARD_ROW0 + screenpos(0, 1) COLOR COL_LABEL, "ONE "
78 PRINT AT CARD_ROW0 + screenpos(0, 2) COLOR COL_LABEL, "TWO "
79 PRINT AT CARD_ROW0 + screenpos(0, 3) COLOR COL_LABEL, "THR "
80 PRINT AT CARD_ROW0 + screenpos(0, 4) COLOR COL_LABEL, "FOU "
81 PRINT AT CARD_ROW0 + screenpos(0, 5) COLOR COL_LABEL, "FIV "
82 PRINT AT CARD_ROW0 + screenpos(0, 6) COLOR COL_LABEL, "SIX "
83 PRINT AT CARD_ROW0 + screenpos(0, 7) COLOR COL_LABEL, "UP "
84 PRINT AT CARD_ROW0 + screenpos(0, 8) COLOR COL_LABEL, "BON "
85
86 PRINT AT CARD_ROW0 + screenpos(10, 1) COLOR COL_LABEL, "SET3"
87 PRINT AT CARD_ROW0 + screenpos(10, 2) COLOR COL_LABEL, "SET4"
88 PRINT AT CARD_ROW0 + screenpos(10, 3) COLOR COL_LABEL, "HOUS"
89 PRINT AT CARD_ROW0 + screenpos(10, 4) COLOR COL_LABEL, "SRUN"
90 PRINT AT CARD_ROW0 + screenpos(10, 5) COLOR COL_LABEL, "LRUN"
91 PRINT AT CARD_ROW0 + screenpos(10, 6) COLOR COL_LABEL, "CNT "
92 PRINT AT CARD_ROW0 + screenpos(10, 7) COLOR COL_LABEL, "FUJI"
93 PRINT AT CARD_ROW0 + screenpos(10, 8) COLOR COL_LABEL, "TOT "
94 END
95
96 -----
97 ' calc_running_total: sum every filled category (UP+BON+SET3..FUJI, unset
98 ' treated as 0) for the player record at #ctr_addr into #ctr_total. The

```

```

99 ' server only fills the wire's grand-total slot (score index 15) at game
100 ' end, so the live TOT row and the standings overlay compute it themselves.
101 -----
102 DIM #ctr_addr, #ctr_total, ctr_i, #ctr_v
103 calc_running_total: PROCEDURE
104   #ctr_total = 0
105   FOR ctr_i = 6 TO 14
106     #ctr_v = score_at(#ctr_addr, ctr_i)
107     IF #ctr_v <> SCORE_UNSET AND #ctr_v <> SCORE_VIEWING THEN #ctr_total =
#ctr_total + #ctr_v
108   NEXT ctr_i
109 END
110 -----
111 ' draw_cat_cell: render one 3-char value field. Inputs:
112 ' dcc_row BACKTAB row (1-8)
113 ' dcc_valcol BACKTAB column of the value field's first cell
114 ' dcc_cat score index 0-5 or 8-14 for a real category; 100=UP,
115 ' 101=BON, 102=TOT for the three computed cells
116 ' #dcc_addr viewed player's 42-byte record in FN_RX
117 ' dcc_own 1 if this is your own card and it's your turn (enables the
118 ' green "potential score" hint on unscored categories)
119 ' dcc_cursor 1 if this cell is the SCORE-mode cursor's current selection
120 -----
121 DIM dcc_row, dcc_valcol, dcc_cat, #dcc_addr, dcc_own, dcc_cursor
122 DIM #dcc_v, #dcc_color, #dcc_pot
123 draw_cat_cell: PROCEDURE
124   #dcc_color = COL_TEXT
125   IF dcc_cursor THEN #dcc_color = COL_CURSOR
126
127   IF dcc_cat = 100 OR dcc_cat = 101 THEN
128     #dcc_v = score_at(#dcc_addr, 6 + (dcc_cat - 100))
129     IF #dcc_v = SCORE_UNSET OR #dcc_v = SCORE_VIEWING THEN
130       IF dcc_cursor = 0 THEN #dcc_color = COL_DASH
131       PRINT AT screenpos(dcc_valcol, dcc_row) COLOR #dcc_color, " -"
132     ELSE
133       PRINT AT screenpos(dcc_valcol, dcc_row) COLOR #dcc_color, <.3>#dcc_v
134     END IF
135   ELSEIF dcc_cat = 102 THEN
136     #ctr_addr = #dcc_addr
137     GOSUB calc_running_total
138     PRINT AT screenpos(dcc_valcol, dcc_row) COLOR #dcc_color, <.3>#ctr_total
139   ELSE
140     #dcc_v = score_at(#dcc_addr, dcc_cat)
141     IF #dcc_v = SCORE_UNSET OR #dcc_v = SCORE_VIEWING THEN
142       #dcc_pot = 255
143       IF dcc_own THEN #dcc_pot = valid_at(dcc_cat)
144       IF #dcc_pot <> 255 THEN
145         ' A potential of exactly 0 (category open but this roll
146         ' doesn't fill it) is only shown on the row the cursor is
147         ' currently sitting on -- matching the reference clients
148         ' (gamelogic.c's drawTextAlt/drawBlank cursor-move
149         ' handling), which hides the "0" everywhere else so an
150
151         ' open board isn't a wall of zeros before you've decided.
152         IF #dcc_pot > 0 THEN
153           IF dcc_cursor = 0 THEN #dcc_color = COL_POTENTIAL
154           PRINT AT screenpos(dcc_valcol, dcc_row) COLOR #dcc_color,
<.3>#dcc_pot
155         ELSEIF dcc_cursor THEN
156           PRINT AT screenpos(dcc_valcol, dcc_row) COLOR #dcc_color, "
0"
157         ELSE
158           PRINT AT screenpos(dcc_valcol, dcc_row) COLOR COL_TEXT, "
"
159         END IF
160       ELSE
161         IF dcc_cursor = 0 THEN #dcc_color = COL_DASH
162         PRINT AT screenpos(dcc_valcol, dcc_row) COLOR #dcc_color, " -"
163       END IF
164     ELSE
165       PRINT AT screenpos(dcc_valcol, dcc_row) COLOR #dcc_color, <.3>#dcc_v
166     END IF
167   END IF
168 END
169 -----
170 ' render_card: redraw every value cell of the viewed player's card.
171 ' Globals expected: #rc_addr (player record), rc_own (1 if own card + your
172 ' turn, enables potentials), rc_mode (0=DICE,1=SCORE), rc_selcat (score
173 ' index the SCORE-mode cursor sits on; only consulted when rc_mode=1),
174 ' rc_reveal (score index 0-14 to force-highlight regardless of rc_own/
175 ' rc_mode -- used to flash another player's just-scored category; 255 for
176 ' none, the normal case).
177 ' Split into a left-column half and a right-column half with a WAIT between
178 ' (and another before the header/seat strip) so a full repaint -- 16 value
179 ' cells worth of PRINT -- can't tear a frame the way one unbroken burst
180 ' covering half the visible screen could (see board.bas/5card.bas's own
181 ' per-region WAIT breaks for the same reason: no page-flip on the STIC).
182 -----
183 DIM #rc_addr, rc_own, rc_mode, rc_selcat, rc_reveal, rc_i, rc_cur
184 render_card: PROCEDURE
185   FOR rc_i = 0 TO 7
186     rc_cur = 0
187     IF rc_mode = 1 AND rc_own AND rc_i < 6 AND rc_selcat = rc_i THEN rc_cur
= 1
188
189     IF rc_reveal = rc_i THEN rc_cur = 1
190     dcc_row = rc_i + 1 : dcc_valcol = CARD_LCOL_VALUE : #dcc_addr = #rc_addr
191     dcc_own = rc_own : dcc_cursor = rc_cur
192     IF rc_i < 6 THEN
193       dcc_cat = rc_i
194     ELSEIF rc_i = 6 THEN
195       dcc_cat = 100
196     ELSE
197       dcc_cat = 101
198     END IF
199     GOSUB draw_cat_cell

```

```

200 NEXT rc_i
201 WAIT
202
203 FOR rc_i = 0 TO 7
204   rc_cur = 0
205   IF rc_mode = 1 AND rc_own AND rc_i < 7 AND rc_selcat = rc_i + 8 THEN
rc_cur = 1
206     IF rc_reveal = rc_i + 8 THEN rc_cur = 1
207     dcc_row = rc_i + 1 : dcc_valcol = CARD_RCOL_VALUE : dcc_addr = #rc_addr
208     dcc_own = rc_own : dcc_cursor = rc_cur
209     IF rc_i < 7 THEN dcc_cat = rc_i + 8 ELSE dcc_cat = 102
210     GOSUB draw_cat_cell
211   NEXT rc_i
212 END
213
214 '-----
215 ' render_seatstrip: one letter per seated player (up to 6), spaced 3
216 ' columns apart, color-coded: active player highlighted, the currently
217 ' viewed player (if different) in a second color, everyone else plain
218 ' white, empty seats a dim dot.
219
220 ' This is the player's *first* name character (PL_NAME+0), not the
221 ' server's PL_ALIAS-indexed "unique highlight character" the field
222 ' technically exists for. The alias-indirection version (PEEK an alias
223 ' index byte, then PEEK name[that index]) was extensively verified
224 ' correct -- traced the exact compiled instructions, cross-checked real
225 ' alias/name bytes pulled live from both the dev and production servers
226 ' (every case seen had alias=0, i.e. this simpler version is what it
227 ' would have shown anyway) -- and still rendered '?' for every seat in
228 ' practice. Simplifying to a single direct PEEK removes the double
229 ' indirection entirely rather than continuing to chase a mismatch between
230 ' verified-correct analysis and observed behavior.
231 '-----
232 DIM ss_i, ss_addr, #ss_ch, #ss_color
233 render_seatstrip: PROCEDURE
234   FOR ss_i = 0 TO 5
235     IF ss_i < state_playercount THEN
236       ss_addr = player_addr(ss_i)
237       #ss_ch = (PEEK(ss_addr + PL_NAME) AND 255)
238       IF #ss_ch >= 97 AND #ss_ch <= 122 THEN #ss_ch = #ss_ch - 32
239       IF #ss_ch < 32 OR #ss_ch > 95 THEN #ss_ch = 63 ' '?' fallback
240       ' Active/viewed status wins the BG (black-on-yellow/green, as
241       ' before -- needs to stay readable and unambiguous regardless
242       ' of which player it is); otherwise the letter uses that
243       ' player's own color, so it reads consistently with their
244       ' name everywhere else even while just sitting in the strip.
245       IF ss_i = active_player THEN
246         #ss_color = COL_ACTIVE
247       ELSEIF ss_i = view_idx THEN
248         #ss_color = COL_VIEWED
249       ELSE
250         pc_idx = ss_i
251         GOSUB player_color
252         #ss_color = #pc_color
253       END IF
254     ELSE
255       #ss_ch = 46 ' '.'
256       #ss_color = COL_SEATEMPTY
257     END IF
258     #BACKTAB(screenpos(ss_i * 3, SEAT_ROW)) = (#ss_ch - 32) * 8 + #ss_color
259   NEXT ss_i
260 END
261
262 '-----
263 ' render_player_list: name + running total for up to 8 seated players,
264 ' starting at BACKTAB row prl_top. Shared by show_standings (the keypad-
265 ' ENTER overlay during play) and render_gameover (the round-99 screen) --
266 ' both just want "who's got how many points," and calc_running_total's sum
267 ' of every filled category equals the server's own final total once every
268 ' category is filled, so the same running-total math is correct in both
269 ' places without needing to trust the wire's game-over-only total field.
270 '-----
271 DIM prl_top, prl_i, prl_addr, #prl_pc
272 render_player_list: PROCEDURE
273   #prl_pc = state_playercount
274   IF #prl_pc > 8 THEN #prl_pc = 8
275   FOR prl_i = 0 TO #prl_pc - 1
276     prl_addr = player_addr(prl_i)
277     pc_idx = prl_i
278     GOSUB player_color
279     #df_src = prl_addr + PL_NAME : df_pos = screenpos(0, prl_top + prl_i) :
df_len = 9 : #df_color = #pc_color
280     GOSUB draw_field
281     #ctr_addr = prl_addr
282     GOSUB calc_running_total
283     PRINT AT screenpos(15, prl_top + prl_i) COLOR COL_TEXT, <.4>#ctr_total
284   NEXT prl_i
285 END
286
287 '-----
288 ' show_standings: keypad-ENTER overlay. Blanks rows 1-8 and lists seated
289 ' players' names and running totals instead of the card. Blocks on its own
290 ' input loop (like ingame_menu) rather than folding into the poll loop --
291 ' holding ENTER for a couple of frames of paused polling is harmless, and
292 ' this keeps render_card's cell layout untouched by an overlay that only
293 ' needs to exist while the button is held.
294 '-----
295 DIM sd_i
296 show_standings: PROCEDURE
297   FOR sd_i = 1 TO 8
298     PRINT AT screenpos(0, sd_i) COLOR COL_TEXT, "
299   NEXT sd_i
300   prl_top = 1
301   GOSUB render_player_list
302
303 sd_wait:

```

```

304 WAIT
305 IF CONT1.KEY = 11 THEN GOTO sd_wait
306 END
307

```

LISTING 14 sound.bas — Fujitzee effects

```

1 ' sound.bas -- sound effects, adapted from fujinet-battleship/intv/sound.bas.
2 ' The transport primitive (play_tone) and the UI-generic effects are kept
3 ' verbatim; the game-specific effects are replaced with Fujitzee's own set
4 ' (roll/hold/score/fujitzee in place of Battleship's place/attack/hit/sink).
5
6 ' All effects play on PSG channel 0 -- turn-based game, non-overlapping UI
7 ' events, no need for polyphony (and SOUND's channel argument must be a
8 ' compile-time constant anyway, which rules out a generic multi-channel
9 ' helper).
10
11 SOUND's frequency argument is a 12-bit PSG divisor, computed as
12 ' round(3579545/32/hz) for NTSC. These are precomputed rather than divided
13 ' at runtime: 3579545/32 alone is ~111861, which overflows IntyBASIC's
14 ' 16-bit variables (max 65535), so it only works as a compile-time-folded
15 ' literal division -- not as CONST-divided-by-a-runtime-variable. Every
16 ' effect here uses a fixed, known-in-advance frequency, so precomputing
17 ' sidesteps the overflow entirely.
18 50Hz->2237 60Hz->1864 70Hz->1598 80Hz->1398 100Hz->1119
19 150Hz->746 200Hz->559 300Hz->373 311Hz->360 330Hz->339
20 340Hz->329 350Hz->320 392Hz->285 415Hz->270 430Hz->260
21 500Hz->224 600Hz->187 700Hz->160 800Hz->140
22 CONST SND_VOL = 12
23
24 DIM #snd_val, snd_gate, snd_post, snd_i
25
26 -----
27 ' play_tone: one square-wave note on channel 0. #snd_val = precomputed PSG
28 ' divisor (see table above), snd_gate = frames the tone sounds, snd_post =
29 ' frames of silence after.
30 -----
31 play_tone: PROCEDURE
32 SOUND 0, #snd_val, SND_VOL
33 FOR snd_i = 1 TO snd_gate
34 WAIT
35 NEXT snd_i
36 SOUND 0, 0, 0
37 FOR snd_i = 1 TO snd_post
38 WAIT
39 NEXT snd_i
40 END
41
42 ' sound_join: sit down at a table (3-tone rising figure: 430,340,500 Hz).
43 sound_join: PROCEDURE
44 #snd_val = 260 : snd_gate = 5 : snd_post = 8 : GOSUB play_tone
45 #snd_val = 329 : snd_gate = 5 : snd_post = 0 : GOSUB play_tone
46 #snd_val = 224 : snd_gate = 5 : snd_post = 0 : GOSUB play_tone
47 END
48
49 ' sound_myturn: it's your turn to roll (double beep, 430,430 Hz).
50 sound_myturn: PROCEDURE
51 #snd_val = 260 : snd_gate = 4 : snd_post = 2 : GOSUB play_tone
52 #snd_val = 260 : snd_gate = 4 : snd_post = 2 : GOSUB play_tone
53 END
54
55 ' sound_gamedone: game over, win fanfare (4-tone rising: 311,330,392,415 Hz).
56 sound_gamedone: PROCEDURE
57 #snd_val = 360 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
58 #snd_val = 339 : snd_gate = 20 : snd_post = 0 : GOSUB play_tone
59 #snd_val = 285 : snd_gate = 10 : snd_post = 0 : GOSUB play_tone
60 #snd_val = 270 : snd_gate = 20 : snd_post = 0 : GOSUB play_tone
61 END
62
63 ' sound_player_join: another player sits down mid-lobby (5-tone rising
64 ' sweep: 50,60,70,80 Hz).
65 sound_player_join: PROCEDURE
66 #snd_val = 2237 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
67 #snd_val = 1864 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
68 #snd_val = 1598 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
69 #snd_val = 1398 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
70 END
71
72 ' sound_player_left: a player leaves mid-game (falling sweep, mirror of join).
73 sound_player_left: PROCEDURE
74 #snd_val = 1398 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
75 #snd_val = 1598 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
76 #snd_val = 1864 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
77 #snd_val = 2237 : snd_gate = 2 : snd_post = 15 : GOSUB play_tone
78 END
79
80 ' sound_select: a menu/table/ready choice was confirmed (2-tone rising
81 ' chirp: 300,350 Hz).
82 sound_select: PROCEDURE
83 #snd_val = 373 : snd_gate = 3 : snd_post = 1 : GOSUB play_tone
84 #snd_val = 320 : snd_gate = 3 : snd_post = 0 : GOSUB play_tone
85 END
86
87 ' sound_cursor: cursor moved one position (300 Hz).
88 sound_cursor: PROCEDURE

```



```

89  #snd_val = 373 : snd_gate = 2 : snd_post = 0 : GOSUB play_tone
90  END
91
92  ' sound_tick: countdown clock tick during your turn (short 300 Hz click).
93  sound_tick: PROCEDURE
94  #snd_val = 373 : snd_gate = 1 : snd_post = 0 : GOSUB play_tone
95  END
96
97  ' sound_hold: a die's re-roll flag was toggled (short click, 600 Hz).
98  sound_hold: PROCEDURE
99  #snd_val = 187 : snd_gate = 2 : snd_post = 0 : GOSUB play_tone
100  END
101
102  ' sound_roll: the dice were rolled (rattling triplet: 700,600,700 Hz).
103  sound_roll: PROCEDURE
104  #snd_val = 160 : snd_gate = 2 : snd_post = 1 : GOSUB play_tone
105  #snd_val = 187 : snd_gate = 2 : snd_post = 1 : GOSUB play_tone
106  #snd_val = 160 : snd_gate = 2 : snd_post = 4 : GOSUB play_tone
107  END
108
109  ' sound_score: a category was scored (rising chirp, 350,500 Hz).
110  sound_score: PROCEDURE
111  #snd_val = 320 : snd_gate = 3 : snd_post = 0 : GOSUB play_tone
112  #snd_val = 224 : snd_gate = 4 : snd_post = 2 : GOSUB play_tone
113  END
114
115  ' sound_fujitzee: five of a kind scored (5-tone rising fanfare, brighter and
116  ' longer than sound_gamedone -- the rare, celebrated roll).
117  sound_fujitzee: PROCEDURE
118  #snd_val = 373 : snd_gate = 6 : snd_post = 0 : GOSUB play_tone
119  #snd_val = 320 : snd_gate = 6 : snd_post = 0 : GOSUB play_tone
120  #snd_val = 285 : snd_gate = 6 : snd_post = 0 : GOSUB play_tone
121  #snd_val = 260 : snd_gate = 6 : snd_post = 0 : GOSUB play_tone
122  #snd_val = 224 : snd_gate = 16 : snd_post = 0 : GOSUB play_tone
123  END
124
125  ' sound_invalid: rejected input -- all-zero re-roll mask, out-of-turn action,
126  ' unselectable category (single low buzz, 100 Hz, held).
127  sound_invalid: PROCEDURE
128  #snd_val = 1119 : snd_gate = 10 : snd_post = 5 : GOSUB play_tone
129  END
130

```

APPENDIX D

NETCAT

*The traditional closing program of every FujiNet programmer's guide: a terminal. Type any devicespec on the on-screen keyboard, press **OK**, and talk to it. It compiles, assembles, and defaults to an echo server so it demonstrates itself.*

Netcat borrows CONFIG's own on-screen keyboard — the character grid you type WiFi SSIDs on — ported here as `kbd.bas` with one change: the value display spans three rows, a 60-character tail-anchored window onto a 256-byte buffer, so a long URL wraps and then scrolls while you type it. The disc moves the highlight; the action button types the highlighted character; the bottom row offers **SPC** **DEL** **OK** **ESC**, and the keypad seconds them (0 space, **CLEAR** backspace, **ENTER** = OK).

The URL screen dials whatever you accept — a failed connection keeps your text for editing, and **ESC** restores the default. In the terminal, the action button opens the same keyboard to compose a line (**OK** sends it plus CR LF), and keypad **CLEAR** hangs up and returns to the URL screen. The terminal keeps a 200-byte shadow of its cells in scratch RAM, so it repaints intact after the keyboard has been over it; while the keyboard is open the connection simply is not polled — incoming bytes wait on the FujiNet and drain when the terminal returns.

LISTING 15 `netcat.bas` — a line-mode network terminal

```

1 ' netcat.bas -- a line-mode network terminal in IntyBASIC. Type any N:
2 ' devicespec on the on-screen keyboard (the same character grid FujiNet
3 ' CONFIG uses for WiFi SSIDs), connect, and everything the far end sends
4 ' scrolls up rows 0-9 of the screen. The action button opens the keyboard
5 ' again to compose a line; OK sends it (plus CR LF). The default URL is
6 ' tcpbin.com's echo service, so an untouched OK at the URL screen gives a
7 ' self-test needing no server of your own.
8
9 ' URL screen:      disc + action button  pick characters on the grid
10 '                OK (or keypad ENTER)  connect
11 '                ESC                    restore the default URL
12 '                keypad 0 / CLEAR       space / backspace
13 '
14 '                The URL buffer holds 256 bytes; rows 0-2 are a 60-character
15 '                window that scrolls once a long URL outgrows them.
16 ' Terminal:      action button          open the keyboard to compose a line
17 '                (OK sends + CR LF, ESC cancels)
18 '                keypad CLEAR           hang up, back to the URL screen
19 '
20 ' While the keyboard is open the connection is not polled; incoming bytes
21 ' simply wait on the FujiNet and are drained when the terminal returns.
22 '
23 ' Build:  intybasic netcat.bas netcat.asm && as1600 -o netcat netcat.asm
24 '       GOTO main

```

```

25 INCLUDE "fujinet.bas"
26 INCLUDE "kbd.bas"
27
28
29 CONST TERM_CELLS = 200 ' rows 0-9 are the terminal
30 CONST STATUS_ROW = 220 ' row 11: status + key hints
31
32 ' Scratch RAM, ours, above fujinet.bas's buffers (which end at $917F).
33 CONST SC_URL = $9200 ' devicespec, 256 bytes (255 chars + NUL)
34 CONST SC_LINE = $9300 ' composed line, 253 bytes (252 + NUL)
35 CONST SC_TERM = $9400 ' 200-byte shadow of the terminal cells
36
37 ' The default devicespec (24 bytes): "N:TCP://TCPBIN.COM:4242/"
38 lit_spec:
39 DATA 78,58,84,67,80,58,47,47,84,67,80,66,73,78
40 DATA 46,67,79,77,58,52,50,52,50,47
41 CONST LEN_SPEC = 24
42
43 DIM term_pos, nc_i, nc_c, nc_cr, nc_row
44
45
-----
46 ' term_clear_row: blank the terminal row term_pos sits in, screen and
47 ' shadow both -- called on entering a fresh row so wrapped-around output
48 ' never interleaves with a stale line.
49
-----
50 term_clear_row: PROCEDURE
51 nc_row = (term_pos / 20) * 20
52 FOR nc_i = 0 TO 19
53 #BACKTAB(nc_row + nc_i) = CS_BLACK
54 POKE (SC_TERM + nc_row + nc_i), 32
55 NEXT nc_i
56 END
57
58
-----
59 ' term_putc: draw ASCII nc_c at the terminal cursor, handling CR/LF and
60 ' wrap-around, mirroring every cell into SC_TERM so the display can be
61 ' repainted after the keyboard has been over it. GROM cards 0-94 cover
62 ' ASCII 32-126 directly.
63
-----
64 term_putc: PROCEDURE
65 IF nc_c = 13 THEN nc_cr = 1 : GOSUB term_newline : RETURN
66 IF nc_c = 10 THEN
67 ' collapse the LF of a CR LF pair; a bare LF is a newline
68 IF nc_cr = 0 THEN GOSUB term_newline
69 nc_cr = 0

```

```

70 RETURN
71 END IF
72 nc_cr = 0
73 IF nc_c < 32 OR nc_c > 126 THEN RETURN
74 #BACKTAB(term_pos) = (nc_c - 32) * 8 + COL_NORMAL
75 POKE (SC_TERM + term_pos), nc_c
76 term_pos = term_pos + 1
77 IF term_pos >= TERM_CELLS THEN term_pos = 0
78 IF (term_pos % 20) = 0 THEN GOSUB term_clear_row
79 END
80
81 term_newline: PROCEDURE
82 term_pos = (term_pos / 20) * 20 + 20
83 IF term_pos >= TERM_CELLS THEN term_pos = 0
84 GOSUB term_clear_row
85 END
86
87
-----
88 ' term_init / term_repaint: reset the pane, or redraw all 200 cells from
89 ' the shadow after the keyboard borrowed the screen.
90
-----
91 term_init: PROCEDURE
92 term_pos = 0
93 nc_cr = 0
94 FOR nc_i = 0 TO TERM_CELLS - 1
95 POKE (SC_TERM + nc_i), 32
96 NEXT nc_i
97 END
98
99 term_repaint: PROCEDURE
100 FOR nc_i = 0 TO TERM_CELLS - 1
101 nc_c = PEEK(SC_TERM + nc_i) AND 255
102 IF nc_c < 32 OR nc_c > 126 THEN nc_c = 32
103 #BACKTAB(nc_i) = (nc_c - 32) * 8 + COL_NORMAL
104 NEXT nc_i
105 END
106
107
-----
108 ' seed_url: (re)load the default devicespec into SC_URL.
109
-----
110 seed_url: PROCEDURE
111 FOR nc_i = 0 TO LEN_SPEC - 1
112 POKE (SC_URL + nc_i), PEEK(VARPTR lit_spec(0) + nc_i) AND 255
113 NEXT nc_i
114 POKE (SC_URL + LEN_SPEC), 0

```

```

115 END
116
117 '
-----
118 ' url_screen: full-screen URL editor on the character grid. Returns with
119 ' fn_ok = 1 and the accepted devicespec NUL-terminated in SC_URL. ESC
120 ' restores the default and keeps editing (there is nothing to cancel to).
121 '
-----
122 url_screen: PROCEDURE
123 us_again:
124   CLS
125   PRINT AT STATUS_ROW COLOR COL_DIM, "TYPE URL - OK DIALS "
126   #ge_dst = SC_URL
127   #g_max = 256
128   GOSUB grid_entry
129   IF fn_ok = 0 THEN
130     GOSUB seed_url
131     GOTO us_again
132   END IF
133   IF g_len = 0 THEN
134     GOSUB seed_url
135     GOTO us_again
136   END IF
137 END
138
139 '
-----
140 ' compose_line: the same grid over the terminal screen. On OK, sends the
141 ' line plus CR LF out the open channel. Restores the terminal afterward.
142 '
-----
143 compose_line: PROCEDURE
144   CLS
145   PRINT AT STATUS_ROW COLOR COL_DIM, "OK SENDS - ESC BACK "
146   POKE SC_LINE, 0 ' fresh line every time
147   #ge_dst = SC_LINE
148   #g_max = 253
149   GOSUB grid_entry
150
151   IF fn_ok THEN
152     #fn_txlen = 0
153     #fn_src = SC_LINE : ls_max = 253 : GOSUB fn_strlen : GOSUB
fn_putstr
154     POKE (FN_TX + #fn_txlen), 13
155     POKE (FN_TX + #fn_txlen + 1), 10
156     fn_len = #fn_txlen + 2
157     GOSUB net_write
158   END IF

```

```

159
160   CLS
161   GOSUB term_repaint
162   PRINT AT STATUS_ROW COLOR COL_DIM, "BTN TYPE - CLR URL "
163 END
164
165 main:
166   MODE 0, 0, 0, 0, 0 : WAIT
167   CLS
168   PRINT AT 0 COLOR COL_NORMAL, "FUJINET NETCAT"
169   PRINT AT 40, "CONNECTING TO FUJINET"
170   GOSUB fn_wait_mailbox
171   IF fn_ok = 0 THEN
172     PRINT AT 40, "NO CARTRIDGE MAILBOX "
173     GOTO halt
174   END IF
175   GOSUB seed_url
176
177 dial:
178   GOSUB url_screen
179
180   ' Open the accepted devicespec: read-write, no translation (we handle
181   ' CR LF ourselves).
182   CLS
183   PRINT AT 0 COLOR COL_NORMAL, "DIALING..."
184   #fn_txlen = 0
185   #fn_src = SC_URL : ls_max = 255 : GOSUB fn_strlen : GOSUB fn_putstr
186   mb_dev = NET_DEVICEID
187   mb_cmd = NETCMD_OPEN
188   mb_nparam = 2
189   pm_i = 0 : pm_size = 1 : #pm_val = OPEN_MODE_RW : GOSUB fn_param
190   pm_i = 1 : pm_size = 1 : #pm_val = OPEN_TRANS_NONE : GOSUB fn_param
191   GOSUB fn_transact
192   IF fn_ok = 0 THEN
193     PRINT AT 40 COLOR COL_ERROR, "CONNECT FAILED"
194     PRINT AT 60 COLOR COL_DIM, "PRESS BUTTON TO EDIT"
195   con_wait:
196     WAIT
197     GOSUB in_poll
198     IF in_btn = 0 THEN GOTO con_wait
199     GOTO dial ' SC_URL still holds the typo -- fix it
200   END IF
201
202   CLS
203   GOSUB term_init
204   GOSUB term_clear_row
205   PRINT AT STATUS_ROW COLOR COL_DIM, "BTN TYPE - CLR URL "
206
207 term_loop:

```

```

208 WAIT
209
210 ' --- receive: anything waiting? read up to 64 bytes and print it ---
211 GOSUB net_status
212 IF fn_ok = 0 THEN
213     PRINT AT STATUS_ROW COLOR COL_ERROR, "CONNECTION LOST"
214 lost_wait:
215     WAIT
216     GOSUB in_poll
217     IF in_btn = 0 THEN GOTO lost_wait
218     GOSUB net_close 'free the unit regardless
219     GOTO dial
220 END IF
221 IF #net_avail > 0 THEN
222     #net_readlen = #net_avail
223     IF #net_readlen > 64 THEN #net_readlen = 64
224     GOSUB net_read
225     IF fn_ok THEN
226         FOR nc_i = 0 TO #net_gotlen - 1
227             nc_c = PEEK(FN_RX + nc_i) AND 255
228             GOSUB term_putc
229         NEXT nc_i
230     END IF
231 END IF
232
233 ' --- input ---
234 GOSUB in_poll
235 IF in_btn THEN GOSUB compose_line
236 IF in_key = KEYPAD_CLEAR THEN
237     GOSUB net_close
238     GOTO dial
239 END IF
240 GOTO term_loop
241
242 halt:
243 WAIT
244 GOTO halt
245

```

LISTING 16 kbd.bas — the on-screen keyboard, from CONFIG

```

1 ' kbd.bas -- edge-detected input + on-screen character-grid keyboard,
2 ' ported from fujinet-config/intv (input.bas + the scr_recolor helper from
3 ' screen.bas), with one change: the value display spans THREE rows (0-2)
4 ' instead of one, a 60-character tail-anchored window onto a buffer of up
5 ' to 256 bytes -- long N: URLs wrap across the rows and scroll once they
6 ' outgrow them.

```

```

7 '
8 ' All 95 printable characters (32-126) fit in 6 rows x 16 columns, so the
9 ' cursor position IS the character (ch = 32 + gy*16 + gx) -- no SHIFT/SYM
10 ' paging. A row of action buttons (SPC/DEL/OK/ESC) sits below the grid.
11 ' The keypad works too: 0 = space, CLEAR = backspace, ENTER = accept.
12 '

```

```

13 ' grid_entry contract (same as fujinet-config's): caller sets #ge_dst
14 ' (destination buffer) and #g_max (its size, including the NUL) BEFORE
15 ' calling, and primes the buffer -- NUL at offset 0 for a fresh field, or
16 ' existing NUL-terminated text to edit (it is preserved and editable).
17 ' #ge_dst is mutated live REGARDLESS of accept or cancel. Returns fn_ok =
18 ' 1 (OK button or keypad ENTER) or 0 (ESC button only); g_len holds the
19 ' final length and #ge_dst is NUL-terminated there. g_len is an 8-bit
20 ' variable, so the ceiling is #g_max = 256 (255 characters + NUL) -- one
21 ' character shy of what the mailbox TX window could carry to an OPEN, and
22 ' #g_max must be the 16-bit variable it is: as an 8-bit variable, 256
23 ' would silently wrap to 0 and disable grid_append's overflow guard.
24 '
25 ' Screen layout used by grid_entry (rows 9 and 11 are never touched, so
26 ' the caller can keep hints there):
27 '   rows 0-2   value window (60 cells, tail-anchored, green cursor block)
28 '   rows 3-8   the character grid
29 '   row 10      SPC / DEL / OK / ESC
30 '
31 ' Color-stack mode foreground colors.
32 CONST CS_BLACK      = 0
33 CONST CS_BLUE       = 1
34 CONST CS_RED        = 2
35 CONST CS_TAN        = 3
36 CONST CS_DARKGREEN  = 4
37 CONST CS_GREEN      = 5
38 CONST CS_YELLOW     = 6
39 CONST CS_WHITE      = 7
40
41 CONST COL_NORMAL    = CS_WHITE
42 CONST COL_HILIGHT   = CS_YELLOW
43 CONST COL_DIM       = CS_BLUE
44 CONST COL_ERROR     = CS_RED
45 CONST COL_VALUE     = CS_TAN
46 CONST COL_CURSOR   = CS_GREEN
47
48 CONST SCREEN_COLS = 20
49 DEF FN screenpos(aColumn, aRow) = ((aRow)*SCREEN_COLS)+(aColumn))
50
51 ' Disc directions (as reported by in_poll).
52 CONST DISC_UP      = $0004
53 CONST DISC_RIGHT   = $0002
54 CONST DISC_DOWN    = $0001
55 CONST DISC_LEFT    = $0008
56
57 ' Keypad, as decoded by CONT.KEY.
58 CONST KEYPAD_0     = 0
59 CONST KEYPAD_CLEAR = 10
60 CONST KEYPAD_ENTER = 11
61 CONST KEYPAD_NONE  = 12

```

```

62
63 ' Grid geometry. Value rows 0-2; charset rows 3-8; actions row 10.
64 CONST VAL_ROW0     = 0
65 CONST VAL_ROWS     = 3
66 CONST VAL_CELLS    = 60 ' VAL_ROWS * SCREEN_COLS
67 CONST GRID_ROW0    = 3
68 CONST GRID_COL0    = 2
69 CONST GRID_ROWS    = 6
70 CONST GRID_COLS    = 16
71 CONST GRID_ACTION_ROW = 10
72 CONST GRID_ACT_COL0 = 2 ' SPC
73 CONST GRID_ACT_COL1 = 7 ' DEL
74 CONST GRID_ACT_COL2 = 12 ' OK
75 CONST GRID_ACT_COL3 = 17 ' ESC
76
77 CONST IN_REPEAT_DELAY = 18 ' frames held before auto-repeat (~0.3s)
78 CONST IN_REPEAT_RATE = 6 ' frames between repeats (~0.1s)
79
80 DIM in_disc, in_pdisc, in_rdelay
81 DIM in_braw, in_btn, in_pbtn
82 DIM in_key, in_pkey
83
84 '
85 ' in_poll: call once per frame (after WAIT). Sets, per call:
86 '   in_disc - DISC_UP/DOWN/LEFT/RIGHT on a fresh press or an auto-repeat
87 '             tick while held; 0 otherwise.
88 '   in_btn  - 1 on a fresh action-button press (any of B0/B1/B2).
89 '   in_key  - the decoded keypad value on a fresh press, else KEYPAD_NONE.
90 '   Uses the unqualified CONT.* pseudo-variables (both controllers OR'd).
91 '
92 in_poll: PROCEDURE
93   in_disc = 0
94   IF CONT.UP THEN in_disc = DISC_UP
95   IF CONT.DOWN THEN in_disc = DISC_DOWN
96   IF CONT.LEFT THEN in_disc = DISC_LEFT
97   IF CONT.RIGHT THEN in_disc = DISC_RIGHT
98
99   IF in_disc <> 0 THEN
100     IF in_disc <> in_pdisc THEN
101       in_rdelay = IN_REPEAT_DELAY
102     ELSE
103       IF in_rdelay > 0 THEN
104         in_rdelay = in_rdelay - 1
105         in_disc = 0
106       ELSE
107         in_rdelay = IN_REPEAT_RATE
108     END IF

```

```

109     END IF
110 END IF
111 in_pdisc = 0
112 IF CONT.UP THEN in_pdisc = DISC_UP
113 IF CONT.DOWN THEN in_pdisc = DISC_DOWN
114 IF CONT.LEFT THEN in_pdisc = DISC_LEFT
115 IF CONT.RIGHT THEN in_pdisc = DISC_RIGHT
116
117 in_braw = 0
118 IF CONT.B0 OR CONT.B1 OR CONT.B2 THEN in_braw = 1
119 IF in_braw <> 0 AND in_pbtn = 0 THEN
120     in_btn = 1
121 ELSE
122     in_btn = 0
123 END IF
124 in_pbtn = in_braw
125
126 in_key = KEYPAD_NONE
127 IF CONT.KEY <> KEYPAD_NONE AND CONT.KEY <> in_pkey THEN
128     in_key = CONT.KEY
129 END IF
130 in_pkey = CONT.KEY
131 END
132 '
133 '-----
134 ' grid_entry -- see the header comment for the contract.
135 '-----
136 DIM g_x, g_y, g_px, g_py, g_len, g_ch, ga_idx
137 DIM #ge_dst, #g_max
138 DIM k_row, k_col, k_i, k_c, k_max, k_color
139 DIM #k_word
140
141 grid_entry: PROCEDURE
142     GOSUB grid_draw_charset
143     GOSUB grid_draw_actions
144
145     g_len = 0
146     WHILE (g_len < #g_max - 1) AND ((PEEK(#ge_dst + g_len) AND 255) <> 0)
147         g_len = g_len + 1
148     WEND
149     GOSUB grid_draw_value
150
151     g_x = 0 : g_y = 0
152     g_px = 255 : g_py = 255
153
154     DO WHILE 1
155         WAIT

```

```

156 GOSUB grid_draw_cursor
157 GOSUB in_poll
158
159 IF in_disc = DISC_UP AND g_y > 0 THEN g_y = g_y - 1
160 IF in_disc = DISC_DOWN AND g_y < GRID_ROWS THEN g_y = g_y + 1
161 IF in_disc = DISC_LEFT AND g_x > 0 THEN g_x = g_x - 1
162 IF in_disc = DISC_RIGHT THEN
163     IF g_y = GRID_ROWS THEN
164         IF g_x < 3 THEN g_x = g_x + 1
165     ELSE
166         IF g_x < GRID_COLS - 1 THEN g_x = g_x + 1
167     END IF
168 END IF
169 IF g_y = GRID_ROWS AND g_x > 3 THEN g_x = 3
170
171 IF in_key = KEYPAD_0 THEN g_ch = 32 : GOSUB grid_append
172 IF in_key = KEYPAD_CLEAR THEN GOSUB grid_backspace
173 IF in_key = KEYPAD_ENTER THEN
174     fn_ok = 1
175     EXIT DO
176 END IF
177
178 IF in_btn <> 0 THEN
179     IF g_y < GRID_ROWS THEN
180         g_ch = 32 + g_y * GRID_COLS + g_x
181         IF g_ch <= 126 THEN GOSUB grid_append
182     ELSE
183         IF g_x = 0 THEN g_ch = 32 : GOSUB grid_append
184         IF g_x = 1 THEN GOSUB grid_backspace
185         IF g_x = 2 THEN
186             fn_ok = 1
187             EXIT DO
188         END IF
189         IF g_x = 3 THEN
190             fn_ok = 0
191             EXIT DO
192         END IF
193     END IF
194 END IF
195 LOOP
196 END
197
198 ' grid_draw_charset: paints all 96 cells (95 real chars + one always-
199 blank).
200 grid_draw_charset: PROCEDURE
201     FOR g_y = 0 TO GRID_ROWS - 1
202         FOR g_x = 0 TO GRID_COLS - 1
203             g_ch = 32 + g_y * GRID_COLS + g_x
204             IF g_ch > 126 THEN g_ch = 32

```

```

204      #BACKTAB((GRID_ROW0 + g_y) * SCREEN_COLS + GRID_COL0 + g_x) =
(g_ch - 32) * 8 + COL_VALUE
205      NEXT g_x
206      NEXT g_y
207  END
208
209  grid_draw_actions: PROCEDURE
210      PRINT AT screenpos(GRID_ACT_COL0, GRID_ACTION_ROW) COLOR COL_DIM,"SPC"
211      PRINT AT screenpos(GRID_ACT_COL1, GRID_ACTION_ROW) COLOR COL_DIM,"DEL"
212      PRINT AT screenpos(GRID_ACT_COL2, GRID_ACTION_ROW) COLOR COL_DIM," OK"
213      PRINT AT screenpos(GRID_ACT_COL3, GRID_ACTION_ROW) COLOR COL_DIM,"ESC"
214  END
215
216  ' grid_recolor: change only the COLOR of k_max+1 already-drawn cells on
row
217  ' k_row starting at column k_col -- the glyphs underneath stay. (The
218  ' scr_recolor helper from fujinet-config's screen.bas, renamed so a
program
219  ' including this file can keep its own s_* variables.)
220  grid_recolor: PROCEDURE
221      FOR k_i = 0 TO k_max
222          #k_word = (#BACKTAB(k_row * SCREEN_COLS + k_col + k_i) AND $FFF8)
+ k_color
223          #BACKTAB(k_row * SCREEN_COLS + k_col + k_i) = #k_word
224      NEXT k_i
225  END
226
227  ' grid_draw_cursor: un-highlight the previous cell (g_px/g_py), highlight
228  ' the current one (g_x/g_y). g_px=255 on the very first call skips the
229  ' un-highlight.
230  grid_draw_cursor: PROCEDURE
231      IF g_px <= 255 THEN
232          IF g_py = GRID_ROWS THEN
233              ga_idx = g_px : GOSUB grid_action_col
234              k_row = GRID_ACTION_ROW : k_max = 2 : k_color = COL_DIM
235              GOSUB grid_recolor
236          ELSE
237              g_ch = 32 + g_py * GRID_COLS + g_px
238              IF g_ch > 126 THEN g_ch = 32
239              #BACKTAB((GRID_ROW0 + g_py) * SCREEN_COLS + GRID_COL0 + g_px)
= (g_ch - 32) * 8 + COL_VALUE
240          END IF
241      END IF
242
243      IF g_y = GRID_ROWS THEN
244          ga_idx = g_x : GOSUB grid_action_col
245          k_row = GRID_ACTION_ROW : k_max = 2 : k_color = COL_HIGHLIGHT
246          GOSUB grid_recolor
247      ELSE

```

```

248          g_ch = 32 + g_y * GRID_COLS + g_x
249          IF g_ch > 126 THEN g_ch = 32
250          #BACKTAB((GRID_ROW0 + g_y) * SCREEN_COLS + GRID_COL0 + g_x) =
(g_ch - 32) * 8 + COL_HIGHLIGHT
251      END IF
252
253      g_px = g_x : g_py = g_y
254  END
255
256  ' grid_action_col: given ga_idx (0-3), sets k_col to that action button's
257  ' starting column.
258  grid_action_col: PROCEDURE
259      IF ga_idx = 0 THEN k_col = GRID_ACT_COL0
260      IF ga_idx = 1 THEN k_col = GRID_ACT_COL1
261      IF ga_idx = 2 THEN k_col = GRID_ACT_COL2
262      IF ga_idx = 3 THEN k_col = GRID_ACT_COL3
263  END
264
265  ' grid_draw_value: tail-anchored 60-cell window (rows 0-2) onto #ge_dst,
266  ' with a trailing cursor block. A value longer than 59 characters scrolls:
267  ' the window always shows the tail, where typing is happening.
268  grid_draw_value: PROCEDURE
269      k_i = 0
270      IF g_len > VAL_CELLS - 1 THEN k_i = g_len - (VAL_CELLS - 1)
271      FOR k_col = 0 TO VAL_CELLS - 1
272          k_c = 32
273          IF k_i + k_col < g_len THEN k_c = PEEK(#ge_dst + k_i + k_col) AND
255          IF k_c < 32 OR k_c > 126 THEN k_c = 32
275          #BACKTAB(VAL_ROW0 * SCREEN_COLS + k_col) = (k_c - 32) * 8 +
COL_VALUE
276          NEXT k_col
277      IF g_len - k_i < VAL_CELLS THEN
278          #BACKTAB(VAL_ROW0 * SCREEN_COLS + (g_len - k_i)) = (95 - 32) * 8 +
COL_CURSOR
279      END IF
280  END
281
282  grid_append: PROCEDURE
283      IF g_len >= #g_max - 1 THEN RETURN
284      POKE (#ge_dst + g_len), g_ch
285      g_len = g_len + 1
286      POKE (#ge_dst + g_len), 0
287      GOSUB grid_draw_value
288  END
289
290  grid_backspace: PROCEDURE
291      IF g_len = 0 THEN RETURN
292      g_len = g_len - 1

```


APPENDIX D

```
293 POKE (#ge_dst + g_len), 0
294 GOSUB grid_draw_value
295 END
296
```

Now go put an Intellivision on the Internet.